

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
Учреждения образования «БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет информационных технологий
Кафедра программной инженерии
Специальность 6-05-0611-01 Информационные системы и технологии
Направление специальности 6-05-0611-01 Информационные системы

**ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
КУРСОВОГО ПРОЕКТА:**

по дисциплине «Объектно-ориентированное проектирование и
программирование»
Тема: Программное средство «Компьютерный клуб»

Исполнитель
студент 3 курса группы 1 Васильев Владислав Васильевич
(Ф.И.О.)

Руководитель работы ст. препод. Сухорукова И. Г.
(учен. степень, звание, должность, подпись, Ф.И.О.)

Курсовой проект защищен с оценкой _____

Председатель _____ Мущук А. Н.
(подпись)

Содержание

Введение.....	5
1 Постановка задачи и обзор аналогичных решений	6
1.1 Аналитический обзор аналогов	6
1.1.1 Аналог «Cyberspace».....	6
1.1.2 Аналог «Pudge»	7
1.1.3 Аналог «Colizeum».....	7
1.1.4 Аналог «Oneshot».....	8
1.1.5 Аналог «Alliance».....	9
1.2 Разработка функциональных требований	10
2 Проектирование архитектуры проекта.....	11
2.1 Диаграммы UML, взаимосвязь всех компонентов	11
2.1.1 Определение вариантов использования для роли «Гость».....	11
2.1.2 Определение вариантов использования для роли «Клиент»	12
2.1.3 Определение вариантов использования для роли «Администратор».....	12
2.2 Логическая схема базы данных	13
2.3 Описание информационных объектов и ограничений целостности	14
2.4 Реализация модели данных.....	16
2.5 Диаграмма развертывания приложения	19
2.6 Визуализация алгоритмов	20
2.6.1 Блок-схема процесса оформления заказа компьютера	20
2.6.2 Диаграмма последовательности авторизации	21
2.7 Диаграмма классов	22
2.8 Вывод по разделу	23
3 Разработка функциональной модели и модели данных ПС.....	24
3.1 Основные классы программного средства.....	24
3.2 Описание классов и методов программного средства	24
3.2.1 Регистрация	24
3.2.2 Авторизация	25
3.2.3 Просмотр компьютеров	25
3.2.4 Просмотр игр и запуск игр	25
3.2.5 Возможность поменять язык и тему приложения.....	25
3.2.6 Бронирование компьютеров	26
3.2.7 Использование личного кабинета	26
3.2.8 Составление отзывов	26
3.2.9 Управление проектом	26
3.3 Вывод по разделу	26
4 Тестирование	28
4.1 Тестирование регистрации.....	28
4.2 Тестирование авторизации.....	30
4.3 Тестирование бронирования.....	31
4.4 Тестирование поиска компьютеров и игр	34
4.5 Тестирование личного кабинета	34
4.6 Тестирование управления данными проекта	35

4.7 Вывод по разделу	37
Заключение	38
Список использованных источников.....	39
ПРИЛОЖЕНИЕ А. Полный код программного средства	40

Введение

В современном мире компьютерные клубы являются популярными центрами развлечений, работы и киберспортивных мероприятий. Однако многие посетители сталкиваются с проблемами, такими как отсутствие возможности забронировать желаемый компьютер или игровую зону. Кроме того, геймеры часто испытывают трудности с доступом к специализированному оборудованию, например, мощным игровым ПК или гарнитурам, поскольку не всегда могут проверить их наличие.

Данное программное решение незаменимо для любителей компьютерных игр и киберспорта, так как благодаря интуитивно понятному интерфейсу позволяет самому в клубе выбрать подходящий компьютер, ознакомиться с его характеристиками и доступным софтом, а также забронировать его на нужное время, не обращаясь к администратору.

Пользователь может создать учетную запись или войти в существующую. Это дает ему возможность: просматривать список доступных компьютеров с детальными характеристиками (процессор, видеокарта, ОЗУ, установленные игры), забронировать компьютер и выбирать удобное время.

Неавторизованный пользователь может: просматривать компьютеры доступные для бронирования и игры/приложения.

Авторизованный пользователь может: бронировать компьютеры на определенное время, просматривать компьютеры доступные для бронирования и игры/приложения, а также, если сеанс активный, играть или пользоваться приложениями, оставлять отзывы и оценки после использования компьютера.

Администратор может: управлять базой компьютеров, пользователей, заказов, приложений и отзывов, а также имеет те же возможности, что и авторизованный пользователь.

Главная задача данного курсового проекта – разработка программного обеспечения, которое объединит функционал для клиентов и администраторов компьютерного клуба. В качестве инструментов используются: язык программирования C#, База данных MS SQL Server для хранения информации, и технология WPF (Windows Presentation Foundation).

1 Постановка задачи и обзор аналогичных решений

1.1 Аналитический обзор аналогов

В ходе разработки программного обеспечения для компьютерного клуба был проведен детальный анализ целей и задач проекта. Также изучены существующие аналогичные решения, используемые в индустрии. На основе выявленных преимуществ и недостатков этих систем сформирован перечень ключевых требований к разрабатываемому программному средству.

1.1.1 Аналог «Cyberspace»

CyberSpace [1] представляет собой современный киберспортивно-развлекательный комплекс, сочетающий игровые зоны с лаунж-пространством. Клуб предлагает подробную информацию о характеристиках ПК (процессоры, видеокарты, мониторы), списке установленных игр и описании VIP-зон, однако существенно уступает в цифровизации услуг.

CyberSpace не предоставляет полноценной системы оценок оборудования и сервиса, а также не имеет персонализированных профилей с историей посещений. Для повышения конкурентоспособности клубу необходимо внедрить цифровую платформу с функционалом онлайн-бронирования. Интерфейс интернет-ресурса представлен на рисунке 1.1.

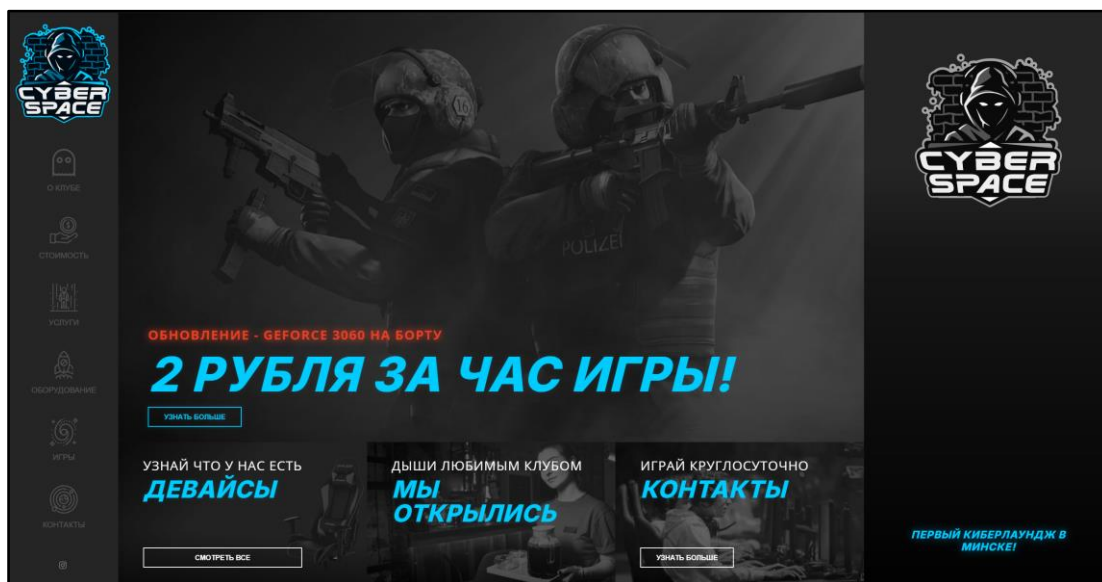


Рисунок 1.1 – Интернет-ресурс «CyberSpace»

Главный недостаток - отсутствие системы онлайн-бронирования, из-за чего клиенты вынуждены звонить для резервации мест, не имея возможности заранее выбрать конкретный компьютер через сайт.

1.1.2 Аналог «Pudge»

Pudge Club [2] — это игровой клуб с атмосферой уютного геймерского андеграунда, где стоят 32 компьютера разного уровня: от топовых VIP-станций для киберспортивных баталий до консолей PS5 для любителей комфортного геймплея. Однако его сайт — это настоящий цифровой артефакт эпохи интернета. Кривой дизайн, пёстрые шрифты, мигающие кнопки и полное отсутствие полезной информации. Здесь нет ни списка игр, ни фотографий залов, ни даже примерных цен — только телефон и адрес, затерянные среди хаотичного нагромождения элементов.

Хочешь узнать, какие ПК свободны или во что можно поиграть? Придётся звонить или идти вслепую, как в старые добрые времена, когда интернет-бронирование было чем-то из разряда фантастики. Ни отзывов, ни онлайн-записи, ни даже намёка на то, что заведение существует в 2020-х.



Рисунок 1.2 – Интернет-ресурс «Pubge»

Клубу срочно нужен апгрейд: онлайн-бронирование, прозрачность информации и хотя бы минимальный современный дизайн, чтобы не отпугивать новых геймеров.

Современный игровой клуб — это не только железо, но и удобство для посетителей. Pudge Club мог бы стать местом притяжения для киберспортивных команд, стримеров и просто любителей поиграть в хорошей компании, но для этого ему нужны: честный прайс-лист, онлайн-расписание, фото и описание залов, актуальный список игр, отзывы и рейтинги.

1.1.3 Аналог «Colizeum»

Colizeum [3] – это федеральная сеть киберспортивных арен, которая активно использует цифровые технологии для повышения удобства пользователей. Ключевым элементом её инфраструктуры является мобильное приложение, предоставляющее геймерам широкий функционал: онлайн-бронирование ПК,

мониторинг занятости залов в реальном времени, а также возможность участия в киберспортивных турнирах. Приложение также включает систему лояльности с персонализированными бонусами и ведёт детализированный геймерский профиль, где сохраняется история посещений и активность пользователя. Такой подход значительно упрощает взаимодействие клиентов с клубом и повышает вовлечённость в игровой процесс. Интерфейс интернет-ресурса представлен на рисунке 1.3.

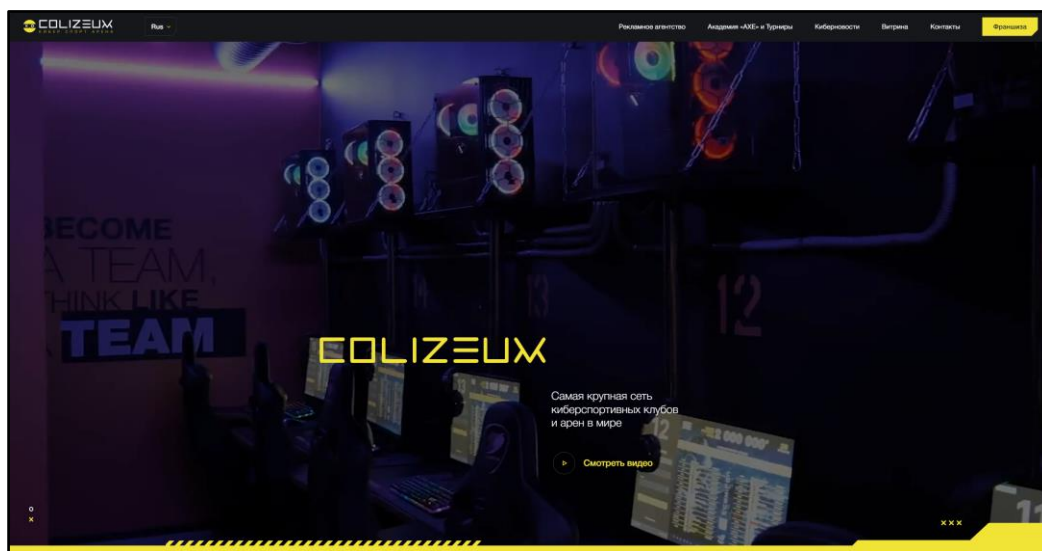


Рисунок 1.3 – Интернет-ресурс «Colizeum»

Приложение предлагает персонализированные бонусы и ведёт геймерский профиль с историей посещений, хотя имеет недостатки: неполный функционал в некоторых филиалах и отсутствие детальных характеристик ПК (например, FPS в играх). Это пример успешной цифровизации игрового клуба, достойный для подражания.

Чтобы добиться максимальной эффективности, необходимо обеспечить полную интеграцию функционала во всех филиалах, а также добавить детализированную информацию о техническом оснащении. Это позволит создать по-настоящему удобный и прозрачный сервис для геймеров.

Несмотря на впечатляющий цифровой функционал, у Colizeum есть пространство для роста. Внедрение AI-ассистента для подбора оптимального ПК под конкретные игры, интеграция стриминговых сервисов для трансляций турниров прямо из приложения, а также добавление VR-зон с возможностью бронирования могли бы вывести сервис на новый уровень.

1.1.4 Аналог «Oneshot»

Oneshot [4] — компьютерный клуб с уникальным стилем, где приложение выделяется яркими кислотно-розовыми и неоновыми-синими градиентами, что иногда затрудняет чтение текста.

Клуб предлагает детальные характеристики компьютеров — от процессоров до периферии. Из минусов: перенасыщенный дизайн утомляет глаза, а анимации могут работать нестабильно. Тем не менее, приложение остается удобным инструментом для бронирования мест без необходимости звонков или живого посещения клуба. Интерфейс интернет-ресурса представлен на рисунке 1.4.

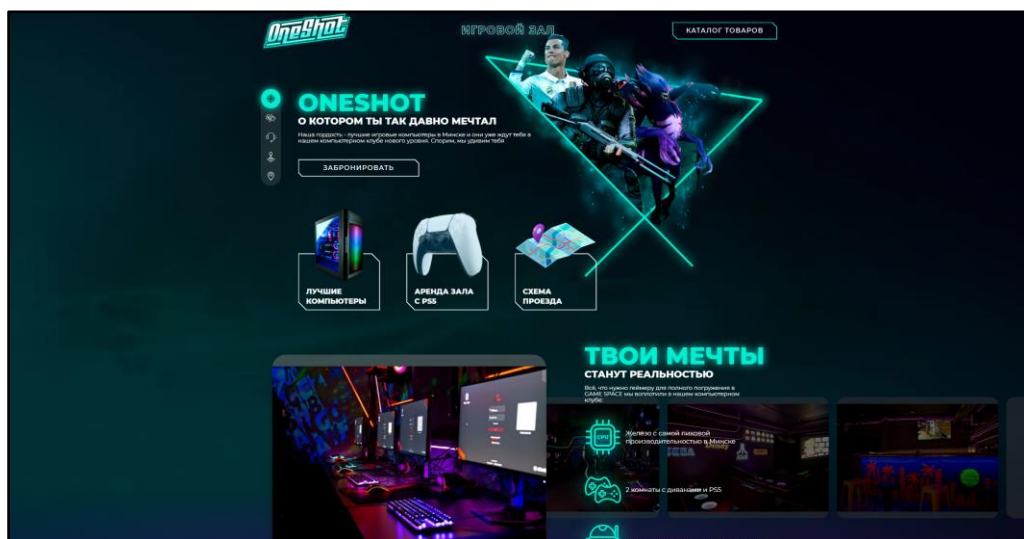


Рисунок 1.4 – Интернет-ресурс «Oneshot»

Однако функционал впечатляет: интерактивная 3D-карта бронирования позволяет выбирать ПК в виртуальном зале, а подробный список игр включает фильтры по жанрам и показатели FPS для каждой конфигурации. Одним из ключевых преимуществ клуба является полная прозрачность технических возможностей ПК. В мобильном приложении и на сайте доступен подробный список игр с фильтрами по жанрам, мультиплеерным режимам и даже показателям FPS на разных настройках графики.

Несмотря на технологическую продвинутость, клубу стоит пересмотреть визуальную составляющую — заменить кислотные градиенты на более сбалансированную палитру, сохранив при этом фирменный «киберпанковый» дух. Добавление темного режима снизило бы нагрузку на глаза, а оптимизация анимаций улучшила бы работу приложения на слабых устройствах.

1.1.5 Аналог «Alliance»

Alliance [5]— сеть компьютерных клубов с фирменным фиолетовым дизайном, предлагающая удобное приложение с полным каталогом игр (с фильтрацией по жанрам и системным требованиям) и детальными спецификациями каждого ПК — от моделей процессоров до периферии.

Однако, несмотря на богатый функционал, приложение Alliance имеет существенный недостаток — отсутствие возможности онлайн-бронирования. Интерфейс ресурса представлен на рисунке 1.5.

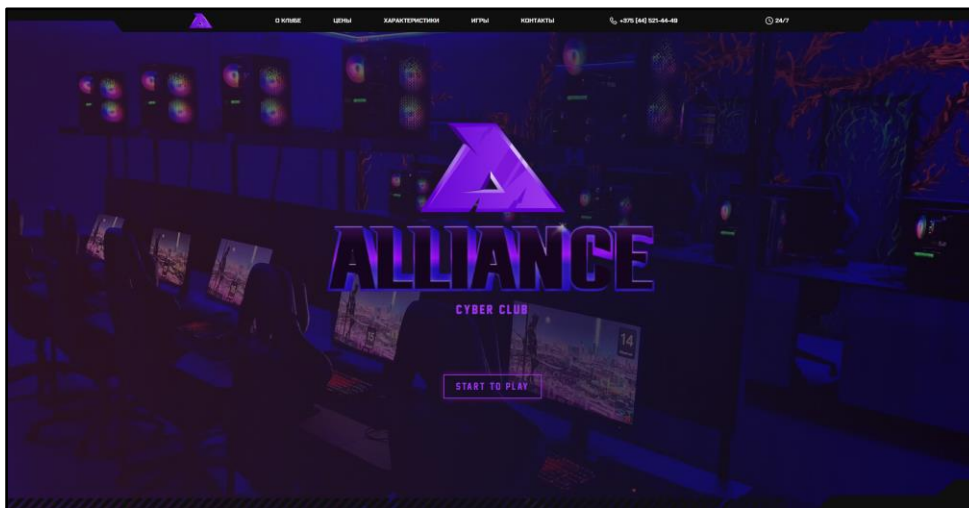


Рисунок 1.5 – Интернет-ресурс «Alliance»

Тем не менее, Alliance остаётся полезным инструментом для тех, кто ценит информационную прозрачность. Возможность заранее проверить технические характеристики ПК и список установленных игр помогает пользователям принимать осознанное решение о посещении.

1.2 Разработка функциональных требований

Проведенный анализ существующих решений выявил ключевые сильные и слабые стороны аналогичных сервисов, что позволило определить основные требования к разрабатываемому программному обеспечению.

Система должна реализовывать следующий функционал:

- управление администратором базой данных;
- возможность пользователю зарегистрироваться или войти в существующую учетную запись;
- возможность авторизованным пользователям просмотра информации о компьютерах, бронировании и играх;
- возможность просмотра личного кабинета пользователя с отображением всех заказов;
- возможность оценки компьютера после заказа;
- возможность запустить игры на выбранном компьютере.

1.3 Вывод по разделу

В данном разделе были проанализированы аналогичные решения компьютерных клубов.

Были определены задачи для реализации, включая определение основных ролей и соответствующего им функционала, разработку структуры базы данных а также программирование самого приложения.

2 Проектирование архитектуры проекта

2.1 Диаграммы UML, взаимосвязь всех компонентов

UsecCase диаграммы (диаграммы вариантов использования) – это тип диаграмм в Unified Modeling Language (UML), которые используются для моделирования взаимодействий между пользователями (акторами) и системой. Они помогают визуализировать функциональные требования системы, показывая, какие действия могут выполнять пользователи и как система реагирует на эти действия. Данный курсовой проект подразумевает собой 3 роли: гость, клиент, администратор.

2.1.1 Определение вариантов использования для роли «Гость»

На рисунке 2.1 представлена диаграмма вариантов использования для роли «Гость».



Рисунок 2.1 – Диаграмма вариантов использования для роли «Гость»

В системе предусмотрена регистрация и авторизация пользователей. Без регистрации пользователь может только просматривать доступные бронирования, смотреть перечень подключенных игр, а также взаимодействовать с интерфейсом программы.

2.1.2 Определение вариантов использования для роли «Клиент»

На рисунке 2.2 представлена диаграмма вариантов использования для роли «Клиент»

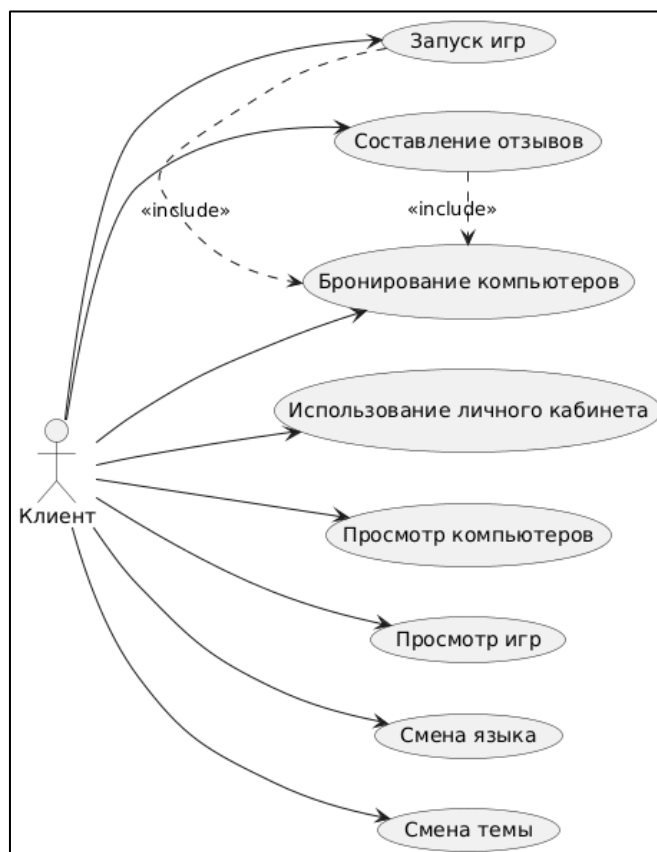


Рисунок 2.2 – Диаграмма вариантов использования для роли «Клиент»

Диаграмма описывает основной функционал, доступный после авторизации. Клиент имеет доступ к просмотру компьютеров, просмотру игр, смене языка, смене темы, использованию личного кабинета, бронированию компьютеров, запуску игр и составлению отзывов.

2.1.3 Определение вариантов использования для роли «Администратор»

На рисунке 2.3 представлена диаграмма вариантов использования для роли «Администратор».

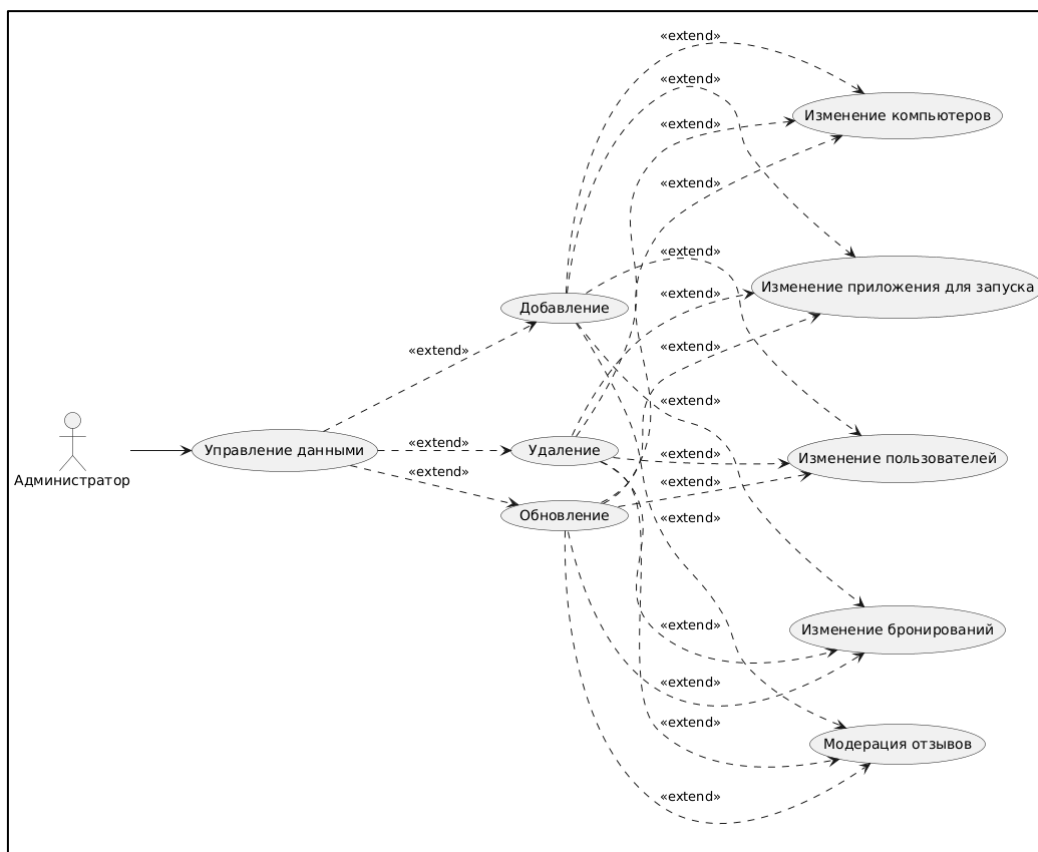


Рисунок 2.3 – Диаграмма вариантов использования для роли «Администратор»

Диаграмма отражает функции управления всей системой. Администратор взаимодействует с управлением данными, изменением компьютеров, изменением приложений для запуска, изменением отзывов и изменением пользователей

2.2 Логическая схема базы данных

Логическая схема базы данных, её ограничения целостности, связи и поля представлена на рисунке 2.4.

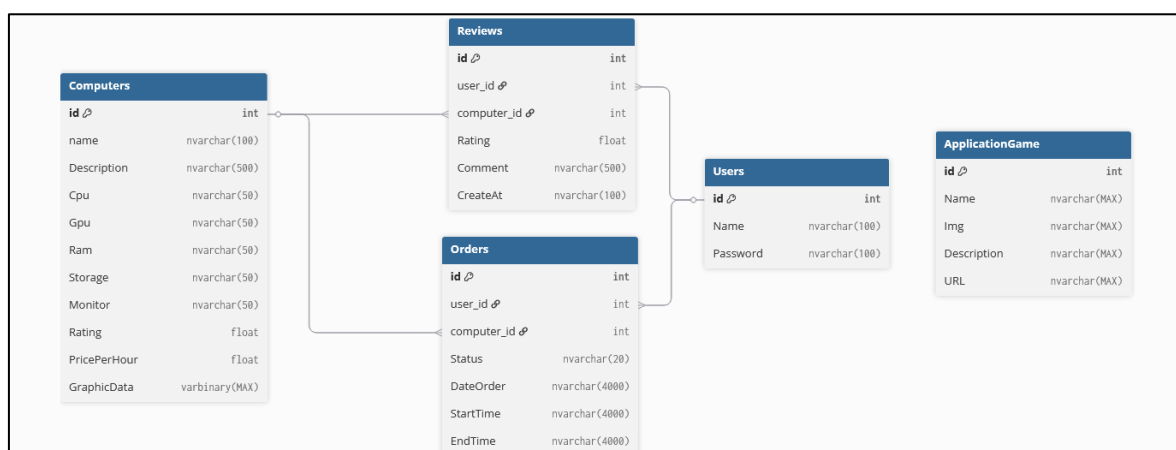


Рисунок 2.4 – Логическая схема базы данных

На данной логической схеме отображены пять таблиц, которые будут созданы

в рамках курсового проекта.

2.3 Описание информационных объектов и ограничений целостности

В таблице 2.1 описаны таблицы, которые будут созданы в базе данных, а в таблицах 2.2 – 2.6 приводится описание каждой таблицы, включая её поля и описание этих полей. В таблице 2.7 описаны связи между таблицами базы данных.

Таблица 2.1 – Сведения о таблицах базы данных

Имя таблицы	Назначение таблицы
Users	Хранение информации о пользователях системы, основные данные.
Computers	Информация о компьютерах: технические характеристики, рейтинг, стоимость в час.
Orders	Информация о бронированиях компьютеров: время, дата, пользователь, компьютер.
Reviews	Информация об отзывах компьютера
ApplicationGame	Информация о запускаемых играх: название, описание, картинка, ссылка на файл.

Здесь описано назначение таблиц, которые будут реализованы в рамках данного курсового проекта.

Таблица 2.2 – Таблица «Users»

Поле	Назначение поля
id	Уникальный идентификатор пользователя.
Name	Имя пользователя.
Password	Хэшированный пароль пользователя .

Здесь описаны поля и назначение каждого из поля в таблице «Users».

Таблица 2.3 – Таблица «Computers»

Поле	Назначение поля
1	2
id	Уникальный идентификатор компьютера.
Name	Название компьютера.
Description	Описание компьютера.
Cpu	Информация о центральном процессоре компьютера.
Gpu	Информация о графическом процессоре компьютера.

Продолжение таблицы 2.3

1	2
Ram	Объем оперативной памяти
Storage	Объем постоянной памяти.
Monitor	Информация о мониторе
Rating	Рейтинг компьютера.
PricePerHour	Цена за час пользования.
GraphicData	Изображение компьютера.

Здесь описаны поля и назначение каждого из поля в таблице «Users».

Таблица 2.4 – Таблица «Orders»

Поле	Назначение поля
id	Уникальный идентификатор заказа.
User_id	Идентификатор пользователя.
Computer_id	Идентификатор компьютера.
Status	Статус заказа.
DateOrder	Дата бронирования.
StartTime	Время начала бронирования.
EndTime	Время конца бронирования.

Здесь описаны поля и назначение каждого из поля в таблице «Orders».

Таблица 2.5 – Таблица «Reviews»

Поле	Назначение поля
id	Уникальный идентификатор отзыва.
User_id	Идентификатор пользователя.
Computer_id	Идентификатор компьютера.
Rating	Оценка компьютера.
Comment	Комментарий пользователя.
CreateAt	Дата написания отзыва.

Здесь описаны поля и назначение каждого из поля в таблице «Reviews».

Таблица 2.6 – Таблица «ApplicationGame»

Поле	Назначение поля
1	2
id	Уникальный идентификатор приложения.
Name	Название приложения.

Продолжение таблицы 2.6

1	2
Description	Описание приложения.
URL	Ссылка для запуска приложения.

Здесь описаны поля и назначение каждого из поля в таблице «ApplicationGame».

Таблица 2.7 – Сведения о связях между таблицами базы данных

Таблица PK (поле PK)	Таблица FK (поле FK)	Тип связи	Описание связи
Computers(id)	Orders(computer_id)	Один ко многим	Компьютер может быть в нескольких заказах(в разные даты и время).
Computers(id)	Reviews(computer_id)	Один ко многим	Компьютер может иметь несколько отзывов.
Users(id)	Reviews(user_id)	Один ко многим	Пользователь может оставлять несколько отзывов.
Users(id)	Orders(user_id)	Один ко многим	Пользователь может оформить несколько заказов.

Здесь описаны связи между полями в таблицах базы данных с приведением описания этой связи и его типа.

2.4 Реализация модели данных

Модель данных представлена пятью классами, описанными в пункте 2.3 данной пояснительной записки. В листингах 2.4.1 – 2.4.5 представлен код реализации данных моделей данных.

```
public class User
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
    [Column("id")]
    public int Id { get; set; }
    [Required]
    [StringLength(100)]
    [Index(IsUnique = true)]
    [Column(TypeName = "nvarchar")]
    public string Name { get; set; }
    [Required]
    [StringLength(100)]
```

```

    [Column(TypeName = "nvarchar")]
    public string Password { get; set; }
    public virtual ICollection<Order> Orders { get; set; }
}

```

Листинг 2.4.1 – Код реализации модели данных «User»

Данный код определяет собой программную реализацию класса «Users» из базы данных.

```

public class Reviews
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
    public int Id { get; set; }
    [Required]
    public int UserId { get; set; }
    [Required]
    public int ComputerId { get; set; }
    [Required]
    [Range(0.0, 5.0)]
    public double Rating { get; set; }
    [Required]
    [StringLength(500)]
    public string Comment { get; set; }
    public string CreateAt { get; set; }
    [ForeignKey("UserId")]
    public virtual User User { get; set; }
    [ForeignKey("ComputerId")]
    public virtual Computer Computer { get; set; }
}

```

Листинг 2.4.2 – Код реализации модели данных «Reviews»

Данный код определяет собой программную реализацию класса «Reviews» из базы данных.

```

public class Computer
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
    [Column("id")]
    public int Id { get; set; }
    [Required]
    [StringLength(100)]
    [Column(TypeName = "nvarchar")]
    public string Name { get; set; }
    [Required]
    [Column(TypeName = "nvarchar")]
    [MaxLength(500)]
    public string Description { get; set; }
    [Required]
}

```

```

[StringLength(50)]
[Column(TypeName = "nvarchar")]
public string Cpu { get; set; }
[Required]
[Column(TypeName = "nvarchar")]
[StringLength(50)]
public string Gpu { get; set; }
[Required]
[Column(TypeName = "nvarchar")]
[StringLength(50)]
public string Ram { get; set; }
[Required]
[Column(TypeName = "nvarchar")]
[StringLength(50)]
public string Storage { get; set; }
[Required]
[Column(TypeName = "nvarchar")]
[StringLength(50)]
public string Monitor { get; set; }
public double Rating { get; set; } = 0.0;
[Required]
public double PricePerHour { get; set; }
[MaxLength]
public byte[] GraphicData { get; set; }
public virtual ICollection<Order> Orders { get; set; }
}

```

Листинг 2.4.3 – Код реализации модели данных «Computer»

Данный код определяет собой программную реализацию класса «Computer» из базы данных.

```

public class Order
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
    [Column("id")]
    public int Id { get; set; }
    [Required]
    [Column("user_id")]
    public int UserId { get; set; }
    [Required]
    [Column("computer_id")]
    public int ComputerId { get; set; }
    [Required]
    [StringLength(20)]
    [Column(TypeName = "nvarchar")]
    public string Status { get; set; } = "active";
    [Column(TypeName = "nvarchar")]
    public string DateOrder { get; set; }
    [Required]
    [Column(TypeName = "nvarchar")]
    public string StartTime { get; set; }
}

```

```

[Column(TypeName = "nvarchar")]
public string EndTime { get; set; }
[ForeignKey("UserId")]
public virtual User User { get; set; }
[ForeignKey("ComputerId")]
public virtual Computer Computer { get; set; }
}

```

Листинг 2.4.4 – Код реализации модели данных «Order»

Данный код определяет собой программную реализацию класса «Order» из базы данных.

```

public class ApplicationGame
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
    public int Id { get; set; }
    [Required]
    public string Name { get; set; } = string.Empty;
    public string IMG { get; set; } = string.Empty;
    public string Description { get; set; } = string.Empty;
    public string URL { get; set; } = string.Empty;
}

```

Листинг 2.4.5 – Код реализации модели данных «ApplicationGame»

Данный код определяет собой программную реализацию класса «ApplicationGame» из базы данных.

2.5 Диаграмма развертывания приложения

Диаграмма развертывания приложения показана на рисунке 2.5.

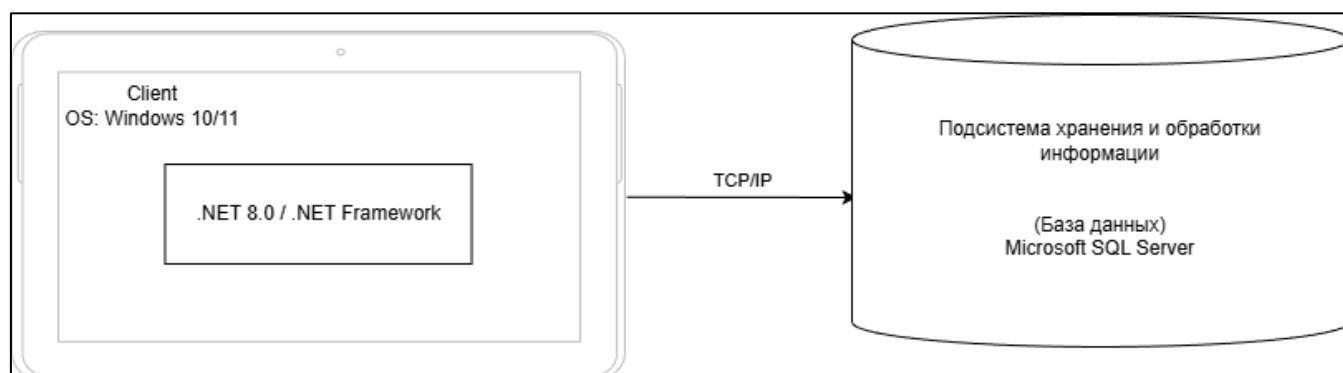


Рисунок 2.5 – Диаграмма развертывания приложения

Данная диаграмма развертывания иллюстрирует архитектуру WPF-приложения с многоуровневой структурой, где клиентская часть, серверная логика и база данных разделены для обеспечения безопасности, масштабируемости и поддержки.

Клиентская часть представляет собой WPF-приложение, работающее под управлением операционной системы Windows 10 или 11. Приложение разработано с использованием платформы .NET 8.0 или .NET Framework, что обеспечивает богатый пользовательский интерфейс и глубокую интеграцию с возможностями Windows.

Хранилище данных реализовано на системе управления базами данных Microsoft SQL Server версии 16.0.1000.6. Эта СУБД обеспечивает надежное хранение структурированных данных, поддержку транзакций, высокую доступность и безопасность информации.

Взаимодействие между компонентами осуществляется по протоколу TCP/IP, который гарантирует надежную и эффективную передачу данных по сети.

2.6 Визуализация алгоритмов

Визуализация алгоритмов состоит из двух частей: визуализация использования средства путем составления блок-схем, а также диаграммы последовательностей.

2.6.1 Блок-схема процесса оформления заказа компьютера

Блок-схема процесса оформления заказа представлена на рисунке 2.6.

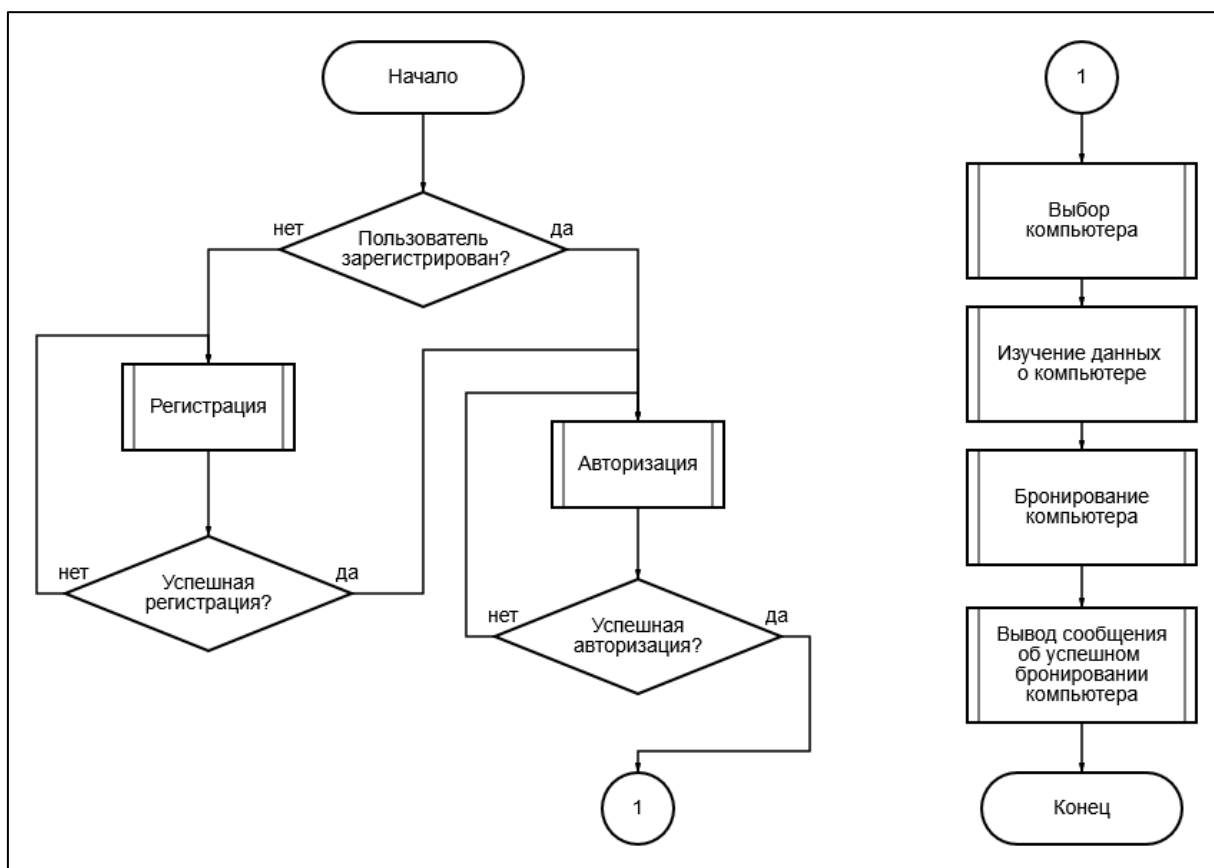


Рисунок 2.6 – Блок-схема процесса оформления заказа компьютера

Данная блок-схема описывает стандартный пользовательский сценарий для оформления заказа компьютера.

Процесс начинается с этапа верификации пользователя в приложении. В начале проверяется, зарегистрирован пользователь, или нет. Если пользователь уже имеет учетную запись, он переходит к этапу авторизации. В случае успешной авторизации пользователь получает доступ к основному функционалу приложения.

Если пользователь не имеет учетной записи, то ему предоставляется возможность создать новую учетную запись, предоставив необходимые данные. После успешной регистрации пользователь может войти в свою учетную запись.

После успешной авторизации пользователь переходит к этапу выбора компьютера.

После выбора конкретного компьютера пользователь изучает информацию о нем: описание, технические характеристики, стоимость пользования.

Убедившись в выборе, пользователь переходит к процедуре оформления заказа компьютера. На этом этапе он проверяет корректность информации, выбирает удобную дату и время, и подтверждает заказ.

В завершении процесса система отображает информацию об успешном заказе. На этом пользовательский сценарий завершен.

2.6.2 Диаграмма последовательности авторизации

Диаграмма последовательности авторизации пользователя представлена на рисунке 2.7.

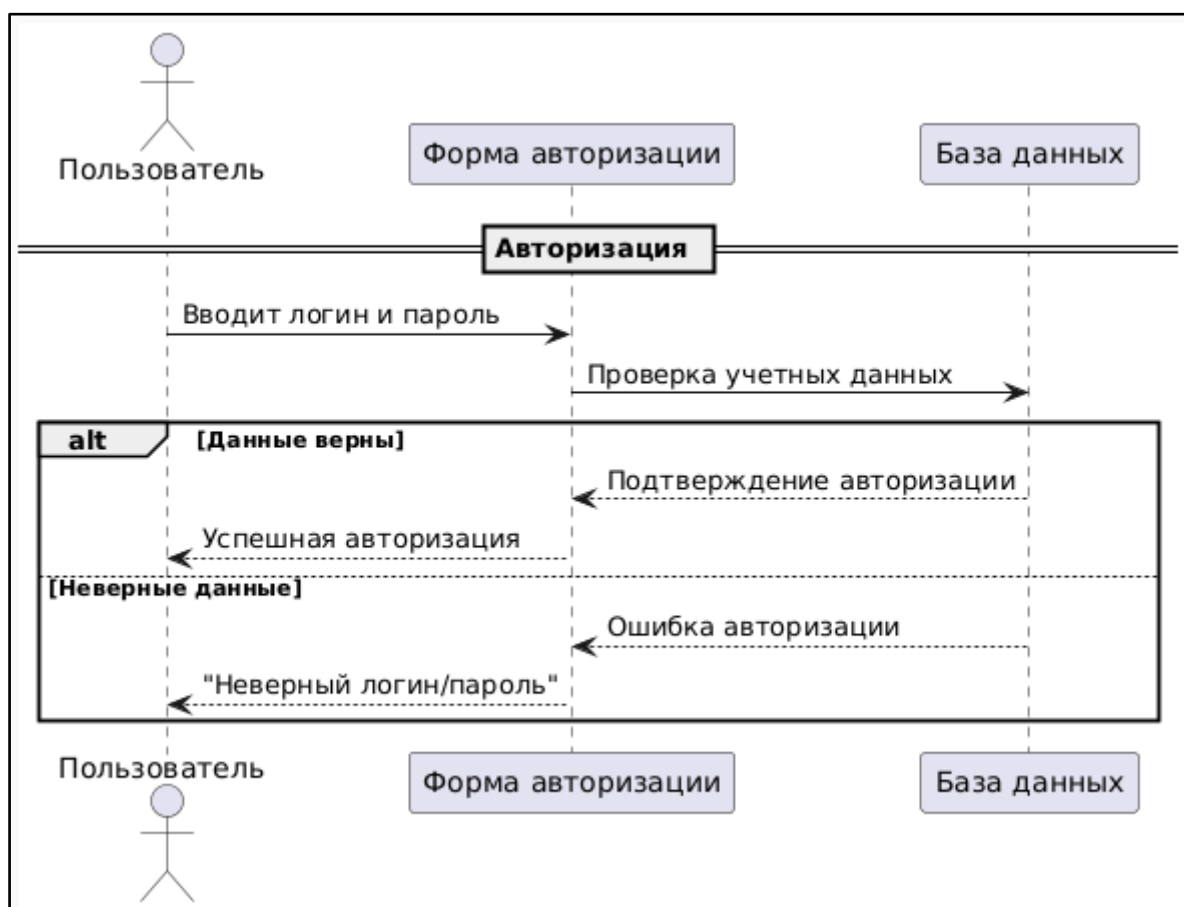


Рисунок 2.7 – Диаграмма последовательности авторизации пользователя

Данная диаграмма последовательности описывает процесс авторизации пользователя в приложении. Процесс включает взаимодействие между пользователем, формой авторизации (интерфейсом приложения) и базой данных, а также охватывает как успешный сценарий входа в систему, так и обработку ошибок при неверных учетных данных.

Процесс начинается с действий пользователя, который открывает форму авторизации в приложении. Интерфейс отображает соответствующую форму с полями для ввода логина и пароля. Пользователь вводит свои учетные данные и инициирует процесс авторизации.

После отправки данных приложение передает введенные логин и пароль на проверку в базу данных. Система выполняет запрос для поиска соответствия введенных учетных данных сохранённым записям пользователей.

В случае успешного сценария, когда данные верны (логин и пароль совпадают с записью в базе данных), база данных возвращает подтверждение авторизации. Приложение получает положительный ответ и передаёт пользователю сообщение об успешной авторизации. После этого пользователь получает доступ к основному функционалу приложения.

В случае ошибки, когда данные неверны (логин не существует или пароль не соответствует), база данных возвращает отрицательный результат. Приложение формирует сообщение об ошибке авторизации с текстом «Неверный логин/пароль» и отображает его пользователю в форме авторизации.

Таким образом, диаграмма наглядно демонстрирует простой и безопасный процесс аутентификации, подчёркивая важность проверки учетных данных на стороне сервера и информирования пользователя о результате попытки входа.

2.7 Диаграмма классов

Диаграмма классов представлена на рисунке 2.8.

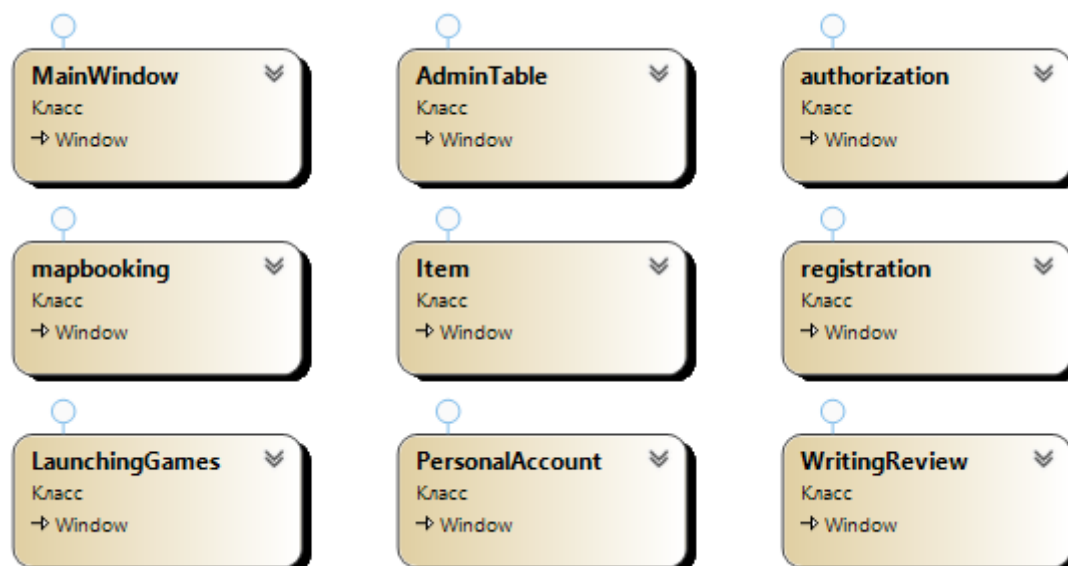


Рисунок 2.8 – Диаграмма классов

Данная диаграмма классов представляет структуру основных компонентов пользовательского интерфейса приложения. Все классы наследуются от базового класса Window. Диаграмма включает классы, отвечающие за авторизацию, регистрацию, систему бронирования, профиль, панель администратора, запуск игры.

2.8 Вывод по разделу

В данном разделе была разработана функциональная модель и модель данных программного средства.

Разработана модель данных, представленная пятью классами: Users, Computers, Orders, Reviews, ApplicationGame.

Спроектирована диаграмма развертывания приложения, демонстрирующая многоуровневую архитектуру системы с разделением на клиентскую и серверные части.

Проведена визуализация алгоритмов работы системы: блок-схема и диаграмма последовательности.

Также была спроектирована диаграмма классов, демонстрирующая структуру основных компонентов пользовательского интерфейса программного средства.

3 Разработка функциональной модели и модели данных ПС.

3.1 Основные классы программного средства

Для выполнения технических задач программного средства «Компьютерный клуб» были реализованы следующие функции и соответствующие им классы и методы:

Для неавторизованного пользователя:

- регистрация;
- авторизация;
- просмотр компьютеров;
- просмотр игр;
- возможность поменять язык и тему приложения.

Для авторизованного пользователя:

- функциональные возможности неавторизованного пользователя;
- бронирование компьютеров;
- использование личного кабинета;
- запуск игр, при активном бронировании;
- составление отзывов после завершения сеанса.

Для администратора:

- функциональные возможности авторизованного пользователя;
- добавление, удаление и обновление информации о компьютерах;
- добавление, удаление и обновление информации о бронировании пользователей;
- добавление, удаление и обновление информации приложений для запуска;
- добавление, удаление и обновление информации пользовательских отзывов;
- добавление, удаление и обновление информации о пользователе.

3.2 Описание классов и методов программного средства

3.2.1 Регистрация

Функционал регистрации пользователей реализован на странице registration, где при заполнении формы вызывается метод AddNewUser, выполняющий проверку введённых данных (валидацию полей, соответствие паролей, уникальность логина). При успешной проверке система обращается к классу database, где вызывается метод AddUserInDataBase, который хэширует пароль с помощью модуля Hash.cs и сохраняет данные пользователя в базу данных. В случае успешной регистрации происходит автоматическое перенаправление на страницу авторизации, а при ошибках выводятся соответствующие уведомления, обеспечивая удобный и безопасный процесс создания учётной записи.

3.2.2 Авторизация

Функционал авторизации пользователей реализован на странице `authorization`, где при вводе корректных данных происходит вход в приложение. На данной странице используется метод `AuthorizationClick`, выполняющий валидацию введённых данных. При успешной проверке система обращается к классу `database`, где вызывается функция `CorrectEntrance`, которая проверяет соответствие логина и пароля в базе данных. В случае успешной проверки метод возвращает значение `true` после чего пользователь получает доступ к приложению. При неверных данных выводится соответствующее сообщение об ошибке, обеспечивая безопасный и удобный процесс аутентификации.

3.2.3 Просмотр компьютеров

Функционал просмотра конфигураций компьютеров реализован в главном окне `MainWindow`. Система получает данные из базы данных через класс `database` и метод `ReadElementsInTableComputers`, после чего автоматически генерирует информационные блоки с помощью функции `GenerateComputerInfoBorder`. При нажатии на любой из отображённых компьютеров пользователь может перейти в окно бронирования `mapbooking`, где доступны функции резервирования выбранного устройства. Данная реализация обеспечивает удобный и интуитивно понятный интерфейс для просмотра и выбора компьютерной техники.

3.2.4 Просмотр игр и запуск игр

Функционал просмотра и запуска игр реализован в окне `LaunchingGames`. Для отображения списка игр система использует метод `GenerateInformation`, который получает данные из базы данных через метод `ReadElementsInTableApplicationGame` класса `database`. Сформированный список игр позволяет пользователю запускать любую игру в рамках активного сеанса бронирования компьютера, обеспечивая удобный доступ к игровому контенту во время аренды. Данный подход гарантирует корректное отображение доступных игр и их бесперебойный запуск.

3.2.5 Возможность поменять язык и тему приложения

Функционал смены языка и темы оформления реализован на главной странице `MainWindow`. Для изменения языка пользователь может нажать соответствующую кнопку: `SwitchToRussian` для русского или `SwitchToEnglish` для английского языка, что активирует переключение между языковыми ресурсами `Strings.ru-RU.xaml` и `Strings.en-US.xaml`. Аналогично реализована смена темы оформления: кнопка `ChangeThemeBtn` вызывает функцию `ChangeTheme`, которая переключает цветовую схему приложения между светлой и темной темами.

3.2.6 Бронирование компьютеров

Процесс бронирования компьютера осуществляется через окно `Item`, доступное из интерфейса карты бронирования `mapbooking`. После заполнения необходимых данных в форме бронирования система через функцию `WriteInfoByItem` выполняет проверку корректности введённой информации (включая дату, время и продолжительность сеанса), после чего при успешной валидации вызывает метод `AddElementsInTableOrders` из класса `database`, который сохраняет данные бронирования в таблицу базы данных.

3.2.7 Использование личного кабинета

Доступ к функционалу бронирования компьютеров предоставляется исключительно зарегистрированным и авторизованным пользователям, при этом в личном кабинете `PersonalAccount` реализованы ключевые функции: отображение персональных данных через метод `GetInfoAboutUser`, получающий информацию из базы данных, и безопасная смена пароля с помощью функции `SaveInfo`, которая выполняет валидацию нового пароля и его хеширование перед сохранением в систему.

3.2.8 Составление отзывов

Функционал оставления отзывов доступен пользователям только после завершения активного заказа через специальное окно `WritingReview`, где после заполнения формы и нажатия кнопки отправки вызывается функция `SaveReview`, которая проверяет корректность введённых данных и передаёт информацию в базу данных через класс `database`, используя метод `AddElementsInTableReviews` для сохранения отзыва в соответствующую таблицу.

3.2.9 Управление проектом

Административная панель `AdminTable` предоставляет исключительные права управления данными проекта администратору, реализуя полный набор CRUD-операций (`Create`, `Read`, `Update`, `Delete`) для каждой таблицы базы данных: функция `ReadElementsInTable` выводит текущие данные, `AddElementInTable` добавляет новые записи, `UpdateElementInTable` изменяет существующие, а `DeleteElementInTable` удаляет информацию.

3.3 Вывод по разделу

В данном разделе была разработана функциональная модель программного средства, включая такие функциональные блоки как авторизация и регистрация

пользователей, запуск игр, бронирование компьютеров и другие. Для каждого функционального блока реализованы соответствующие классы и методы.

4 Тестирование

4.1 Тестирование регистрации

При попытке регистрации с уже существующим логином система определяет совпадение и выводит пользователю соответствующее уведомление об ошибке (пример визуального отображения показан на рисунке 4.1), что предотвращает дублирование учетных записей и обеспечивает корректную работу механизма уникальности логинов в базе данных.

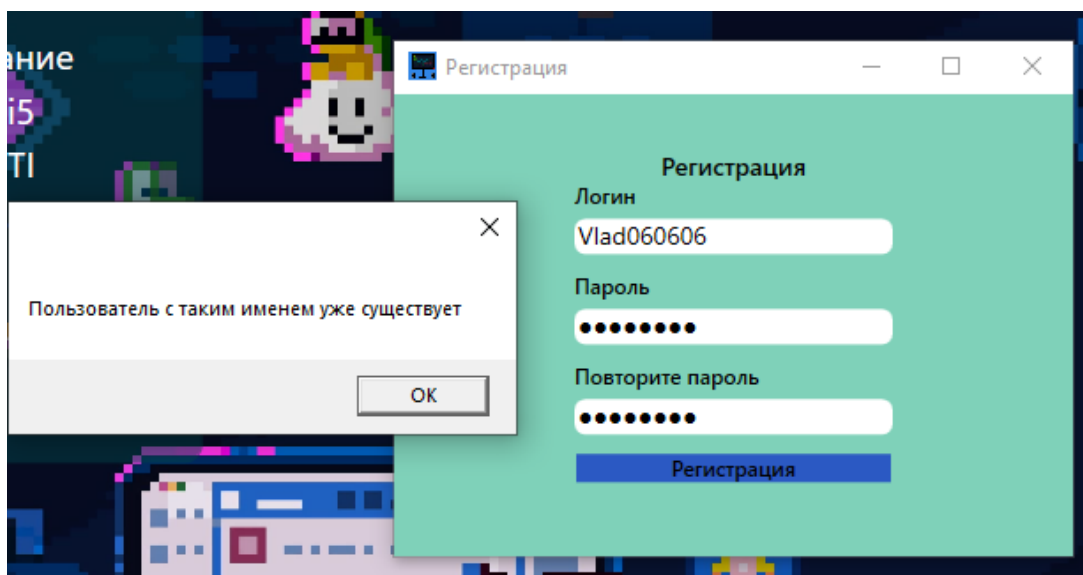


Рисунок 4.1 – Пользователь с таким логином существует

Система регистрации проверяет длину логина и выводит ошибку, если пользователь ввел значение короче 5 или длиннее 99 символов, что гарантирует соответствие установленным требованиям к длине учетных данных (пример сообщения об ошибке представлен на рисунке 4.2).

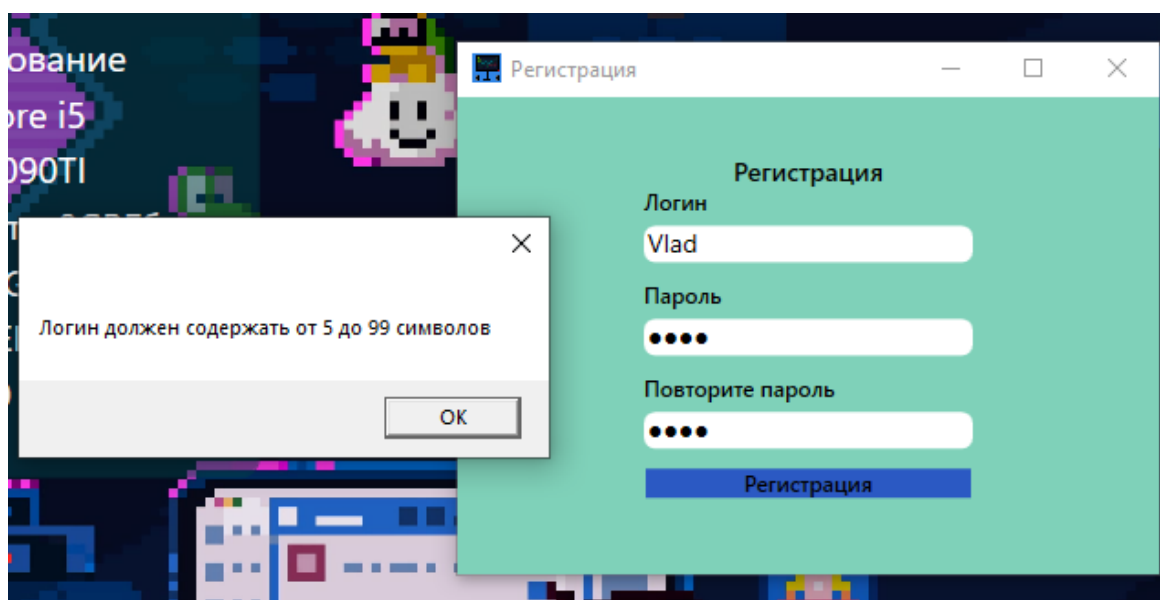


Рисунок 4.2 – Логин должен содержать от 5 до 99 символов

При вводе пароля система проверяет его длину и выводит ошибку, если количество символов не соответствует установленному диапазону от 5 до 99, что обеспечивает соблюдение требований безопасности к пользовательским учетным данным (пример сообщения об ошибке показан на рисунке 4.3).

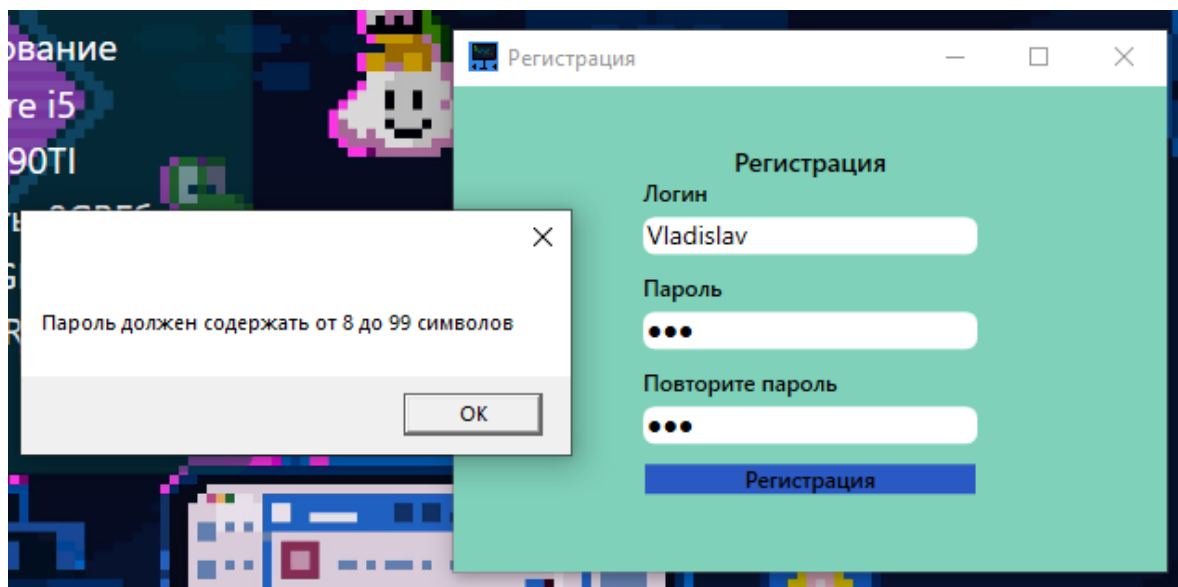


Рисунок 4.3 – Пароль должен содержать от 5 до 99 символов

Система регистрации проверяет совпадение паролей в полях "Пароль" и "Подтверждение пароля", и при их несоответствии выводит соответствующую ошибку (как показано на рисунке 4.4), что исключает возможность опечаток и обеспечивает корректность вводимых учетных данных.

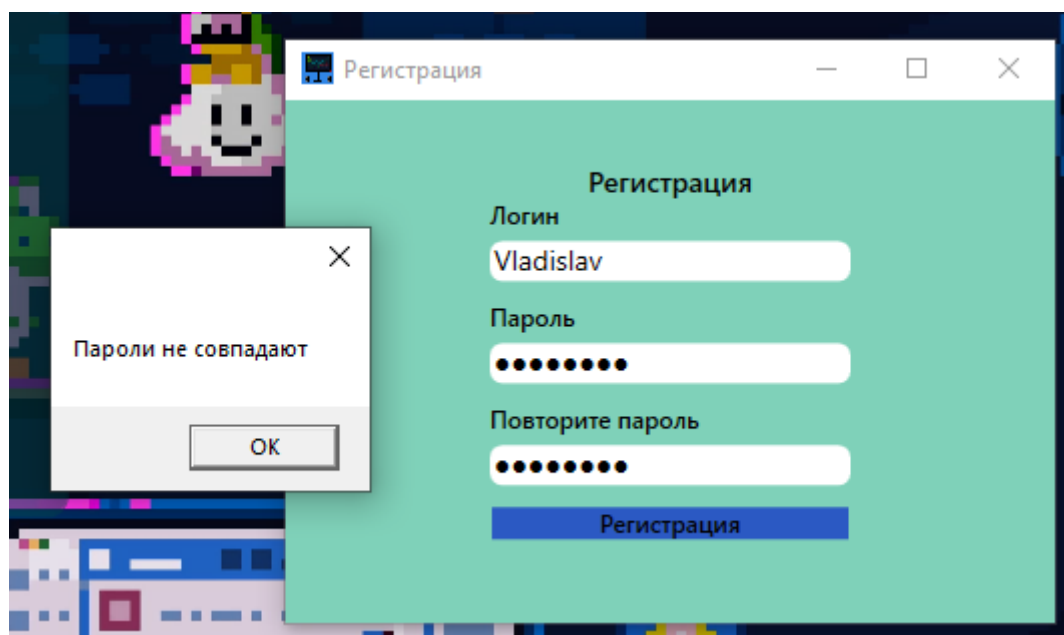


Рисунок 4.4 – Пароли не совпадают

При успешном заполнении всех обязательных полей с корректными данными (уникальный логин, пароль требуемой длины и совпадающие значения в полях "Пароль" и "Подтверждение пароля") система завершает процесс регистрации, сохраняет данные пользователя в базу. Демонстрация на рисунке 4.5.

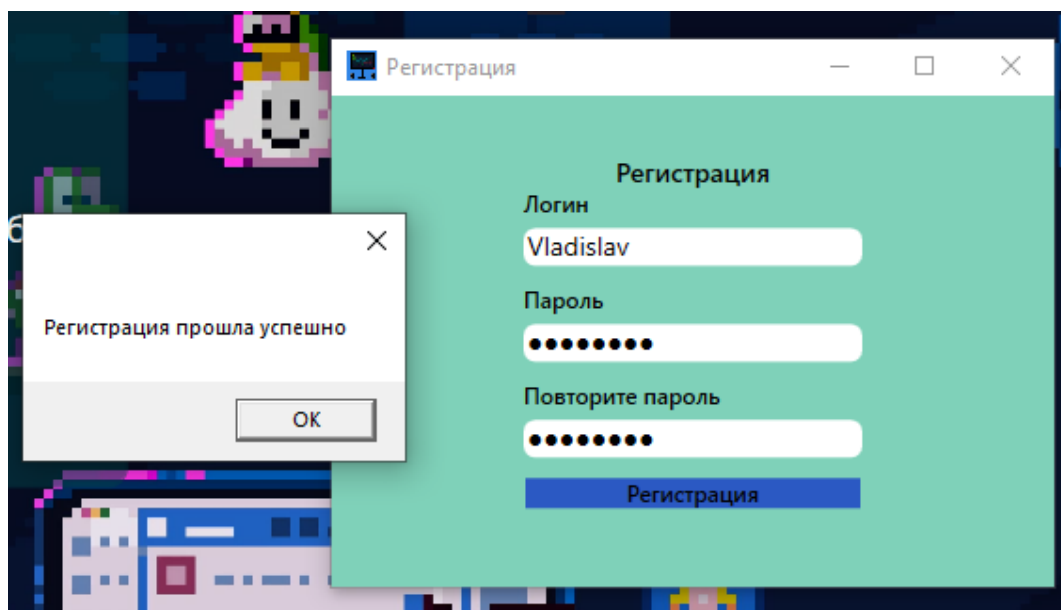


Рисунок 4.5 – Успешная регистрация

4.2 Тестирование авторизации

При попытке авторизации с несуществующим в системе логином приложение выявляет это и отображает пользователю соответствующее сообщение об ошибке (как показано на рисунке 4.6), предотвращая неверные попытки входа и информируя о необходимости проверки введенных данных или прохождения регистрации.

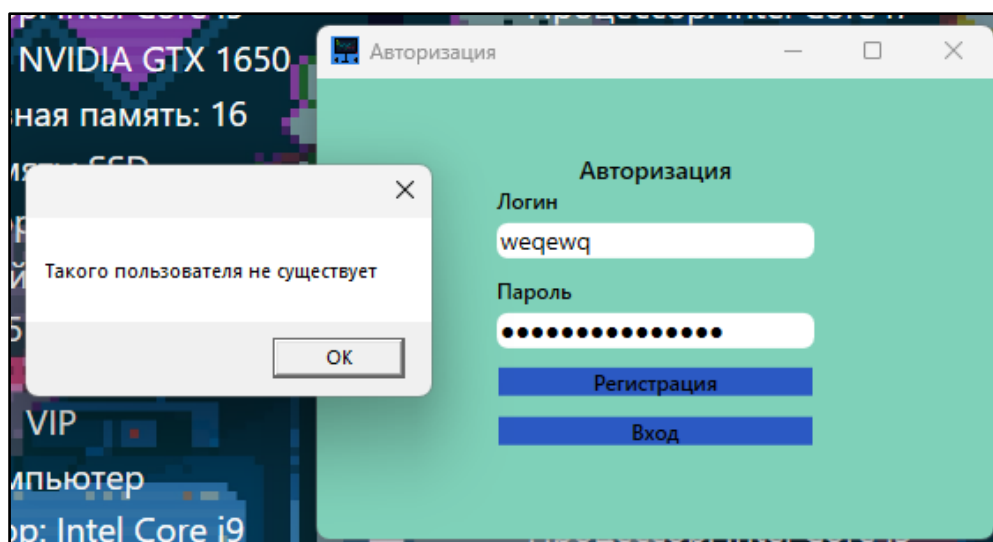


Рисунок 4.6 – Такого пользователя не существует

При вводе неверного пароля во время авторизации система проверяет его соответствие с данными в базе и выводит пользователю уведомление об ошибке (как показано на рисунке 4.7).

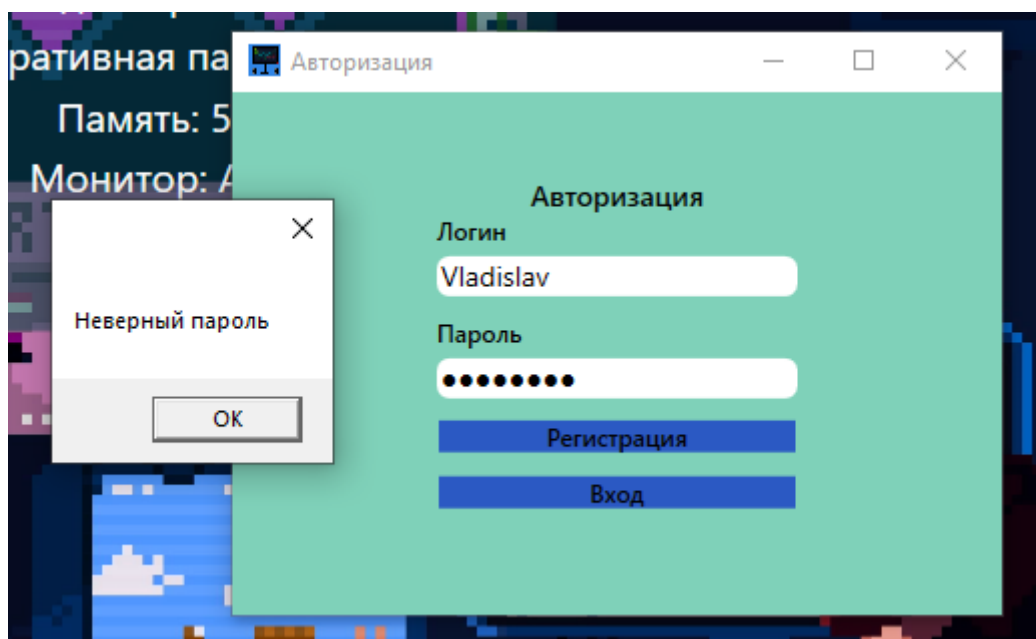


Рисунок 4.7 – Неверный пароль

4.3 Тестирование бронирования

Система бронирования проверяет выбранную дату и предотвращает выбор прошедших дат, выводя соответствующее уведомление об ошибке (как показано на рисунке 4.8).

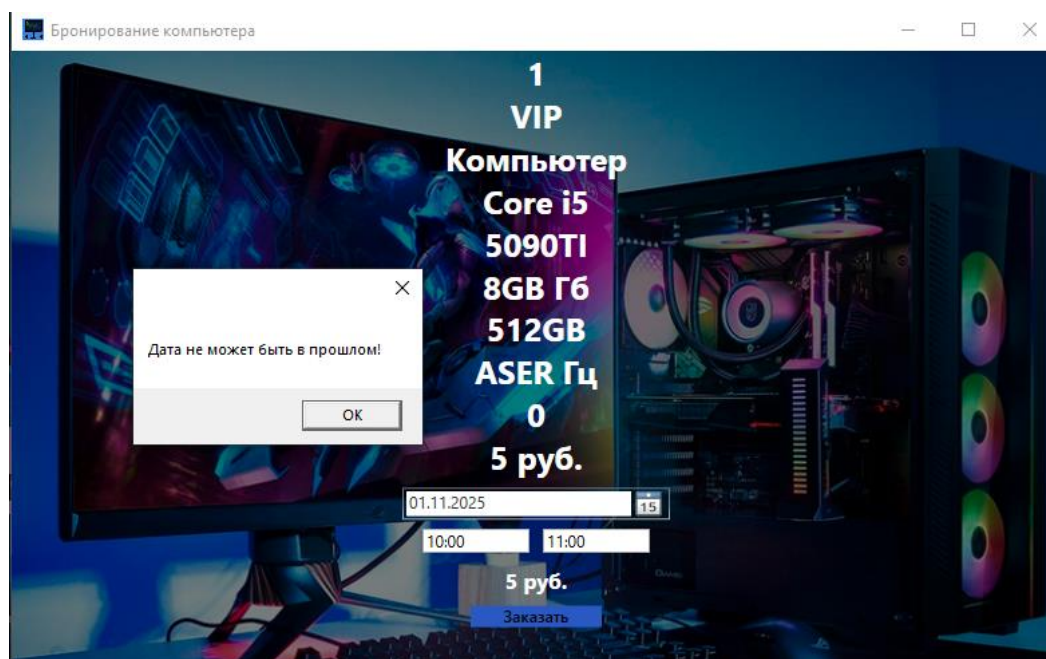


Рисунок 4.8 – Дата не может быть в прошлом

Система бронирования компьютеров имеет строгие временные ограничения: при попытке выбрать дату, превышающую допустимый двухнедельный период (14 дней вперед от текущей даты), появляется уведомление "Бронирование доступно только на 2 недели вперед" (как показано на рисунке 4.9).

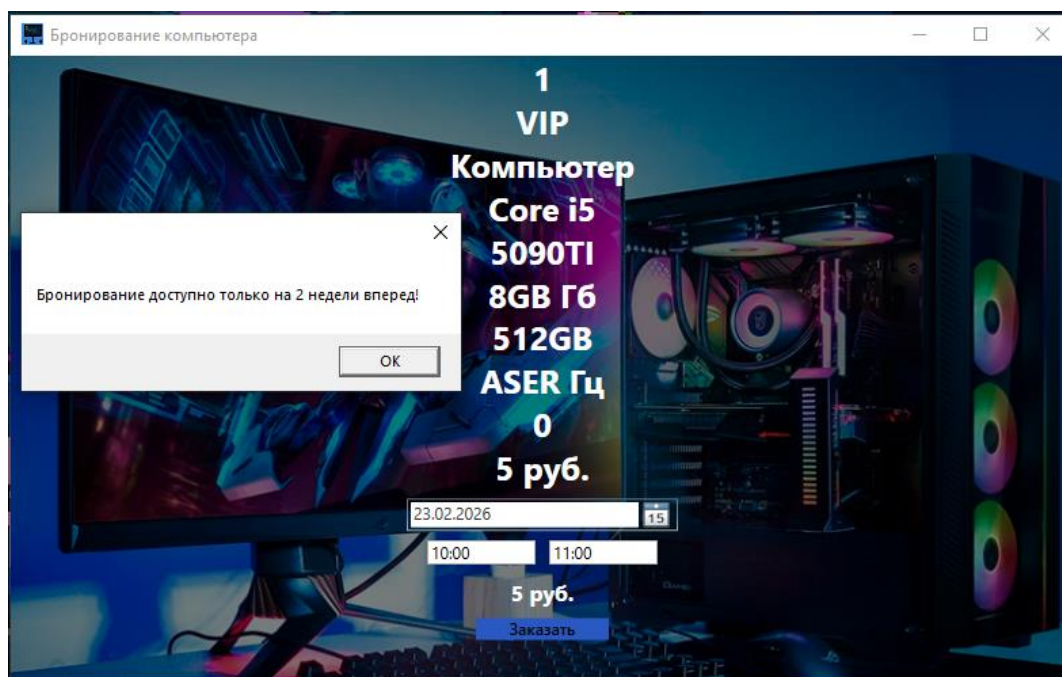


Рисунок 4.9 – Бронирование доступно только на 2 недели вперед

При попытке указать время начала бронирования раньше текущего момента система определяет ошибку и выводит уведомление "Время начала бронирования должно быть позже текущего времени" (как показано на рисунке 4.10).

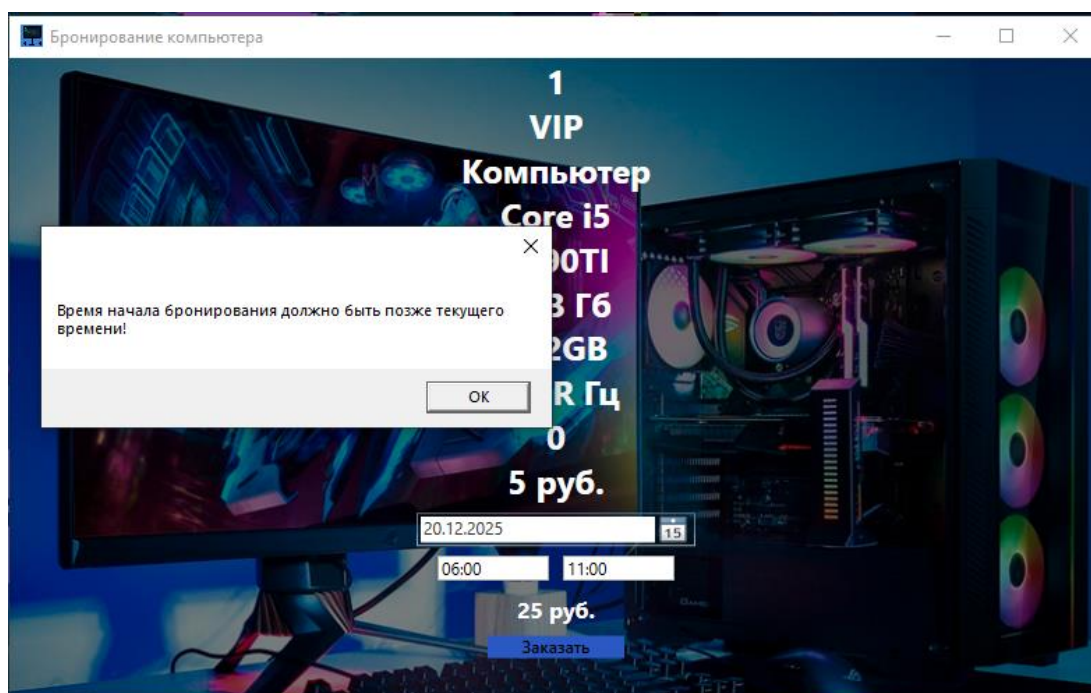


Рисунок 4.10 – Время начала бронирования должно быть позже текущего времени

Система автоматически проверяет корректность временного интервала бронирования и при попытке установить время окончания раньше времени начала немедленно выводит уведомление "Время начала должно быть меньше времени окончания" (как показано на рисунке 4.11).

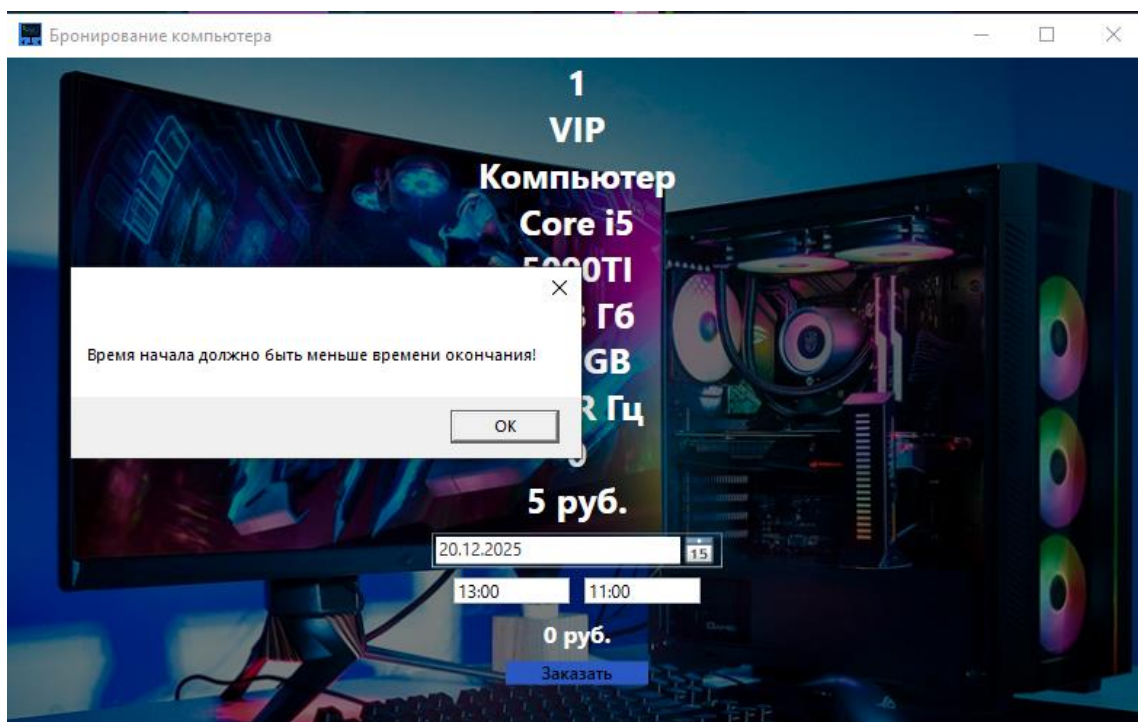


Рисунок 4.11 – Время начала должно быть меньше времени окончания

При успешном завершении операции добавления данных система выводит пользователю уведомление "Запись успешно добавлена" (как показано на рисунке 4.12).

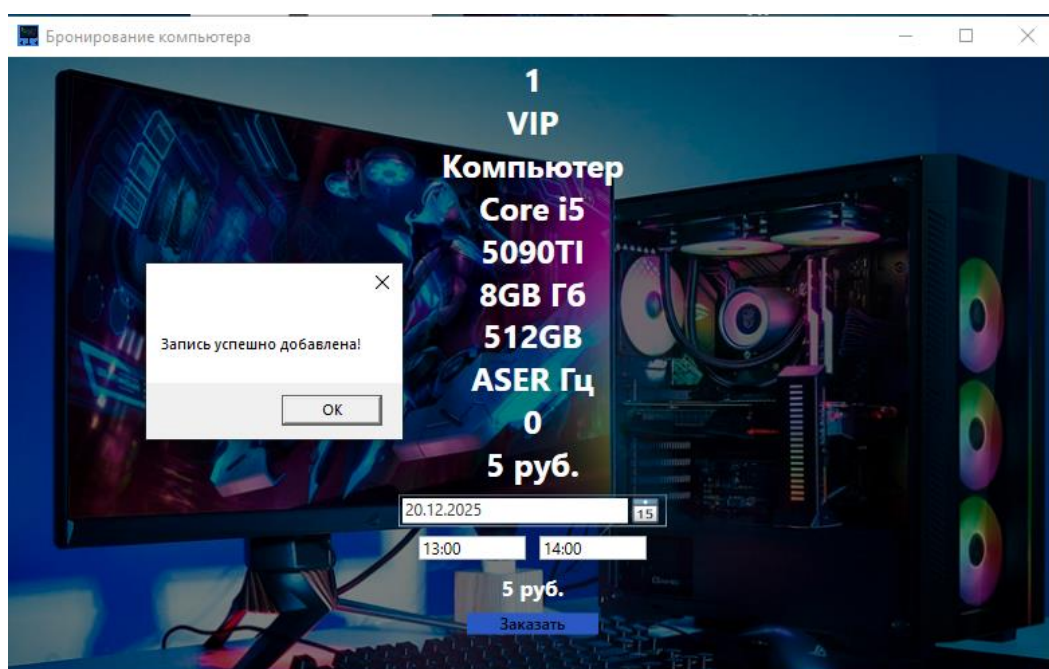


Рисунок 4.12 – Запись успешно добавлена

Теперь пользователь при активном заказе может входить в приложения и игры.

4.4 Тестирование поиска компьютеров и игр

При вводе текста в поле поиска компьютеров и игр система ограничивает максимальную длину запроса до 20 символов: при достижении этого лимита текст подсвечивается красной рамкой. Пример на рисунке 4.13.

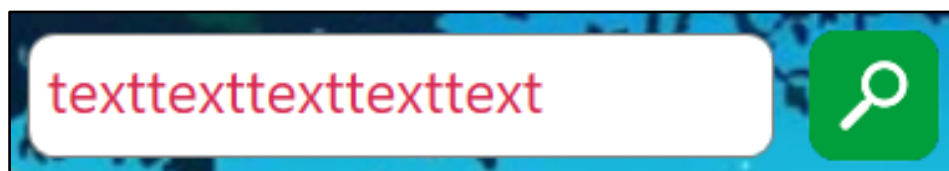


Рисунок 4.13 – Ограничение поиска

4.5 Тестирование личного кабинета

При смене пароля в личном кабинете система требует соблюдения строгих критериев безопасности: новый пароль должен содержать от 8 до 99 символов, при нарушении этого правила поле ввода подсвечивается красным, выводится уведомление "Пароль должен содержать от 8 до 99 символов" (как показано на рисунке 5.14).

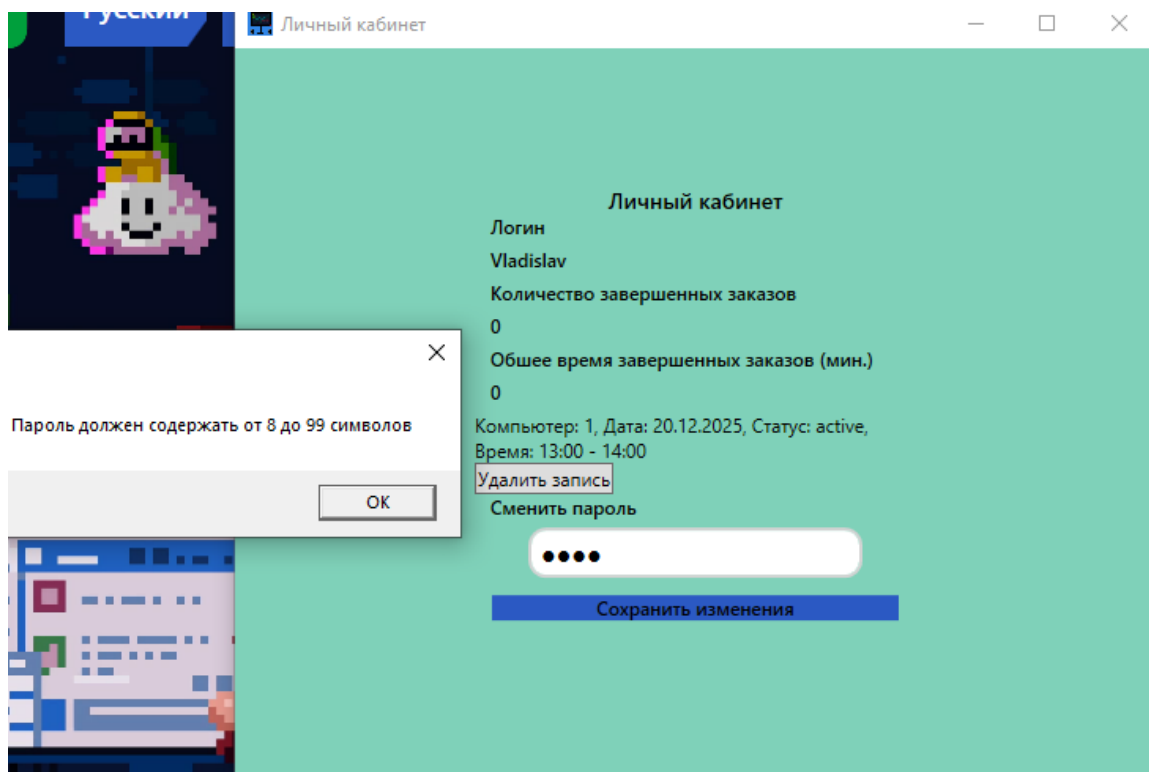


Рисунок 4.14 – Пароль должен содержать от 8 до 99 символов

В личном кабинете система блокирует возможность отмены заказов, время которых уже наступило: при попытке отменить такой заказ появляется уведомление "Нельзя отменить заказ, время которого уже наступило" (как показано на рисунке 4.15).

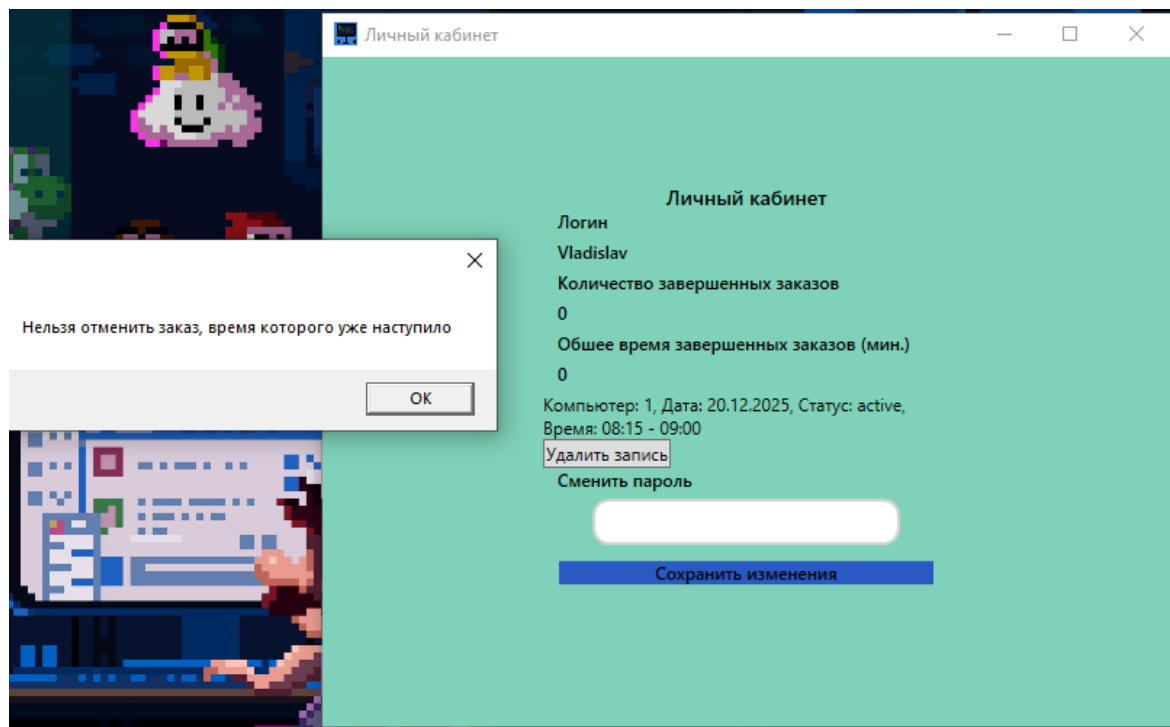


Рисунок 4.15 – Нельзя отменить заказ, время которого уже наступило

При успешной отмене заказа система автоматически инициирует процесс возврата денежных средств: сумма, соответствующая неиспользованному времени бронирования, возвращается на исходный платежный метод пользователя, о чем пользователь получает уведомление.

4.6 Тестирование управления данными проекта

В административной панели управления данными (AdminTable) для таблицы Order реализована та же система валидации, что и для обычных пользователей: при попытке установить дату заказа раньше текущей даты система автоматически блокирует сохранение, подсвечивает поле красным и выводит сообщение "Дата заказа не может быть раньше сегодняшнего дня" (рисунок 4.16.), что гарантирует целостность данных и единые правила работы для всех уровней доступа.

48	5	3	completed	03.05.2025	17:12	17:11	
49	5	9	completed	03.05.2025	17:15	17:16	
50	1	1	completed	05.05.2025	15:33	15:34	
51	1	9	completed	09.05.2025	16:00	16:10	
52	6	5	completed	09.05.2025	16:10	16:15	
53	6	9	cancelled	09.05.2025	16:10	16:20	
54	6	10	cancelled	09.05.2025	16:20	16:24	
55	6	1	cancelled	09.05.2025	16:28	16:35	
56	1	4	completed	09.05.2025	16:37	16:38	

×

Дата заказа не может быть раньше сегодняшнего дня

OK

Рисунок 4.16. – Дата заказа не может быть раньше текущего дня

В административной панели (AdminTable) для таблицы Order сохранена идентичная пользовательской валидация временных интервалов: система автоматически проверяет, чтобы время окончания бронирования было позже времени начала, и при нарушении этого правила выводит администратору сообщение "Время окончания должно быть позже времени начала" (рисунок. 4.17).

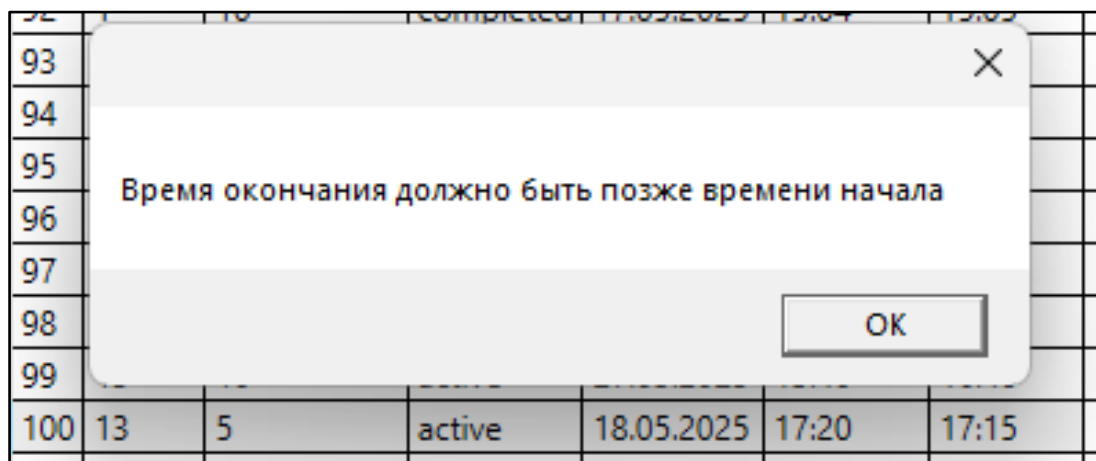


Рисунок 4.17. – Время окончания должно быть позже времени начала

В административной панели (AdminTable) для таблицы **Order** реализована валидация статусов: при попытке установить недопустимое значение система определяет ошибку (рисунок. 4.18).

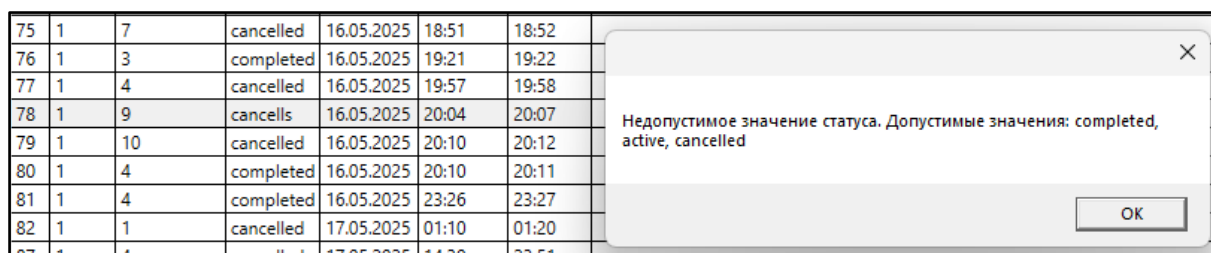


Рисунок 4.18. – Валидация статусов

В административном интерфейсе AdminTable для всех таблиц базы данных реализована строгая валидация типов данных: при вводе информации, не соответствующей ожидаемому формату (например, текст в числовом поле или некорректный формат даты), система мгновенно реагирует – проблемное поле подсвечивается красной рамкой (рисунок 4.19).

Rating	PricePerHour
5	5
4.5	8
4.5	asd

Рисунок 4.19. – Валидация типов данных

Остальные проверки в таблицах работают по схожему принципу, что не дает админу ввести не корректные данные, поэтому автоматически проверяет соответствие вводимых администратором значений установленным правилам: при попытке сохранения некорректных данных (например, отрицательная цена, несуществующий внешний ключ, дата в прошлом) срабатывает встроенная в валидация.

4.7 Вывод по разделу

В данном разделе было проведено функциональное тестирование ключевых модулей программного средства: авторизация и регистрация пользователей, бронирование компьютеров, отмена бронирования, запуск игр и приложений, смена пароля. Тестирование подтвердило корректную работу системы как в штатных сценариях, так и при обработке ошибочных ситуаций.

Заключение

В ходе выполнения курсового проекта было разработано программное средство для бронирования компьютеров и запуска игр. Были выполнены все поставленные задачи: проведен аналитический обзор существующих решений, разработана база данных с пятью таблицами, и установлены связи между ними.

В процессе проектирования были созданы UML-диаграммы вариантов использования для ролей гостя, клиента и администратора, диаграмма развертывания, блок-схема сценария оформления заказа компьютера и диаграмма последовательности авторизации пользователя.

Реализованы основные функции системы: регистрация и авторизация пользователей, бронирование компьютеров, отмена бронирований, запуск игр и приложений, а также для администратора были реализованы функции для управления данными о компьютерах, пользователях, заказах и запускаемых приложениях. Проведенное функциональное тестирование подтвердило корректность работы системы в штатных и исключительных ситуациях.

Разработанное программное средство соответствует поставленным требованиям и готово к использованию для автоматизации процесса бронирования компьютеров в клубе.

Список использованных источников

1. Аналог «Cuberspace» [Электронный ресурс]. – Режим доступа: <https://cyberspace.by/>. – Дата доступа: 17.10.2025.
2. Аналог «Pudge» [Электронный ресурс]. – Режим доступа: <https://pudge.by/>. – Дата доступа: 17.10.2025
3. Аналог «Colizeum» [Электронный ресурс]. – Режим доступа: <https://colizeumarena.com/>. – Дата доступа: 17.10.2025
4. Аналог «OneShot» [Электронный ресурс]. – Режим доступа: <https://lounge-preview.vercel.app/>. – Дата доступа: 17.10.2025
5. Аналог «Alliance» [Электронный ресурс]. – Режим доступа: <https://cyberalliance.by/>. – Дата доступа: 17.10.2025
6. Полное руководство по языку программирования C# 13 и платформе .NET
9. Режим доступа: <https://metanit.com/sharp/tutorial/> – Дата доступа: 23.04.2025
7. Мушук, Н.В. Курс лекций по языку программирования C# / А. Н. Мушук. – Минск: БГТУ, 2024.
8. Руководство по WPF // [Электронный ресурс]. – Режим доступа: <https://metanit.com/sharp/wpf/> – Дата доступа: 25.04.2025
9. Руководство по XAML // [Электронный ресурс]. – Режим доступа: <https://www.tutorialspoint.com/xaml/index.htm> – Дата доступа: 25.04.2025

ПРИЛОЖЕНИЕ А. Полный код программного средства

Style.xaml

```

<ResourceDictionary
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"

xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
    <!-- Блок для темы -->
    <!--<Color x:Key="Primary">#109521</Color>
    <Color x:Key="PrimaryLight">#52af75</Color>
    <Color x:Key="PrimaryAnim">#1bcc32</Color>
    <Color x:Key="InputError">#d92952</Color>-->

    <Color x:Key="Primary">#2B59C3</Color>
    <Color x:Key="PrimaryLight">#7FD1B9</Color>
    <Color x:Key="PrimaryAnim">#EBBAB9</Color>
    <Color x:Key="InputError">#FF6B6B</Color>

    <!--<ImageBrush x:Key="mainfon" ImageSource="/images/fon.jpg"
Stretch="UniformToFill"/>-->
    <ImageBrush x:Key="mainfon" ImageSource="/images/fon1.jpg"
Stretch="UniformToFill"/>

    <SolidColorBrush x:Key="PrimaryBrush" Color="{StaticResource
Primary}"/>
    <SolidColorBrush x:Key="PrimaryLightBrush" Color="{StaticResource
PrimaryLight}"/>
    <SolidColorBrush x:Key="PrimaryAnimBrush" Color="{StaticResource
PrimaryAnim}"/>
    <SolidColorBrush x:Key="InputErrorBrush" Color="{StaticResource
InputError}"/>

    <!-- Стили для всех кнопок -->
    <Style x:Key="TransparentButtonStyle" TargetType="Button">
        <Setter Property="Background" Value="Transparent" />
        <Setter Property="BorderBrush" Value="Transparent" />
        <Setter Property="Foreground" Value="Black" />
        <Setter Property="Template">
            <Setter.Value>
                <ControlTemplate TargetType="Button">
                    <Border Background="{TemplateBinding Background}"
                        BorderBrush="{TemplateBinding BorderBrush}"
                        BorderThickness="{TemplateBinding
BorderThickness}">
                        <ContentPresenter
HorizontalAlignment="Center"
VerticalAlignment="Center" />
                    </Border>
                    <ControlTemplate.Triggers>
                        </ControlTemplate.Triggers>
                    </ControlTemplate>
                </Setter.Value>
            </Setter.Value>
        </Setter>
    </Style>

```

```

        </Setter>
    </Style>

    <!-- Стили для MainWindow -->
    <Style x:Key="RoundedBorderStyle" TargetType="Border">
        <Setter Property="CornerRadius" Value="10"/>
        <Setter Property="Background" Value="White"/>
        <Setter Property="BorderBrush" Value="Gray"/>
        <Setter Property="BorderThickness" Value="1"/>
        <Setter Property="Padding" Value="5"/>
    </Style>

    <Style x:Key="TransparentTextBoxStyle" TargetType="TextBox">
        <Setter Property="BorderThickness" Value="0"/>
        <Setter Property="Background" Value="Transparent"/>
        <Setter Property="FontSize" Value="24"/>
    </Style>

    <Style x:Key="TextBoxDataTrigger" TargetType="TextBox"
    BasedOn="{StaticResource TransparentTextBoxStyle}">
        <Style.Triggers>
            <DataTrigger Binding="{Binding Text.Length,
            RelativeSource={RelativeSource Self}}" Value="20">
                <Setter Property="Foreground" Value="#d92952"/>
            </DataTrigger>
        </Style.Triggers>
    </Style>

    <Style x:Key="LanguageButtonStyle" TargetType="Button"
    BasedOn="{StaticResource TransparentButtonStyle}">
        <Setter Property="Background" Value="{StaticResource
        PrimaryBrush}"/>
        <Setter Property="FontSize" Value="18"/>
        <Setter Property="FontWeight" Value="SemiBold"/>
        <Setter Property="Foreground" Value="White"/>
        <Setter Property="Padding" Value="10 5 10 5"/>
    </Style>

    <Style x:Key="AnimationLangBtn" TargetType="Button"
    BasedOn="{StaticResource LanguageButtonStyle}">
        <Setter Property="RenderTransformOrigin" Value="0.5,0.5"/>
        <Setter Property="RenderTransform">
            <Setter.Value>
                <ScaleTransform ScaleX="1" ScaleY="1"/>
            </Setter.Value>
        </Setter>
        <Style.Triggers>
            <EventTrigger RoutedEvent="MouseEnter">
                <BeginStoryboard>
                    <Storyboard>
                        <DoubleAnimation
Storyboard.TargetProperty="RenderTransform.ScaleX"
To="1.05"

```

```

        Duration="0:0:0.2"/>
        <DoubleAnimation
Storyboard.TargetProperty="RenderTransform.ScaleY"
    To="1.05"
    Duration="0:0:0.2"/>
        <DoubleAnimation
Storyboard.TargetProperty="Opacity"
    To="0.9"
    Duration="0:0:0.2"/>
        <ColorAnimation

Storyboard.TargetProperty="(Panel.Background).(SolidColorBrush.Color)
"
    To="{StaticResource PrimaryAnim}"
    Duration="0:0:0.2"/>
    </Storyboard>
</BeginStoryboard>
</EventTrigger>
<EventTrigger RoutedEvent="MouseLeave">
    <BeginStoryboard>
        <Storyboard>
            <DoubleAnimation

Storyboard.TargetProperty="RenderTransform.ScaleX"
    To="1.0"
    Duration="0:0:0.2"/>
            <DoubleAnimation

Storyboard.TargetProperty="RenderTransform.ScaleY"
    To="1.0"
    Duration="0:0:0.2"/>
            <DoubleAnimation
Storyboard.TargetProperty="Opacity"
    To="1"
    Duration="0:0:0.2"/>
            <ColorAnimation

Storyboard.TargetProperty="(Panel.Background).(SolidColorBrush.Color)
"
    To="{StaticResource Primary}"
    Duration="0:0:0.2"/>
        </Storyboard>
    </BeginStoryboard>
</EventTrigger>
</Style.Triggers>
</Style>

<Style x:Key="UserButtonStyle" TargetType="Border">
    <Setter Property="Background" Value="{StaticResource
PrimaryBrush}"/>
    <Style.Triggers>
        <EventTrigger RoutedEvent="MouseEnter">
            <BeginStoryboard>

```

```

        <Storyboard>
            <DoubleAnimation
Storyboard.TargetProperty="Opacity" To="0.9" Duration="0:0:0.2"/>
            <ColorAnimation
Storyboard.TargetProperty="Background.Color"
                To="{StaticResource PrimaryAnim}"
                Duration="0:0:0.2"/>
        </Storyboard>
    </BeginStoryboard>
</EventTrigger>
<EventTrigger RoutedEvent="MouseLeave">
    <BeginStoryboard>
        <Storyboard>
            <DoubleAnimation
Storyboard.TargetProperty="Opacity" To="1" Duration="0:0:0.2"/>
            <ColorAnimation
Storyboard.TargetProperty="Background.Color"
                To="{StaticResource Primary}"
                Duration="0:0:0.2"/>
        </Storyboard>
    </BeginStoryboard>
</EventTrigger>
</Style.Triggers>
</Style>

<Style x:Key="UserStackPanelStyle" TargetType="StackPanel">
    <Setter Property="Orientation" Value="Horizontal"/>
    <Setter Property="Height" Value="50"/>
</Style>

<Style x:Key="UserNameTextBlockStyle" TargetType="TextBlock">
    <Setter Property="FontSize" Value="18"/>
    <Setter Property="FontWeight" Value="SemiBold"/>
    <Setter Property="Foreground" Value="White"/>
    <Setter Property="VerticalAlignment" Value="Center"/>
    <Setter Property="Margin" Value="10 0 10 0"/>
</Style>

<!-- Стили для AddItem -->
<Style x:Key="AddItemStackPanelStyle" TargetType="StackPanel">
    <Setter Property="MaxWidth" Value="300"/>
    <Setter Property="VerticalAlignment" Value="Center"/>
    <Setter Property="Background" Value="{StaticResource
PrimaryBrush}"/>
    <Setter Property="Opacity" Value="0.9"/>
</Style>

<Style x:Key="AddItemTextBlockStyle" TargetType="TextBlock">
    <Setter Property="Margin" Value="10 10 10 0"/>
    <Setter Property="Foreground" Value="White"/>
    <Setter Property="HorizontalAlignment" Value="Center"/>
</Style>

```

```

<Style x:Key="AddItemTextBoxStyle" TargetType="TextBox">
    <Setter Property="Margin" Value="10 10 10 0"/>
</Style>

<Style x:Key="AddItemButtonStyle" TargetType="Button"
BasedOn="{StaticResource TransparentButtonStyle}">
    <Setter Property="Margin" Value="10 10 10 10"/>
    <Setter Property="Background" Value="{StaticResource
PrimaryBrush}"/>
    <Setter Property="Foreground" Value="white"/>
</Style>

<!-- Стили для authorization -->
<Style x:Key="AuthorizationBorderStyle" TargetType="Border">
    <Setter Property="MaxWidth" Value="200"/>
    <Setter Property="Height" Value="Auto"/>
</Style>

<Style x:Key="AuthorizationStackPanelStyle"
TargetType="StackPanel">
    <Setter Property="VerticalAlignment" Value="Center"/>
</Style>

<Style x:Key="AuthorizationTextBlockStyle" TargetType="TextBlock">
    <Setter Property="FontWeight" Value="SemiBold"/>
    <Setter Property="FontSize" Value="14"/>
    <Setter Property="HorizontalAlignment" Value="Center"/>
</Style>

<Style x:Key="AuthorizationLabelTextBlockStyle"
TargetType="TextBlock">
    <Setter Property="FontWeight" Value="SemiBold"/>
    <Setter Property="Margin" Value="10 0 0 5"/>
</Style>

<Style x:Key="AuthorizationInputBorderStyle" TargetType="Border">
    <Setter Property="CornerRadius" Value="6"/>
    <Setter Property="Background" Value="White"/>
    <Setter Property="Height" Value="20"/>
    <Setter Property="Margin" Value="10 0 10 10"/>
</Style>

<Style x:Key="AuthorizationTextBoxStyle" TargetType="TextBox">
    <Setter Property="BorderThickness" Value="0"/>
    <Setter Property="Background" Value="Transparent"/>
    <Setter Property="FontSize" Value="14"/>
</Style>

<Style x:Key="AuthorizationPasswordBoxStyle"
TargetType="PasswordBox">
    <Setter Property="BorderThickness" Value="0"/>
    <Setter Property="Background" Value="Transparent"/>
    <Setter Property="FontSize" Value="14"/>
</Style>

```

```

        <Style      x:Key="AuthorizationButtonStyle"      TargetType="Button"
BasedOn="{StaticResource TransparentButtonStyle}">
            <Setter Property="FontWeight" Value="SemiBold"/>
            <Setter      Property="Background"      Value="{StaticResource
PrimaryLightBrush}"/>
            <Setter Property="Margin" Value="10 0 10 10"/>
        </Style>

        <Style TargetType="Button" x:Key="AuthorizationButtonStyleTrigger"
BasedOn="{StaticResource AuthorizationButtonStyle}">
            <Setter Property="RenderTransformOrigin" Value="0.5,0.5"/>
            <Setter Property="RenderTransform">
                <Setter.Value>
                    <ScaleTransform ScaleX="1" ScaleY="1"/>
                </Setter.Value>
            </Setter>

            <Style.Triggers>
                <MultiTrigger>
                    <MultiTrigger.Conditions>
                        <Condition Property="IsMouseOver" Value="True"/>
                    </MultiTrigger.Conditions>
                    <MultiTrigger.EnterActions>
                        <BeginStoryboard>
                            <Storyboard>
                                <DoubleAnimation
Storyboard.TargetProperty="RenderTransform.ScaleX"
                                    To="1.03"
                                    Duration="0:0:0.2"/>
                                <DoubleAnimation
Storyboard.TargetProperty="RenderTransform.ScaleY"
                                    To="1.15"
                                    Duration="0:0:0.2"/>
                            </Storyboard>
                        </BeginStoryboard>
                    </MultiTrigger.EnterActions>
                    <MultiTrigger.ExitActions>
                        <BeginStoryboard>
                            <Storyboard>
                                <DoubleAnimation
Storyboard.TargetProperty="RenderTransform.ScaleX"
                                    To="1"
                                    Duration="0:0:0.2"/>
                                <DoubleAnimation
Storyboard.TargetProperty="RenderTransform.ScaleY"
                                    To="1"
                                    Duration="0:0:0.2"/>
                            </Storyboard>
                        </BeginStoryboard>
                    </MultiTrigger.ExitActions>
                </MultiTrigger>
            </Style.Triggers>
        </Style>

```

```

        </MultiTrigger.ExitActions>
    </MultiTrigger>
</Style.Triggers>
</Style>

<!-- Стили для Editing -->
<Style x:Key="EditingPopupBorderStyle" TargetType="Border">
    <Setter Property="Background" Value="{StaticResource
PrimaryLightBrush}"/>
    <Setter Property="BorderBrush" Value="Black"/>
    <Setter Property="BorderThickness" Value="1"/>
    <Setter Property="Padding" Value="10"/>
</Style>

<Style x:Key="EditingPopupImageStyle" TargetType="Image">
    <Setter Property="Width" Value="20"/>
    <Setter Property="Height" Value="20"/>
    <Setter Property="HorizontalAlignment" Value="Right"/>
</Style>

<Style x:Key="EditingPopupTextBoxStyle" TargetType="TextBox">
    <Setter Property="Margin" Value="0 5 0 5"/>
</Style>

<Style x:Key="EditingPopupButtonStyle" TargetType="Button"
BasedOn="{StaticResource TransparentButtonStyle}">
    <Setter Property="Margin" Value="0 10 0 0"/>
    <Setter Property="Background" Value="{StaticResource
PrimaryBrush}"/>
    <Setter Property="Foreground" Value="white"/>
</Style>

<!-- Стили для Filtering -->
<Style x:Key="FilteringSearchBorderStyle" TargetType="Border">
    <Setter Property="Width" Value="300"/>
    <Setter Property="Height" Value="50"/>
    <Setter Property="Margin" Value="30 0 0 0"/>
    <Setter Property="CornerRadius" Value="10"/>
    <Setter Property="Background" Value="White"/>
    <Setter Property="BorderBrush" Value="Gray"/>
    <Setter Property="BorderThickness" Value="1"/>
    <Setter Property="Padding" Value="5"/>
</Style>

<Style x:Key="FilteringSearchTextBoxStyle" TargetType="TextBox">
    <Setter Property="BorderThickness" Value="0"/>
    <Setter Property="Background" Value="Transparent"/>
    <Setter Property="FontSize" Value="24"/>
</Style>

<Style x:Key="FilteringSearchImageStyle" TargetType="Image">
    <Setter Property="Width" Value="60"/>
    <Setter Property="Margin" Value="10 0 0 0"/>
</Style>

```

```

        <Style      x:Key="FilteringPopupButtonStyle"      TargetType="Button"
BasedOn="{StaticResource TransparentButtonStyle}">
            <Setter Property="MaxWidth" Value="150"/>
            <Setter Property="HorizontalAlignment" Value="Left"/>
            <Setter Property="VerticalAlignment" Value="Center"/>
            <Setter Property="FontSize" Value="12"/>
            <Setter      Property="Background"      Value="{StaticResource
PrimaryAnimBrush}"/>
            <Setter Property="Foreground" Value="white"/>
            <Setter Property="Padding" Value="5" />
        </Style>

        <Style x:Key="FilteringPopupBorderStyle" TargetType="Border">
            <Setter      Property="Background"      Value="{StaticResource
PrimaryBrush}"/>
            <Setter Property="CornerRadius" Value="5"/>
            <Setter Property="Padding" Value="10"/>
        </Style>

        <Style
                                x:Key="FilteringPopupTextBlockStyle"
TargetType="TextBlock">
            <Setter Property="Foreground" Value="White"/>
            <Setter Property="Margin" Value="5"/>
        </Style>

        <Style x:Key="FilteringPopupTextBoxStyle" TargetType="TextBox">
            <Setter Property="Width" Value="100"/>
            <Setter Property="Margin" Value="5"/>
        </Style>

        <Style x:Key="FilteringPopupApplyButtonStyle" TargetType="Button"
BasedOn="{StaticResource TransparentButtonStyle}">
            <Setter Property="HorizontalAlignment" Value="Right"/>
            <Setter Property="Margin" Value="5"/>
            <Setter Property="Foreground" Value="White"/>
            <Setter      Property="Background"      Value="{StaticResource
PrimaryBrush}"/>
        </Style>
</ResourceDictionary>

```

StyleTheme1.xaml

```

<ResourceDictionary
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"

xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
    <!-- Блок для темы -->

    <Color x:Key="PrimaryAdminLite">#8f8f8f</Color>
    <Color x:Key="PrimaryAdmin">#616161</Color>
    <Color x:Key="PrimaryBlack">#0c1338</Color>
    <Color x:Key="Primary">#2B59C3</Color>

```

```

    <Color x:Key="PrimaryLight">#7FD1B9</Color>
    <Color x:Key="PrimaryAnim">#EBBAB9</Color>
    <Color x:Key="InputError">#FF6B6B</Color>

    <ImageBrush x:Key="mainfon" ImageSource="/images/fon1.jpg"
Stretch="UniformToFill"/>
    <BitmapImage x:Key="DynamicGifSource"
UriSource="/images/animfon.gif"/>

    <SolidColorBrush x:Key="PrimaryAdminLiteBrush"
Color="{StaticResource PrimaryAdminLite}"/>
    <SolidColorBrush x:Key="PrimaryAdminBrush" Color="{StaticResource
PrimaryAdmin}"/>
    <SolidColorBrush x:Key="PrimaryBlackBrush" Color="{StaticResource
PrimaryBlack}"/>
    <SolidColorBrush x:Key="PrimaryBrush" Color="{StaticResource
Primary}"/>
    <SolidColorBrush x:Key="PrimaryLightBrush" Color="{StaticResource
PrimaryLight}"/>
    <SolidColorBrush x:Key="PrimaryAnimBrush" Color="{StaticResource
PrimaryAnim}"/>
    <SolidColorBrush x:Key="InputErrorBrush" Color="{StaticResource
InputError}"/>

    <!-- ToolTip -->
    <Style x:Key="CustomToolTip" TargetType="ToolTip">
        <Setter Property="BorderThickness" Value="0"/>
        <Setter Property="Background" Value="{StaticResource
PrimaryLightBrush}" />
        <Setter Property="Foreground" Value="Black" />
    </Style>

    <!-- Стили для всех кнопок -->
    <Style x:Key="TransparentButtonStyle" TargetType="Button">
        <Setter Property="Background" Value="Transparent" />
        <Setter Property="BorderBrush" Value="Transparent" />
        <Setter Property="Foreground" Value="Black" />
        <Setter Property="Padding" Value="0" />
        <Setter Property="Template">
            <Setter.Value>
                <ControlTemplate TargetType="Button">
                    <Border Background="{TemplateBinding Background}"
                        BorderBrush="{TemplateBinding BorderBrush}"
                        BorderThickness="{TemplateBinding
BorderThickness}"
                        Padding="{TemplateBinding Padding}">
                        <ContentPresenter
HorizontalAlignment="Center"
VerticalAlignment="Center" />
                    </Border>
                </ControlTemplate.Triggers>
            </ControlTemplate.Triggers>
        </ControlTemplate>
    </Setter.Value>

```

```

        </Setter>
    </Style>

    <!-- Стили для MainWindow -->
    <Style x:Key="RoundedBorderStyle" TargetType="Border">
        <Setter Property="CornerRadius" Value="10"/>
        <Setter Property="Background" Value="White"/>
        <Setter Property="BorderBrush" Value="Gray"/>
        <Setter Property="BorderThickness" Value="1"/>
        <Setter Property="Padding" Value="5"/>
    </Style>

    <Style x:Key="TransparentTextBoxStyle" TargetType="TextBox">
        <Setter Property="BorderThickness" Value="0"/>
        <Setter Property="Background" Value="Transparent"/>
        <Setter Property="FontSize" Value="24"/>
    </Style>

    <Style x:Key="TextBoxDataTrigger" TargetType="TextBox"
    BasedOn="{StaticResource TransparentTextBoxStyle}">
        <Style.Triggers>
            <DataTrigger Binding="{Binding Text.Length,
            RelativeSource={RelativeSource Self}}" Value="20">
                <Setter Property="Foreground" Value="#d92952"/>
            </DataTrigger>
        </Style.Triggers>
    </Style>

    <Style x:Key="LanguageButtonStyle" TargetType="Button"
    BasedOn="{StaticResource TransparentButtonStyle}">
        <Setter Property="Background" Value="{StaticResource
        PrimaryBrush}"/>
        <Setter Property="FontSize" Value="18"/>
        <Setter Property="FontWeight" Value="SemiBold"/>
        <Setter Property="Foreground" Value="White"/>
        <Setter Property="Padding" Value="10 5 10 5"/>
    </Style>

    <Style x:Key="AnimationLangBtn" TargetType="Button"
    BasedOn="{StaticResource LanguageButtonStyle}">
        <Setter Property="Margin" Value="10 0 0 0"/>
        <Setter Property="MaxHeight" Value="50"/>
        <Setter Property="Padding" Value="10 0 10 0"/>
        <Setter Property="RenderTransformOrigin" Value="0.5,0.5"/>
        <Setter Property="RenderTransform">
            <Setter.Value>
                <ScaleTransform ScaleX="1" ScaleY="1"/>
            </Setter.Value>
        </Setter>
        <Style.Triggers>
            <EventTrigger RoutedEvent="MouseEnter">
                <BeginStoryboard>
                    <Storyboard>
                        <DoubleAnimation

```

```

Storyboard.TargetProperty="RenderTransform.ScaleX"
    To="1.05"
    Duration="0:0:0.2"/>
<DoubleAnimation

Storyboard.TargetProperty="RenderTransform.ScaleY"
    To="1.05"
    Duration="0:0:0.2"/>
<DoubleAnimation
Storyboard.TargetProperty="Opacity"
    To="0.9"
    Duration="0:0:0.2"/>
<ColorAnimation

Storyboard.TargetProperty="(Panel.Background).(SolidColorBrush.Color)"
"
    To="{StaticResource PrimaryAnim}"
    Duration="0:0:0.2"/>
</Storyboard>
</BeginStoryboard>
</EventTrigger>
<EventTrigger RoutedEvent="MouseLeave">
    <BeginStoryboard>
        <Storyboard>
            <DoubleAnimation

Storyboard.TargetProperty="RenderTransform.ScaleX"
    To="1.0"
    Duration="0:0:0.2"/>
<DoubleAnimation

Storyboard.TargetProperty="RenderTransform.ScaleY"
    To="1.0"
    Duration="0:0:0.2"/>
<DoubleAnimation
Storyboard.TargetProperty="Opacity"
    To="1"
    Duration="0:0:0.2"/>
<ColorAnimation

Storyboard.TargetProperty="(Panel.Background).(SolidColorBrush.Color)"
"
    To="{StaticResource Primary}"
    Duration="0:0:0.2"/>
</Storyboard>
</BeginStoryboard>
</EventTrigger>
</Style.Triggers>
</Style>

<Style x:Key="UserButtonStyle" TargetType="Border">
    <Setter Property="Background" Value="{StaticResource
PrimaryBrush}"/>

```

```

        <Style.Triggers>
            <EventTrigger RoutedEvent="MouseEnter">
                <BeginStoryboard>
                    <Storyboard>
                        <DoubleAnimation
Storyboard.TargetProperty="Opacity" To="0.9" Duration="0:0:0.2"/>
                        <ColorAnimation
Storyboard.TargetProperty="Background.Color"
                                To="{StaticResource PrimaryAnim}"
                                Duration="0:0:0.2"/>
                    </Storyboard>
                </BeginStoryboard>
            </EventTrigger>
            <EventTrigger RoutedEvent="MouseLeave">
                <BeginStoryboard>
                    <Storyboard>
                        <DoubleAnimation
Storyboard.TargetProperty="Opacity" To="1" Duration="0:0:0.2"/>
                        <ColorAnimation
Storyboard.TargetProperty="Background.Color"
                                To="{StaticResource Primary}"
                                Duration="0:0:0.2"/>
                    </Storyboard>
                </BeginStoryboard>
            </EventTrigger>
        </Style.Triggers>
    </Style>

    <Style x:Key="UserStackPanelStyle" TargetType="StackPanel">
        <Setter Property="Orientation" Value="Horizontal"/>
        <Setter Property="Height" Value="50"/>
    </Style>

    <Style x:Key="UserNameTextBlockStyle" TargetType="TextBlock">
        <Setter Property="FontSize" Value="18"/>
        <Setter Property="FontWeight" Value="SemiBold"/>
        <Setter Property="Foreground" Value="White"/>
        <Setter Property="VerticalAlignment" Value="Center"/>
        <Setter Property="Margin" Value="10 0 10 0"/>
    </Style>

    <!-- Стили для AdminTable -->

    <Style x:Key="GridAdminStyle" TargetType="Grid">
        <Setter Property="Background" Value="{StaticResource
PrimaryAdminBrush}"/>
    </Style>
    <Style x:Key="GridDataAdminStyle" TargetType="DataGrid">
        <Setter Property="Background" Value="{StaticResource
PrimaryAdminBrush}"/>
        <Setter Property="RowBackground" Value="{StaticResource
PrimaryAdminLiteBrush}"/>
    </Style>

```

```

<!-- Стили для mapbooking -->

<Style x:Key="GridStyle" TargetType="Grid">
    <Setter Property="Background" Value="{StaticResource
PrimaryBlackBrush}"/>
</Style>
<Style x:Key="CanvasStyle" TargetType="Canvas">
    <Setter Property="Background" Value="{StaticResource
PrimaryBlackBrush}"/>
</Style>

<!-- Стили для AddItem -->
<Style x:Key="AddItemStackPanelStyle" TargetType="StackPanel">
    <Setter Property="MaxWidth" Value="300"/>
    <Setter Property="VerticalAlignment" Value="Center"/>
    <Setter Property="Background" Value="{StaticResource
PrimaryBrush}"/>
    <Setter Property="Opacity" Value="0.9"/>
</Style>

<Style x:Key="AddItemTextBlockStyle" TargetType="TextBlock">
    <Setter Property="Margin" Value="10 10 10 0"/>
    <Setter Property="Foreground" Value="White"/>
    <Setter Property="HorizontalAlignment" Value="Center"/>
</Style>

<Style x:Key="AddItemTextBoxStyle" TargetType="TextBox">
    <Setter Property="Margin" Value="10 10 10 0"/>
</Style>

<Style x:Key="AddItemButtonStyle" TargetType="Button"
BasedOn="{StaticResource TransparentButtonStyle}">
    <Setter Property="Margin" Value="10 10 10 10"/>
    <Setter Property="Background" Value="{StaticResource
PrimaryBrush}"/>
    <Setter Property="Foreground" Value="white"/>
</Style>

<!-- Стили для authorization -->
<Style x:Key="GridAuthorization" TargetType="Grid">
    <Setter Property="Background" Value="{StaticResource
PrimaryLightBrush}"/>
</Style>
<Style x:Key="AuthorizationBorderStyle" TargetType="Border">
    <Setter Property="MaxWidth" Value="200"/>
    <Setter Property="Height" Value="Auto"/>
</Style>

<Style x:Key="PersonalBorderStyle" TargetType="Border">
    <Setter Property="MaxWidth" Value="280"/>
    <Setter Property="Height" Value="Auto"/>
</Style>

```

```

        <Style                                x:Key="AuthorizationStackPanelStyle"
TargetType="StackPanel">
            <Setter Property="VerticalAlignment" Value="Center"/>
        </Style>

        <Style x:Key="AuthorizationTextBlockStyle" TargetType="TextBlock">
            <Setter Property="FontWeight" Value="SemiBold"/>
            <Setter Property="FontSize" Value="14"/>
            <Setter Property="HorizontalAlignment" Value="Center"/>
        </Style>

        <Style                                x:Key="AuthorizationLabelTextBlockStyle"
TargetType="TextBlock">
            <Setter Property="FontWeight" Value="SemiBold"/>
            <Setter Property="Margin" Value="10 0 0 5"/>
        </Style>

        <Style x:Key="AuthorizationInputBorderStyle" TargetType="Border">
            <Setter Property="CornerRadius" Value="6"/>
            <Setter Property="Background" Value="White"/>
            <Setter Property="Height" Value="20"/>
            <Setter Property="Margin" Value="10 0 10 10"/>
        </Style>

        <Style x:Key="AuthorizationTextBoxStyle" TargetType="TextBox">
            <Setter Property="BorderThickness" Value="0"/>
            <Setter Property="Background" Value="Transparent"/>
            <Setter Property="FontSize" Value="14"/>
        </Style>

        <Style                                x:Key="AuthorizationPasswordBoxStyle"
TargetType="PasswordBox">
            <Setter Property="BorderThickness" Value="0"/>
            <Setter Property="Background" Value="Transparent"/>
            <Setter Property="FontSize" Value="14"/>
        </Style>

        <Style      x:Key="AuthorizationButtonStyle"      TargetType="Button"
BasedOn="{StaticResource TransparentButtonStyle}">
            <Setter Property="FontWeight" Value="SemiBold"/>
            <Setter      Property="Background"      Value="{StaticResource
PrimaryBrush}"/>
            <Setter Property="Margin" Value="10 0 10 10"/>
        </Style>

        <Style TargetType="Button" x:Key="AuthorizationButtonStyleTrigger"
BasedOn="{StaticResource AuthorizationButtonStyle}">
            <Setter      Property="Background"      Value="{StaticResource
PrimaryBrush}"/>
            <Setter Property="RenderTransformOrigin" Value="0.5,0.5"/>
            <Setter Property="RenderTransform">
                <Setter.Value>
                    <ScaleTransform ScaleX="1" ScaleY="1"/>
                </Setter.Value>
            </Setter>
        </Style>

```

```

        </Setter.Value>
    </Setter>

    <Style.Triggers>
        <MultiTrigger>
            <MultiTrigger.Conditions>
                <Condition Property="IsMouseOver" Value="True"/>
            </MultiTrigger.Conditions>
            <MultiTrigger.EnterActions>
                <BeginStoryboard>
                    <Storyboard>
                        <DoubleAnimation
Storyboard.TargetProperty="RenderTransform.ScaleX"
                        To="1.03"
                        Duration="0:0:0.2"/>
                        <DoubleAnimation
Storyboard.TargetProperty="RenderTransform.ScaleY"
                        To="1.15"
                        Duration="0:0:0.2"/>
                    </Storyboard>
                </BeginStoryboard>
            </MultiTrigger.EnterActions>
            <MultiTrigger.ExitActions>
                <BeginStoryboard>
                    <Storyboard>
                        <DoubleAnimation
Storyboard.TargetProperty="RenderTransform.ScaleX"
                        To="1"
                        Duration="0:0:0.2"/>
                        <DoubleAnimation
Storyboard.TargetProperty="RenderTransform.ScaleY"
                        To="1"
                        Duration="0:0:0.2"/>
                    </Storyboard>
                </BeginStoryboard>
            </MultiTrigger.ExitActions>
        </MultiTrigger>
    </Style.Triggers>
</Style>

<!-- Стили для Editing -->
<Style x:Key="EditingPopupBorderStyle" TargetType="Border">
    <Setter Property="Background" Value="{StaticResource
PrimaryLightBrush}"/>
    <Setter Property="BorderBrush" Value="Black"/>
    <Setter Property="BorderThickness" Value="1"/>
    <Setter Property="Padding" Value="10"/>
</Style>

<Style x:Key="EditingPopupImageStyle" TargetType="Image">

```

```

        <Setter Property="Width" Value="20"/>
        <Setter Property="Height" Value="20"/>
        <Setter Property="HorizontalAlignment" Value="Right"/>
    </Style>

    <Style x:Key="EditingPopupTextBoxStyle" TargetType="TextBox">
        <Setter Property="Margin" Value="0 5 0 5"/>
    </Style>

    <Style x:Key="EditingPopupButtonStyle" TargetType="Button"
    BasedOn="{StaticResource TransparentButtonStyle}">
        <Setter Property="Margin" Value="0 10 0 0"/>
        <Setter Property="Background" Value="{StaticResource
PrimaryBrush}"/>
        <Setter Property="Foreground" Value="white"/>
    </Style>

    <!-- Стили для Filtering -->
    <Style x:Key="FilteringSearchBorderStyle" TargetType="Border">
        <Setter Property="Width" Value="300"/>
        <Setter Property="Height" Value="50"/>
        <Setter Property="Margin" Value="30 0 0 0"/>
        <Setter Property="CornerRadius" Value="10"/>
        <Setter Property="Background" Value="White"/>
        <Setter Property="BorderBrush" Value="Gray"/>
        <Setter Property="BorderThickness" Value="1"/>
        <Setter Property="Padding" Value="5"/>
    </Style>

    <Style x:Key="FilteringSearchTextBoxStyle" TargetType="TextBox">
        <Setter Property="BorderThickness" Value="0"/>
        <Setter Property="Background" Value="Transparent"/>
        <Setter Property="FontSize" Value="24"/>
    </Style>

    <Style x:Key="FilteringSearchImageStyle" TargetType="Image">
        <Setter Property="Width" Value="60"/>
        <Setter Property="Margin" Value="10 0 0 0"/>
    </Style>

    <Style x:Key="FilteringPopupButtonStyle" TargetType="Button"
    BasedOn="{StaticResource TransparentButtonStyle}">
        <Setter Property="MaxWidth" Value="150"/>
        <Setter Property="HorizontalAlignment" Value="Left"/>
        <Setter Property="VerticalAlignment" Value="Center"/>
        <Setter Property="FontSize" Value="12"/>
        <Setter Property="Background" Value="{StaticResource
PrimaryAnimBrush}"/>
        <Setter Property="Foreground" Value="white"/>
        <Setter Property="Padding" Value="5" />
    </Style>

    <Style x:Key="FilteringPopupBorderStyle" TargetType="Border">

```

```

        <Setter      Property="Background"      Value="{StaticResource
PrimaryBrush}"/>
        <Setter Property="CornerRadius" Value="5"/>
        <Setter Property="Padding" Value="10"/>
    </Style>

    <Style                                x:Key="FilteringPopupTextBlockStyle"
TargetType="TextBlock">
        <Setter Property="Foreground" Value="White"/>
        <Setter Property="Margin" Value="5"/>
    </Style>

    <Style x:Key="FilteringPopupTextBoxStyle" TargetType="TextBox">
        <Setter Property="Width" Value="100"/>
        <Setter Property="Margin" Value="5"/>
    </Style>

    <Style x:Key="FilteringPopupApplyButtonStyle" TargetType="Button"
BasedOn="{StaticResource TransparentButtonStyle}">
        <Setter Property="HorizontalAlignment" Value="Right"/>
        <Setter Property="Margin" Value="5"/>
        <Setter Property="Foreground" Value="White"/>
        <Setter      Property="Background"      Value="{StaticResource
PrimaryBrush}"/>
    </Style>

    <!-- Стили для ThemeButton -->
    <ControlTemplate x:Key="ThemeButtonTemplate" TargetType="Button">
        <Grid>
            <Path x:Name="hexagon"
                Fill="{TemplateBinding Background}"
                Stroke="{TemplateBinding BorderBrush}"
                StrokeThickness="{TemplateBinding BorderThickness}">
                <Path.Data>
                    <PathGeometry>
                        <PathFigure StartPoint="12,0">
                            <LineSegment Point="140,0"/>
                            <LineSegment Point="140,33.33"/>
                            <LineSegment Point="128,50"/>
                            <LineSegment Point="0,50"/>
                            <LineSegment Point="0,16"/>
                            <LineSegment Point="12,0"/>
                        </PathFigure>
                    </PathGeometry>
                </Path.Data>
            </Path>

            <ContentPresenter HorizontalAlignment="Center"
                VerticalAlignment="Center"/>
        </Grid>
    </ControlTemplate>

    <!-- Стили для LangButton -->
    <ControlTemplate x:Key="LangButtonTemplate" TargetType="Button">

```

```

<Grid>
    <Path x:Name="hexagon"
        Fill="{TemplateBinding Background}"
        Stroke="{TemplateBinding BorderBrush}"
        StrokeThickness="{TemplateBinding BorderThickness}">
        <Path.Data>
            <PathGeometry>
                <PathFigure StartPoint="12,0">
                    <LineSegment Point="90,0"/>
                    <LineSegment Point="90,33.33"/>
                    <LineSegment Point="78,50"/>
                    <LineSegment Point="0,50"/>
                    <LineSegment Point="0,16"/>
                    <LineSegment Point="12,0"/>
                </PathFigure>
            </PathGeometry>
        </Path.Data>
    </Path>

    <ContentPresenter HorizontalAlignment="Center"
        VerticalAlignment="Center"/>
</Grid>
</ControlTemplate>

<!-- Стили для WritingReview -->

<Style x:Key="WritingReviewFon" TargetType="Grid">
    <Setter Property="Background" Value="{StaticResource
PrimaryLightBrush}"/>
</Style>

<ControlTemplate x:Key="SaveReviewButtonTemplate"
TargetType="Button">
    <Grid>
        <Path x:Name="hexagon"
            Fill="{TemplateBinding Background}"
            Stroke="{TemplateBinding BorderBrush}"
            StrokeThickness="{TemplateBinding BorderThickness}">
            <Path.Data>
                <PathGeometry>
                    <PathFigure StartPoint="18,0">
                        <LineSegment Point="135,0"/>
                        <LineSegment Point="135,50"/>
                        <LineSegment Point="117,75"/>
                        <LineSegment Point="0,75"/>
                        <LineSegment Point="0,24"/>
                        <LineSegment Point="18,0"/>
                    </PathFigure>
                </PathGeometry>
            </Path.Data>
        </Path>

        <ContentPresenter HorizontalAlignment="Center"
            VerticalAlignment="Center"/>
    </Grid>
</ControlTemplate>

```

```

        </Grid>
    </ControlTemplate>

</ResourceDictionary>

```

StyleTheme2.xaml

```

<ResourceDictionary
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"

    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
    <!-- Блок для темы -->

    <Color x:Key="PrimaryAdminLite">#dcdedc</Color>
    <Color x:Key="PrimaryAdmin">#dcdedc</Color>
    <Color x:Key="PrimaryBlack">#1a2b19</Color>
    <Color x:Key="Primary">#109521</Color>
    <Color x:Key="PrimaryLight">#52af75</Color>
    <Color x:Key="PrimaryAnim">#1bcc32</Color>
    <Color x:Key="InputError">#d92952</Color>

    <ImageBrush x:Key="mainfon" ImageSource="/images/fon.jpg"
        Stretch="UniformToFill"/>
    <BitmapImage x:Key="DynamicGifSource"
        UriSource="/images/newgifka.gif"/>

    <SolidColorBrush x:Key="PrimaryAdminLiteBrush"
        Color="{StaticResource PrimaryAdminLite}"/>
    <SolidColorBrush x:Key="PrimaryAdminBrush" Color="{StaticResource
        PrimaryAdmin}"/>
    <SolidColorBrush x:Key="PrimaryBlackBrush" Color="{StaticResource
        PrimaryBlack}"/>
    <SolidColorBrush x:Key="PrimaryBrush" Color="{StaticResource
        Primary}"/>
    <SolidColorBrush x:Key="PrimaryLightBrush" Color="{StaticResource
        PrimaryLight}"/>
    <SolidColorBrush x:Key="PrimaryAnimBrush" Color="{StaticResource
        PrimaryAnim}"/>
    <SolidColorBrush x:Key="InputErrorBrush" Color="{StaticResource
        InputError}"/>

    <!-- ToolTip -->
    <Style x:Key="CustomToolTip" TargetType="ToolTip">
        <Setter Property="BorderThickness" Value="0"/>
        <Setter Property="Background" Value="{StaticResource
        PrimaryLightBrush}" />
        <Setter Property="Foreground" Value="Black" />
    </Style>

    <!-- Стили для всех кнопок -->
    <Style x:Key="TransparentButtonStyle" TargetType="Button">
        <Setter Property="Background" Value="Transparent" />
        <Setter Property="BorderBrush" Value="Transparent" />
        <Setter Property="Foreground" Value="Black" />
    </Style>

```

```

        <Setter Property="Padding" Value="0" />
        <Setter Property="Template">
            <Setter.Value>
                <ControlTemplate TargetType="Button">
                    <Border Background="{TemplateBinding Background}"
                        BorderBrush="{TemplateBinding BorderBrush}"
                        BorderThickness="{TemplateBinding
BorderThickness}"
                            Padding="{TemplateBinding Padding}"
                                >
                                    <ContentPresenter
HorizontalAlignment="Center"
VerticalAlignment="Center" />
                                </Border>
                            <ControlTemplate.Triggers>
                                </ControlTemplate.Triggers>
                            </ControlTemplate>
                        </Setter.Value>
                    </Setter>
                </Style>

<!-- Стили для MainWindow -->
<Style x:Key="RoundedBorderStyle" TargetType="Border">
    <Setter Property="CornerRadius" Value="10"/>
    <Setter Property="Background" Value="White"/>
    <Setter Property="BorderBrush" Value="Gray"/>
    <Setter Property="BorderThickness" Value="1"/>
    <Setter Property="Padding" Value="5"/>
</Style>

<Style x:Key="TransparentTextBoxStyle" TargetType="TextBox">
    <Setter Property="BorderThickness" Value="0"/>
    <Setter Property="Background" Value="Transparent"/>
    <Setter Property="FontSize" Value="24"/>
</Style>

<Style
    x:Key="TextBoxDataTrigger"
    TargetType="TextBox"
BasedOn="{StaticResource TransparentTextBoxStyle}">
    <Style.Triggers>
        <DataTrigger
            Binding="{Binding
RelativeSource={RelativeSource Self}}" Value="20">
            <Setter Property="Foreground" Value="#d92952"/>
        </DataTrigger>
    </Style.Triggers>
</Style>

<Style
    x:Key="LanguageButtonStyle"
    TargetType="Button"
BasedOn="{StaticResource TransparentButtonStyle}">
    <Setter
        Property="Background"
        Value="{StaticResource
PrimaryBrush}" />
    <Setter Property="FontSize" Value="18"/>
    <Setter Property="FontWeight" Value="SemiBold"/>
    <Setter Property="Foreground" Value="White"/>
    <Setter Property="Padding" Value="10 5 10 5"/>

```

```

</Style>

<Style x:Key="AnimationLangBtn" TargetType="Button"
BasedOn="{StaticResource LanguageButtonStyle}">
    <Setter Property="Margin" Value="10 0 0 0"/>
    <Setter Property="MaxHeight" Value="50"/>
    <Setter Property="Padding" Value="10 0 10 0"/>
    <Setter Property="RenderTransformOrigin" Value="0.5,0.5"/>
    <Setter Property="RenderTransform">
        <Setter.Value>
            <ScaleTransform ScaleX="1" ScaleY="1"/>
        </Setter.Value>
    </Setter>
    <Style.Triggers>
        <EventTrigger RoutedEvent="MouseEnter">
            <BeginStoryboard>
                <Storyboard>
                    <DoubleAnimation
Storyboard.TargetProperty="RenderTransform.ScaleX"
                        To="1.05"
                        Duration="0:0:0.2"/>
                    <DoubleAnimation
Storyboard.TargetProperty="RenderTransform.ScaleY"
                        To="1.05"
                        Duration="0:0:0.2"/>
                    <DoubleAnimation
Storyboard.TargetProperty="Opacity"
                        To="0.9"
                        Duration="0:0:0.2"/>
                    <ColorAnimation
Storyboard.TargetProperty="(Panel.Background).(SolidColorBrush.Color)"
                        To="{StaticResource PrimaryAnim}"
                        Duration="0:0:0.2"/>
                </Storyboard>
            </BeginStoryboard>
        </EventTrigger>
        <EventTrigger RoutedEvent="MouseLeave">
            <BeginStoryboard>
                <Storyboard>
                    <DoubleAnimation
Storyboard.TargetProperty="RenderTransform.ScaleX"
                        To="1.0"
                        Duration="0:0:0.2"/>
                    <DoubleAnimation
Storyboard.TargetProperty="RenderTransform.ScaleY"
                        To="1.0"
                        Duration="0:0:0.2"/>

```

```

Storyboard.TargetProperty="Opacity"
To="1"
Duration="0:0:0.2"/>
<ColorAnimation

Storyboard.TargetProperty="(Panel.Background).(SolidColorBrush.Color)"
"
    To="{StaticResource Primary}"
    Duration="0:0:0.2"/>
    </Storyboard>
  </BeginStoryboard>
</EventTrigger>
</Style.Triggers>
</Style>

<Style x:Key="UserButtonStyle" TargetType="Border">
  <Setter Property="Background" Value="{StaticResource
PrimaryBrush}"/>
  <Style.Triggers>
    <EventTrigger RoutedEvent="MouseEnter">
      <BeginStoryboard>
        <Storyboard>
          <DoubleAnimation
Storyboard.TargetProperty="Opacity" To="0.9" Duration="0:0:0.2"/>
          <ColorAnimation
Storyboard.TargetProperty="Background.Color"
            To="{StaticResource PrimaryAnim}"
            Duration="0:0:0.2"/>
        </Storyboard>
      </BeginStoryboard>
    </EventTrigger>
    <EventTrigger RoutedEvent="MouseLeave">
      <BeginStoryboard>
        <Storyboard>
          <DoubleAnimation
Storyboard.TargetProperty="Opacity" To="1" Duration="0:0:0.2"/>
          <ColorAnimation
Storyboard.TargetProperty="Background.Color"
            To="{StaticResource Primary}"
            Duration="0:0:0.2"/>
        </Storyboard>
      </BeginStoryboard>
    </EventTrigger>
  </Style.Triggers>
</Style>

<Style x:Key="UserStackPanelStyle" TargetType="StackPanel">
  <Setter Property="Orientation" Value="Horizontal"/>
  <Setter Property="Height" Value="50"/>
</Style>

<Style x:Key="UserNameTextBlockStyle" TargetType="TextBlock">
  <Setter Property="FontSize" Value="18"/>

```

```

        <Setter Property="FontWeight" Value="SemiBold"/>
        <Setter Property="Foreground" Value="White"/>
        <Setter Property="VerticalAlignment" Value="Center"/>
        <Setter Property="Margin" Value="10 0 10 0"/>
    </Style>

    <!-- Стили для AdminTable -->

    <Style x:Key="GridAdminStyle" TargetType="Grid">
        <Setter Property="Background" Value="{StaticResource
PrimaryAdminBrush}"/>
    </Style>
    <Style x:Key="GridDataAdminStyle" TargetType="DataGrid">
        <Setter Property="Background" Value="{StaticResource
PrimaryAdminBrush}"/>
        <Setter Property="RowBackground" Value="{StaticResource
PrimaryAdminLiteBrush}"/>
    </Style>

    <!-- Стили для mapbooking -->

    <Style x:Key="GridStyle" TargetType="Grid">
        <Setter Property="Background" Value="{StaticResource
PrimaryBlackBrush}"/>
    </Style>
    <Style x:Key="CanvasStyle" TargetType="Canvas">
        <Setter Property="Background" Value="{StaticResource
PrimaryBlackBrush}"/>
    </Style>

    <!-- Стили для AddItem -->
    <Style x:Key="AddItemStackPanelStyle" TargetType="StackPanel">
        <Setter Property="MaxWidth" Value="300"/>
        <Setter Property="VerticalAlignment" Value="Center"/>
        <Setter Property="Background" Value="{StaticResource
PrimaryBrush}"/>
        <Setter Property="Opacity" Value="0.9"/>
    </Style>

    <Style x:Key="AddItemTextBlockStyle" TargetType="TextBlock">
        <Setter Property="Margin" Value="10 10 10 0"/>
        <Setter Property="Foreground" Value="White"/>
        <Setter Property="HorizontalAlignment" Value="Center"/>
    </Style>

    <Style x:Key="AddItemTextBoxStyle" TargetType="TextBox">
        <Setter Property="Margin" Value="10 10 10 0"/>
    </Style>

    <Style x:Key="AddItemButtonStyle" TargetType="Button"
BasedOn="{StaticResource TransparentButtonStyle}">
        <Setter Property="Margin" Value="10 10 10 10"/>
        <Setter Property="Background" Value="{StaticResource
PrimaryBrush}"/>

```

```

        <Setter Property="Foreground" Value="white"/>
    </Style>

    <!-- Стили для authorization -->
    <Style x:Key="GridAuthorization" TargetType="Grid">
        <Setter Property="Background" Value="{StaticResource
PrimaryLightBrush}"/>
    </Style>
    <Style x:Key="AuthorizationBorderStyle" TargetType="Border">
        <Setter Property="MaxWidth" Value="200"/>
        <Setter Property="Height" Value="Auto"/>
    </Style>
    <Style x:Key="PersonalBorderStyle" TargetType="Border">
        <Setter Property="MaxWidth" Value="280"/>
        <Setter Property="Height" Value="Auto"/>
    </Style>

    <Style x:Key="AuthorizationStackPanelStyle"
TargetType="StackPanel">
        <Setter Property="VerticalAlignment" Value="Center"/>
    </Style>

    <Style x:Key="AuthorizationTextBlockStyle" TargetType="TextBlock">
        <Setter Property="FontWeight" Value="SemiBold"/>
        <Setter Property="FontSize" Value="14"/>
        <Setter Property="HorizontalAlignment" Value="Center"/>
    </Style>

    <Style x:Key="AuthorizationLabelTextBlockStyle"
TargetType="TextBlock">
        <Setter Property="FontWeight" Value="SemiBold"/>
        <Setter Property="Margin" Value="10 0 0 5"/>
    </Style>

    <Style x:Key="AuthorizationInputBorderStyle" TargetType="Border">
        <Setter Property="CornerRadius" Value="6"/>
        <Setter Property="Background" Value="White"/>
        <Setter Property="Height" Value="20"/>
        <Setter Property="Margin" Value="10 0 10 10"/>
    </Style>

    <Style x:Key="AuthorizationTextBoxStyle" TargetType="TextBox">
        <Setter Property="BorderThickness" Value="0"/>
        <Setter Property="Background" Value="Transparent"/>
        <Setter Property="FontSize" Value="14"/>
    </Style>

    <Style x:Key="AuthorizationPasswordBoxStyle"
TargetType="PasswordBox">
        <Setter Property="BorderThickness" Value="0"/>
        <Setter Property="Background" Value="Transparent"/>
        <Setter Property="FontSize" Value="14"/>
    </Style>

```

```

        <Style      x:Key="AuthorizationButtonStyle"      TargetType="Button"
BasedOn="{StaticResource TransparentButtonStyle}">
            <Setter Property="FontWeight" Value="SemiBold"/>
            <Setter      Property="Background"      Value="{StaticResource
PrimaryBrush}"/>
            <Setter Property="Margin" Value="10 0 10 10"/>
        </Style>

        <Style TargetType="Button" x:Key="AuthorizationButtonStyleTrigger"
BasedOn="{StaticResource AuthorizationButtonStyle}">
            <Setter      Property="Background"      Value="{StaticResource
PrimaryBrush}"/>
            <Setter Property="RenderTransformOrigin" Value="0.5,0.5"/>
            <Setter Property="RenderTransform">
                <Setter.Value>
                    <ScaleTransform ScaleX="1" ScaleY="1"/>
                </Setter.Value>
            </Setter>

            <Style.Triggers>
                <MultiTrigger>
                    <MultiTrigger.Conditions>
                        <Condition Property="IsMouseOver" Value="True"/>
                    </MultiTrigger.Conditions>
                    <MultiTrigger.EnterActions>
                        <BeginStoryboard>
                            <Storyboard>
                                <DoubleAnimation

Storyboard.TargetProperty="RenderTransform.ScaleX"
                                To="1.03"
                                Duration="0:0:0.2"/>
                                <DoubleAnimation

Storyboard.TargetProperty="RenderTransform.ScaleY"
                                To="1.15"
                                Duration="0:0:0.2"/>
                            </Storyboard>
                        </BeginStoryboard>
                    </MultiTrigger.EnterActions>
                    <MultiTrigger.ExitActions>
                        <BeginStoryboard>
                            <Storyboard>
                                <DoubleAnimation

Storyboard.TargetProperty="RenderTransform.ScaleX"
                                To="1"
                                Duration="0:0:0.2"/>
                                <DoubleAnimation

Storyboard.TargetProperty="RenderTransform.ScaleY"
                                To="1"
                                Duration="0:0:0.2"/>
                            </Storyboard>
                        </BeginStoryboard>
                    </MultiTrigger.ExitActions>
                </MultiTrigger>
            </Style.Triggers>
        </Style>

```

```

        </Storyboard>
    </BeginStoryboard>
    </MultiTrigger.ExitActions>
</MultiTrigger>
</Style.Triggers>
</Style>

<!-- Стили для Editing -->
<Style x:Key="EditingPopupBorderStyle" TargetType="Border">
    <Setter Property="Background" Value="{StaticResource
PrimaryLightBrush}"/>
    <Setter Property="BorderBrush" Value="Black"/>
    <Setter Property="BorderThickness" Value="1"/>
    <Setter Property="Padding" Value="10"/>
</Style>

<Style x:Key="EditingPopupImageStyle" TargetType="Image">
    <Setter Property="Width" Value="20"/>
    <Setter Property="Height" Value="20"/>
    <Setter Property="HorizontalAlignment" Value="Right"/>
</Style>

<Style x:Key="EditingPopupTextBoxStyle" TargetType="TextBox">
    <Setter Property="Margin" Value="0 5 0 5"/>
</Style>

<Style x:Key="EditingPopupButtonStyle" TargetType="Button"
BasedOn="{StaticResource TransparentButtonStyle}">
    <Setter Property="Margin" Value="0 10 0 0"/>
    <Setter Property="Background" Value="{StaticResource
PrimaryBrush}"/>
    <Setter Property="Foreground" Value="white"/>
</Style>

<!-- Стили для Filtering -->
<Style x:Key="FilteringSearchBorderStyle" TargetType="Border">
    <Setter Property="Width" Value="300"/>
    <Setter Property="Height" Value="50"/>
    <Setter Property="Margin" Value="30 0 0 0"/>
    <Setter Property="CornerRadius" Value="10"/>
    <Setter Property="Background" Value="White"/>
    <Setter Property="BorderBrush" Value="Gray"/>
    <Setter Property="BorderThickness" Value="1"/>
    <Setter Property="Padding" Value="5"/>
</Style>

<Style x:Key="FilteringSearchTextBoxStyle" TargetType="TextBox">
    <Setter Property="BorderThickness" Value="0"/>
    <Setter Property="Background" Value="Transparent"/>
    <Setter Property="FontSize" Value="24"/>
</Style>

<Style x:Key="FilteringSearchImageStyle" TargetType="Image">
    <Setter Property="Width" Value="60"/>

```

```

        <Setter Property="Margin" Value="10 0 0 0"/>
    </Style>

    <Style      x:Key="FilteringPopupButtonStyle"      TargetType="Button"
BasedOn="{StaticResource TransparentButtonStyle}">
        <Setter Property="MaxWidth" Value="150"/>
        <Setter Property="HorizontalAlignment" Value="Left"/>
        <Setter Property="VerticalAlignment" Value="Center"/>
        <Setter Property="FontSize" Value="12"/>
        <Setter      Property="Background"      Value="{StaticResource
PrimaryAnimBrush}"/>
        <Setter Property="Foreground" Value="white"/>
        <Setter Property="Padding" Value="5" />
    </Style>

    <Style x:Key="FilteringPopupBorderStyle" TargetType="Border">
        <Setter      Property="Background"      Value="{StaticResource
PrimaryBrush}"/>
        <Setter Property="CornerRadius" Value="5"/>
        <Setter Property="Padding" Value="10"/>
    </Style>

    <Style
                                x:Key="FilteringPopupTextBlockStyle"
TargetType="TextBlock">
        <Setter Property="Foreground" Value="White"/>
        <Setter Property="Margin" Value="5"/>
    </Style>

    <Style x:Key="FilteringPopupTextBoxStyle" TargetType="TextBox">
        <Setter Property="Width" Value="100"/>
        <Setter Property="Margin" Value="5"/>
    </Style>

    <Style x:Key="FilteringPopupApplyButtonStyle" TargetType="Button"
BasedOn="{StaticResource TransparentButtonStyle}">
        <Setter Property="HorizontalAlignment" Value="Right"/>
        <Setter Property="Margin" Value="5"/>
        <Setter Property="Foreground" Value="White"/>
        <Setter      Property="Background"      Value="{StaticResource
PrimaryBrush}"/>
    </Style>

    <!-- Стили для ThemeButton -->
    <ControlTemplate x:Key="ThemeButtonTemplate" TargetType="Button">
        <Grid>
            <Path x:Name="hexagon"
                Fill="{TemplateBinding Background}"
                Stroke="{TemplateBinding BorderBrush}"
                StrokeThickness="{TemplateBinding BorderThickness}">
                <Path.Data>
                    <PathGeometry>
                        <PathFigure StartPoint="12,0">
                            <LineSegment Point="140,0"/>
                            <LineSegment Point="140,33.33"/>

```

```

        <LineSegment Point="128,50"/>
        <LineSegment Point="0,50"/>
        <LineSegment Point="0,16"/>
        <LineSegment Point="12,0"/>
    </PathFigure>
</PathGeometry>
</Path.Data>
</Path>

    <ContentPresenter HorizontalAlignment="Center"
        VerticalAlignment="Center"/>
</Grid>
</ControlTemplate>

<!-- Стили для LangButton -->
<ControlTemplate x:Key="LangButtonTemplate" TargetType="Button">
    <Grid>
        <Path x:Name="hexagon"
            Fill="{TemplateBinding Background}"
            Stroke="{TemplateBinding BorderBrush}"
            StrokeThickness="{TemplateBinding BorderThickness}">
            <Path.Data>
                <PathGeometry>
                    <PathFigure StartPoint="12,0">
                        <LineSegment Point="90,0"/>
                        <LineSegment Point="90,33.33"/>
                        <LineSegment Point="78,50"/>
                        <LineSegment Point="0,50"/>
                        <LineSegment Point="0,16"/>
                        <LineSegment Point="12,0"/>
                    </PathFigure>
                </PathGeometry>
            </Path.Data>
        </Path>

        <ContentPresenter HorizontalAlignment="Center"
            VerticalAlignment="Center"/>
    </Grid>
</ControlTemplate>

<!-- Стили для WritingReview -->

<Style x:Key="WritingReviewFon" TargetType="Grid">
    <Setter Property="Background" Value="{StaticResource
PrimaryLightBrush}"/>
</Style>

    <ControlTemplate x:Key="SaveReviewButtonTemplate"
TargetType="Button">
    <Grid>
        <Path x:Name="hexagon"
            Fill="{TemplateBinding Background}"
            Stroke="{TemplateBinding BorderBrush}"
            StrokeThickness="{TemplateBinding BorderThickness}">

```

```

        <Path.Data>
            <PathGeometry>
                <PathFigure StartPoint="18,0">
                    <LineSegment Point="135,0"/>
                    <LineSegment Point="135,50"/>
                    <LineSegment Point="117,75"/>
                    <LineSegment Point="0,75"/>
                    <LineSegment Point="0,24"/>
                    <LineSegment Point="18,0"/>
                </PathFigure>
            </PathGeometry>
        </Path.Data>
    </Path>

    <ContentPresenter HorizontalAlignment="Center"
        VerticalAlignment="Center"/>
</Grid>
</ControlTemplate>

</ResourceDictionary>

```

AdminTable.xaml

```

<Window x:Class="COURSEPROJECT.AdminTable"

xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006"
    xmlns:local="clr-namespace:COURSEPROJECT"
    mc:Ignorable="d"
    WindowStartupLocation="CenterScreen"
    Title="{DynamicResource Projectdatamanagement}" Height="720"
Width="1280">
    <Grid Style="{DynamicResource GridAdminStyle}">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*" />
        </Grid.RowDefinitions>
        <StackPanel Orientation="Horizontal" Grid.Row="0">
            <ComboBox x:Name="TableBox" Width="200" SelectedIndex="1">
                <ComboBoxItem>Users</ComboBoxItem>
                <ComboBoxItem>Computers</ComboBoxItem>
                <ComboBoxItem>Orders</ComboBoxItem>
                <ComboBoxItem>Reviews</ComboBoxItem>
                <ComboBoxItem>ApplicationGame</ComboBoxItem>
            </ComboBox>
            <TextBox Width="100" x:Name="inputuser">
            </TextBox>
            <TextBox Width="100" x:Name="inputcomputer">
            </TextBox>
            <ComboBox x:Name="TableParam" Width="200">

```

```

        <ComboBoxItem x:Name="comp1" Content="{DynamicResource
Powerfulprocessors}"></ComboBoxItem>
        <ComboBoxItem x:Name="comp2" Content="{DynamicResource
Powerfulgraphicscards}"></ComboBoxItem>
        <ComboBoxItem x:Name="comp3" Content="{DynamicResource
Expensive}"></ComboBoxItem>
        <ComboBoxItem x:Name="ord1" Content="{DynamicResource
Completed}"></ComboBoxItem>
        <ComboBoxItem x:Name="ord2" Content="{DynamicResource
Active}"></ComboBoxItem>
        <ComboBoxItem x:Name="ord3" Content="{DynamicResource
Cancelled}"></ComboBoxItem>
    </ComboBox>
    <Button Grid.Row="0"
        Content="{DynamicResource Adding}"
        Width="150" Height="30"/>
    <Button Grid.Row="0"
        Content="{DynamicResource Editing}"
        Width="150" Height="30" Click="GetUpdateRows"/>
    <Button Grid.Row="0"
        Content="{DynamicResource Deleting}"
        Width="150"
        Height="30"
Click="DeleteRowButton_Click"/>
</StackPanel>
<DataGrid Style="{DynamicResource GridDataAdminStyle}"
    Grid.Row="1"
    x:Name="dataGrid"
    AutoGenerateColumns="True"
    CanUserAddRows="True"
    CanUserDeleteRows="True"
    CanUserReorderColumns="True"
    IsReadOnly="False"
    RowBackground="White"/>
</Grid>
</Window>

```

AdminTable.xaml.cs

```

using Microsoft.Data.Sqlite;
using System;
using System.Collections.Generic;
using System.Collections.ObjectModel;
using System.Data;
using System.Globalization;
using System.IO;
using System.Linq;
using System.Text;
using System.Text.RegularExpressions;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;

```

```

using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Shapes;
using System.Xml.Linq;
using System.Diagnostics;

namespace COURSEPROJECT
{
    /// <summary>
    /// Логика взаимодействия для AdminTable.xaml
    /// </summary>
    public partial class AdminTable : Window
    {
        private string TableInWin = "Computers";

        public AdminTable()
        {
            InitializeComponent();
            // асинхронно загружаем таблицу
            _ = LoadComputersAsync();
            TableBox.SelectionChanged += async (s, e) =>
            {
                await LoadComputersAsync();
                TableParam.Text = "";
            };
            TableParam.SelectionChanged += async (s, e) => await
LoadComputersAsync();
            inputuser.TextChanged += async (s, e) => await
LoadComputersAsync();
            inputcomputer.TextChanged += async (s, e) => await
LoadComputersAsync();
        }
        private void AddNewRow_Click(object sender, RoutedEventArgs e)
        {
            DataView view = (DataView)dataGrid.ItemsSource;
            DataRow newRow = view.Table.NewRow();
            view.Table.Rows.Add(newRow);
            dataGrid.ScrollIntoView(newRow);
        }
        private void DeleteRowButton_Click(object sender,
RoutedEventArgs e)
        {
            if (dataGrid.SelectedItem != null)
            {
                var rowView = (DataRowView)dataGrid.SelectedItem;
                rowView.Row.Delete();
            }
        }
        private static readonly Regex TimeRegex = new Regex(@"^([01][0-9]|2[0-3]):[0-5][0-9]$");

        public static bool IsValidTimeFormat(string time)
        {

```

```

        return !string.IsNullOrEmpty(time) &&
TimeRegex.IsMatch(time);
    }

    // thin wrapper – event handler остаётся async void, логика в
    async Task для тестируемости
    private async void GetUpdateRows(object sender, EventArgs e)
    {
        try
        {
            await HandleUpdateRowsAsync();
        }
        catch (Exception ex)
        {
            Debug.WriteLine("GetUpdateRows wrapper error: " +
ex.Message);
        }
    }

    private async Task HandleUpdateRowsAsync()
    {
        dataGrid.CommitEdit();

        DataView dataView = (DataView)dataGrid.ItemsSource;
        if (dataView == null) return;

        DataTable dataTable = dataView.Table;

        var modifiedRows = dataTable.Rows
            .Cast<DataRow>()
            .Where(row => row.RowState == DataRowState.Modified)
            .ToList();

        var deletedRows =
dataTable.GetChanges(DataRowState.Deleted)?.Rows.Cast<DataRow>() ??
Enumerable.Empty<DataRow>();
        var addedRows = dataTable.Rows
            .Cast<DataRow>()
            .Where(row => row.RowState == DataRowState.Added)
            .ToList();

        if (modifiedRows.Count > 0 || deletedRows.Any() ||
addedRows.Count > 0)
        {
            foreach (DataRow row in addedRows)
            {
                try
                {
                    if (TableInWin == "Users")
                    {
                        string name = row["name"].ToString();
                        if (name.Length == 0)
                        {

```

```

        MessageBox.Show(Lang.lang == "en" ?
"Login cannot be empty" : "Логин не может быть пустым");
        return;
    }
    if (name.Length > 99 || name.Length < 5)
    {
        MessageBox.Show(Lang.lang == "en"
? "Login must be between 5 and 99
characters"
: "Логин должен быть от 5 до 99
СИМВОЛОВ");
        return;
    }
    string password =
row["password"].ToString();
    if (password.Length == 0)
    {
        MessageBox.Show(Lang.lang == "en" ?
"Password cannot be empty" : "Пароль не может быть пустым");
        return;
    }
    if (password.Length > 99 ||
password.Length < 8)
    {
        MessageBox.Show(Lang.lang == "en"
? "Password must be between 8 and
99 characters"
: "Пароль должен быть от 8 до 99
СИМВОЛОВ");
        return;
    }

    await
database.AddElementInTableUsers(name, password);
    MessageBox.Show(Lang.lang == "en" ? "New
user added" : "Добавлен новый пользователь");
}
else if (TableInWin == "Computers")
{
    var comps = await
database.ReadElementsInTableComputersAsync();
    string name = row["name"].ToString();
    if (name.Length == 0)
    {
        MessageBox.Show(Lang.lang == "en" ?
"Name cannot be empty" : "Название не может быть пустым");
        return;
    }

    string description =
row["description"].ToString();
    if (description.Length == 0)
    {

```

```

        MessageBox.Show(Lang.lang == "en" ?
"Description cannot be empty" : "Описание не может быть пустым");
        return;
    }

    string cpu = row["cpu"].ToString();
    if (cpu.Length == 0)
    {
        MessageBox.Show(Lang.lang == "en" ?
"CPU cannot be empty" : "Процессор не может быть пустым");
        return;
    }

    string gpu = row["gpu"].ToString();
    if (gpu.Length == 0)
    {
        MessageBox.Show(Lang.lang == "en" ?
"GPU cannot be empty" : "Видеокарта не может быть пустым");
        return;
    }

    string ram = row["ram"].ToString();
    if (ram.Length == 0)
    {
        MessageBox.Show(Lang.lang == "en" ?
"RAM cannot be empty" : "Оперативная память не может быть пустым");
        return;
    }
    if (!int.TryParse(ram, out int x))
    {
        MessageBox.Show(Lang.lang == "en" ?
"RAM must be an integer number" : "Оперативная память должна быть целым
числом");
        return;
    }

    string storage =
row["storage"].ToString();
    if (storage.Length == 0)
    {
        MessageBox.Show(Lang.lang == "en" ?
"Storage cannot be empty" : "Накопитель не может быть пустым");
        return;
    }
    string monitor =
row["monitor"].ToString();
    if (monitor.Length == 0)
    {
        MessageBox.Show(Lang.lang == "en" ?
"Monitor cannot be empty" : "Монитор не может быть пустым");
        return;
    }
    if (!Regex.IsMatch(monitor, @"^[0-
9\s\\".]*$"))

```

```

        {
            MessageBox.Show(Lang.lang == "en"
                ? "Monitor value can only contain
numbers, spaces, double quotes and dots."
                : "Значение монитора может
содержать только цифры, пробелы, кавычки и точки.");
            return;
        }
        if
(!float.TryParse(row["rating"].ToString(), out float rating))
        {
            MessageBox.Show(Lang.lang == "en" ?
"Error: Invalid rating value" : "Ошибка: Некорректное значение
рейтинга");
            return;
        }
        if (rating < 0 || rating > 5)
        {
            MessageBox.Show(Lang.lang == "en" ?
"Rating must be between 0 and 5" : "Рейтинг должен быть от 0 до 5");
            return;
        }

        if
(!float.TryParse(row["PricePerHour"].ToString(), out float
pricePerHour))
        {
            MessageBox.Show(Lang.lang == "en" ?
"Error: Invalid price per hour" : "Ошибка: Некорректное значение цены
за час");
            return;
        }
        if (pricePerHour < 0)
        {
            MessageBox.Show(Lang.lang == "en" ?
"Error: Price cannot be negative" : "Ошибка: Цена не может быть
отрицательной");
            return;
        }
        byte[] graphicData = null;
        object graphicDataObj =
row["GraphicData"];

        if (graphicDataObj != DBNull.Value)
        {
            if (graphicDataObj is byte[])
            {
                graphicData =
(byte[])graphicDataObj;
            }
            else
            {
                try
                {

```

```

                                graphicData
Convert.FromBase64String(graphicDataObj.ToString());
                                }
                                catch
                                {
                                    MessageBox.Show(Lang.lang ==
"en" ? "Graphic data is corrupted or in wrong format" : "Графические
данные повреждены или в неверном формате");
                                    return;
                                }
                            }
                        }
                    if (comps.Count > 17)
                    {
                        MessageBox.Show(Lang.lang == "en" ?
"Maximum number of computers reached" : "Достигнуто максимальное
количество компьютеров");
                    }
                    else
                    {
                        await
database.AddElementInTableComputers(name, description, cpu, gpu, ram,
storage, monitor, rating, pricePerHour, graphicData);
                        MessageBox.Show(Lang.lang == "en" ?
"New computer added" : "Добавлен новый компьютер");
                    }
                }
                else if (TableInWin == "Orders")
                {
                    try
                    {
                        int                userId                =
Convert.ToInt32(row["UserId"]);
                        var                users                =
                        await
database.ReadElementsInTableUsersAsync();
                        var userExists = users.Any(x => x.Id
== userId);
                        if (!userExists)
                        {
                            MessageBox.Show(Lang.lang == "en"
? "Error: User with specified ID not found" : "Ошибка: Пользователь с
указанным ID не найден");
                            return;
                        }
                    }
                }
                int                computerId                =
Convert.ToInt32(row["ComputerId"]);
                var                computers                =
                await
database.ReadElementsInTableComputersAsync();
                var computerExists = computers.Any(x
=> x.Id == computerId);
                if (!computerExists)
                {

```

```

        MessageBox.Show(Lang.lang == "en"
? "Error: Computer with specified ID not found" : "Ошибка: Компьютер с
указанным ID не найден");

        return;
    }
    string status =
row["status"].ToString().ToLower();
    string dateOrder =
row["DateOrder"].ToString();
    string startTimeStr =
row["StartTime"].ToString().Trim();
    string endTimeStr =
row["EndTime"].ToString().Trim();

    if (status != "completed" && status !=
"active" && status != "cancelled")
    {
        MessageBox.Show(Lang.lang == "en"
? "Invalid status value. Allowed values: completed, active, cancelled"
: "Недопустимое значение статуса. Допустимые значения: completed,
active, cancelled");

        return;
    }

    if
(!DateTime.TryParseExact(dateOrder, "dd.MM.yyyy",
CultureInfo.InvariantCulture, DateTimeStyles.None, out DateTime
orderDate))
    {
        MessageBox.Show(Lang.lang == "en"
? "Invalid date format. Use DD.MM.YYYY" : "Неверный формат даты.
Используйте ДД.ММ.ГГГГ");

        return;
    }

    DateTime today = DateTime.Today;
    DateTime maxDate = today.AddDays(14);

    if (orderDate < today)
    {
        MessageBox.Show(Lang.lang == "en"
? "Order date cannot be earlier than today" : "Дата заказа не может
быть раньше сегодняшнего дня");

        return;
    }

    if (orderDate > maxDate)
    {
        MessageBox.Show(Lang.lang == "en"
? "Order date cannot be more than 2 weeks ahead" : "Дата заказа не
может быть более чем на 2 недели вперед");

        return;
    }

```

```

        if (!IsValidTimeFormat(startTimeStr))
        {
            MessageBox.Show(Lang.lang == "en"
? "Invalid start time format. Use HH:MM (00:00 - 23:59)" : "Неверный
формат времени начала. Используйте ЧЧ:ММ (00:00 - 23:59)");
            return;
        }

        if (!IsValidTimeFormat(endTimeStr))
        {
            MessageBox.Show(Lang.lang == "en"
? "Invalid end time format. Use HH:MM (00:00 - 23:59)" : "Неверный
формат времени окончания. Используйте ЧЧ:ММ (00:00 - 23:59)");
            return;
        }

        TimeSpan                startTime                =
TimeSpan.Parse(startTimeStr);
        TimeSpan                endTime                =
TimeSpan.Parse(endTimeStr);

        if (endTime <= startTime)
        {
            MessageBox.Show(Lang.lang == "en"
? "End time must be later than start time" : "Время окончания должно
быть позже времени начала");
            return;
        }

        DateTime now = DateTime.Now;
        DateTime        selectedStartDateTime        =
orderDate.Date + startTime;

        if (selectedStartDateTime < now)
        {
            MessageBox.Show(Lang.lang == "en"
? "Start time cannot be in the past" : "Время начала не может быть в
прошлом");
            return;
        }

        await
database.AddElementsInTableOrders(userId,        computerId,        status,
dateOrder, startTimeStr, endTimeStr);
        MessageBox.Show(Lang.lang == "en" ?
"New order added" : "Добавлен новый заказ");
    }
    catch (FormatException ex)
    {
        MessageBox.Show(Lang.lang == "en" ?
$"Data format error: {ex.Message}" : $"Ошибка формата данных:
{ex.Message}");
    }
    catch (Exception ex)

```

```

        {
            MessageBox.Show(Lang.lang == "en" ?
$"Unexpected error while adding order: {ex.Message}" : $"Неожиданная
ошибка при добавлении заказа: {ex.Message}");
        }
    }
    else if (TableInWin == "Reviews")
    {
        int                userId                =
Convert.ToInt32(row["UserId"]);
        var                users                =
await
database.ReadElementsInTableUsersAsync();
        var userExists = users.Any(x => x.Id ==
userId);

        if (!userExists)
        {
            MessageBox.Show(Lang.lang == "en" ?
"Error: User with specified ID not found" : "Ошибка: Пользователь с
указанным ID не найден");

            return;
        }

        int                computerId                =
Convert.ToInt32(row["ComputerId"]);
        var                computers                =
await
database.ReadElementsInTableComputersAsync();
        var computerExists = computers.Any(x =>
x.Id == computerId);

        if (!computerExists)
        {
            MessageBox.Show(Lang.lang == "en" ?
"Error: Computer with specified ID not found" : "Ошибка: Компьютер с
указанным ID не найден");

            return;
        }
    }
    if
(!float.TryParse(row["rating"].ToString(), out float rating))
    {
        MessageBox.Show(Lang.lang == "en" ?
"Error: Invalid rating value" : "Ошибка: Некорректное значение
рейтинга");

        return;
    }
    if (rating < 0 || rating > 5)
    {
        MessageBox.Show(Lang.lang == "en" ?
"Rating must be between 0 and 5" : "Рейтинг должен быть от 0 до 5");
        return;
    }
    string                comment                =
row["Comment"].ToString();
    if (comment.Length > 500)
    {

```

```

        MessageBox.Show(Lang.lang == "en" ?
"Comment should not exceed 500 characters" : "Комментарий не должен
превышать 500 символов");
        return;
    }
    if (comment.Length == 0)
    {
        MessageBox.Show(Lang.lang == "en" ?
"Comment cannot be empty" : "Комментарий не должен быть пустым");
        return;
    }
    string createAt =
row["CreateAt"].ToString();
    if (!DateTime.TryParseExact(createAt,
"dd.MM.yyyy", CultureInfo.InvariantCulture, DateTimeStyles.None, out
DateTime orderDate))
    {
        MessageBox.Show(Lang.lang == "en" ?
"Invalid date format. Use DD.MM.YYYY" : "Неверный формат даты.
Используйте ДД.ММ.ГГГГ");
        return;
    }
    if (orderDate.Date != DateTime.Today)
    {
        MessageBox.Show(Lang.lang == "en" ?
"Creation date must be today" : "Дата создания должна быть
сегодняшней");
        return;
    }
    await
database.AddElementsInTableReviews(userId, computerId, rating,
comment, createAt);
    MessageBox.Show(Lang.lang == "en" ? "New
review added" : "Добавлен новый отзыв");
}
else if (TableInWin == "ApplicationGame")
{
    try
    {
        string name =
row["Name"].ToString().Trim();
        if (string.IsNullOrEmpty(name))
        {
            MessageBox.Show(Lang.lang == "en"
? "Name cannot be empty" : "Название не может быть пустым");
            return;
        }
        string img =
row["IMG"].ToString().Trim();
        if (string.IsNullOrEmpty(img))
        {

```

```

        MessageBox.Show(Lang.lang == "en"
? "Image path cannot be empty" : "Путь к изображению не может быть
пустым");

        return;
    }

    string description =
row["Description"].ToString().Trim();
    if
(string.IsNullOrEmpty(description))
    {
        MessageBox.Show(Lang.lang == "en"
? "Description cannot be empty" : "Описание не может быть пустым");
        return;
    }

    string url =
row["URL"].ToString().Trim();
    if (string.IsNullOrEmpty(url))
    {
        MessageBox.Show(Lang.lang == "en"
? "URL cannot be empty" : "URL не может быть пустым");
        return;
    }

    if (!img.EndsWith(".png",
StringComparison.OrdinalIgnoreCase) &&
!img.EndsWith(".jpg",
StringComparison.OrdinalIgnoreCase) &&
!img.EndsWith(".jpeg",
StringComparison.OrdinalIgnoreCase))
    {
        MessageBox.Show(Lang.lang == "en"
? "Image must be in .png or .jpg format"
:
"Изображение должно быть в формате .png или .jpg");
        return;
    }

    if (!url.EndsWith(".exe",
StringComparison.OrdinalIgnoreCase))
    {
        MessageBox.Show(Lang.lang == "en"
? "URL must point to .exe file" : "URL должен указывать на .exe файл");
        return;
    }

    await
database.AddElementsInTableApplicationGame(name, img, description,
url);

    MessageBox.Show(Lang.lang == "en" ?
"Application successfully added" : "Приложение успешно добавлено");
}

```

```

        catch (Exception ex)
        {
            MessageBox.Show(Lang.lang == "en" ?
$"Error: {ex.Message}" : $"Ошибка: {ex.Message}");
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show(Lang.lang == "en" ? $"Error
while adding: {ex.Message}" : $"Ошибка при добавлении: {ex.Message}");
    }
    foreach (DataRow row in modifiedRows)
    {
        try
        {
            if (TableInWin == "Users")
            {
                int id = Convert.ToInt32(row["id"]);
                string name = row["name"].ToString();
                if (string.IsNullOrEmpty(name))
                {
                    MessageBox.Show(Lang.lang == "en" ?
"Login cannot be empty" : "Логин не может быть пустым");
                    return;
                }
                if (name.Length > 99 || name.Length < 5)
                {
                    MessageBox.Show(Lang.lang == "en"
? "Login must be between 5 and 99
characters"
: "Логин должен быть от 5 до 99
СИМВОЛОВ");
                    return;
                }
                string password =
row["password"].ToString();
                if (string.IsNullOrEmpty(password))
                {
                    MessageBox.Show(Lang.lang == "en" ?
"Password cannot be empty" : "Пароль не может быть пустым");
                    return;
                }
                if (password.Length > 99 ||
password.Length < 8)
                {
                    MessageBox.Show(Lang.lang == "en"
? "Password must be between 8 and
99 characters"
: "Пароль должен быть от 8 до 99
СИМВОЛОВ");
                    return;
                }
            }
        }
    }
}

```

```

    }
    if (HashCode.IsHash(password))
    {
        MessageBox.Show(
            Lang.lang == "en"
            ? "The password field contains a hash. Please
enter a new password, not an existing hash."
            : "В поле пароля обнаружен хэш. Введите новый
пароль, а не существующий хэш."
        );
        return;
    }

    await
database.UpdateElementInTableUsers(id, name, password);
        MessageBox.Show(Lang.lang == "en" ? $"User
data (ID: {id}) updated successfully!" : $"Данные пользователя (ID:
{id}) успешно обновлены!");
    }
    else if (TableInWin == "Computers")
    {
        int id = Convert.ToInt32(row["id"]);
        string name = row["name"].ToString();
        if (string.IsNullOrEmpty(name))
        {
            MessageBox.Show(Lang.lang == "en" ?
"Name cannot be empty" : "Название не может быть пустым");
            return;
        }

        string description =
row["description"].ToString();
        if (string.IsNullOrEmpty(description))
        {
            MessageBox.Show(Lang.lang == "en" ?
"Description cannot be empty" : "Описание не может быть пустым");
            return;
        }

        string cpu = row["cpu"].ToString();
        if (string.IsNullOrEmpty(cpu))
        {
            MessageBox.Show(Lang.lang == "en" ?
"CPU cannot be empty" : "Процессор не может быть пустым");
            return;
        }

        string gpu = row["gpu"].ToString();
        if (string.IsNullOrEmpty(gpu))
        {
            MessageBox.Show(Lang.lang == "en" ?
"GPU cannot be empty" : "Видеокарта не может быть пустым");
            return;
        }
    }
}

```

```

        string ram = row["ram"].ToString();
        if (string.IsNullOrEmpty(ram))
        {
            MessageBox.Show(Lang.lang == "en" ?
"RAM cannot be empty" : "Оперативная память не может быть пустым");
            return;
        }
        if (!int.TryParse(ram, out int x))
        {
            MessageBox.Show(Lang.lang == "en" ?
"RAM must be an integer number" : "Оперативная память должна быть целым
числом");
            return;
        }

        string storage =
row["storage"].ToString();
        if (string.IsNullOrEmpty(storage))
        {
            MessageBox.Show(Lang.lang == "en" ?
"Storage cannot be empty" : "Накопитель не может быть пустым");
            return;
        }

        string monitor =
row["monitor"].ToString();
        if (string.IsNullOrEmpty(monitor))
        {
            MessageBox.Show(Lang.lang == "en" ?
"Monitor cannot be empty" : "Монитор не может быть пустым");
            return;
        }
        if (!Regex.IsMatch(monitor, @"^[0-
9\s\\"\\".]*$"))
        {
            MessageBox.Show(Lang.lang == "en"
? "Monitor value can only contain
numbers, spaces, double quotes and dots."
: "Значение монитора может
содержать только цифры, пробелы, кавычки и точки.");
            return;
        }

        if
(!float.TryParse(row["rating"].ToString(), out float rating))
        {
            MessageBox.Show(Lang.lang == "en" ?
"Error: Invalid rating value" : "Ошибка: Некорректное значение
рейтинга");
            return;
        }
        if (rating < 0 || rating > 5)
        {

```

```

        MessageBox.Show(Lang.lang == "en" ?
"Rating must be between 0 and 5" : "Рейтинг должен быть от 0 до 5");
        return;
    }

    if
(!float.TryParse(row["PricePerHour"].ToString(), out float
pricePerHour))
    {
        MessageBox.Show(Lang.lang == "en" ?
"Error: Invalid price per hour" : "Ошибка: Некорректное значение цены
за час");
        return;
    }
    if (pricePerHour < 0)
    {
        MessageBox.Show(Lang.lang == "en" ?
"Error: Price cannot be negative" : "Ошибка: Цена не может быть
отрицательной");
        return;
    }

    byte[] graphicData = null;
    object graphicDataObj =
row["GraphicData"];
    if (graphicDataObj != DBNull.Value)
    {
        if (graphicDataObj is byte[])
        {
            graphicData =
(byte[])graphicDataObj;
        }
        else
        {
            try
            {
                graphicData =
Convert.FromBase64String(graphicDataObj.ToString());
            }
            catch
            {
                MessageBox.Show(Lang.lang ==
"en" ? "Graphic data is corrupted or in wrong format" : "Графические
данные повреждены или в неверном формате");
                return;
            }
        }
    }

    await
database.UpdateElementInTableComputers(id, name, description, cpu,
gpu, ram, storage, monitor, rating, pricePerHour, graphicData);

```

```

        MessageBox.Show(Lang.lang == "en" ?
$"Computer data (ID: {id}) updated successfully!" : $"Данные компьютера
(ID: {id}) успешно обновлены!");
    }
    else if (TableInWin == "Orders")
    {
        try
        {
            int id = Convert.ToInt32(row["Id"]);
            int      userId      =
Convert.ToInt32(row["UserId"]);
            var      users      =      await
database.ReadElementsInTableUsersAsync();
            var userExists = users.Any(x => x.Id
== userId);
            if (!userExists)
            {
                MessageBox.Show(Lang.lang == "en"
? "Error: User with specified ID not found" : "Ошибка: Пользователь с
указанным ID не найден");
                return;
            }
            int      computerId      =
Convert.ToInt32(row["ComputerId"]);
            var      computers      =      await
database.ReadElementsInTableComputersAsync();
            var computerExists = computers.Any(x
=> x.Id == computerId);
            if (!computerExists)
            {
                MessageBox.Show(Lang.lang == "en"
? "Error: Computer with specified ID not found" : "Ошибка: Компьютер с
указанным ID не найден");
                return;
            }
            string      status      =
row["status"].ToString().ToLower();
            if (status != "completed" && status !=
"active" && status != "cancelled")
            {
                MessageBox.Show(Lang.lang == "en"
? "Invalid status value. Allowed values: completed, active, cancelled"
: "Недопустимое значение статуса. Допустимые значения: completed,
active, cancelled");
                return;
            }
            string      dateOrder      =
row["DateOrder"].ToString();
            if
(!DateTime.TryParseExact(dateOrder,
"dd.MM.yyyy",

```

```

CultureInfo.InvariantCulture, DateTimeStyles.None, out DateTime
orderDate))

{
    MessageBox.Show(Lang.lang == "en"
? "Invalid date format. Use DD.MM.YYYY" : "Неверный формат даты.
Используйте ДД.ММ.ГГГГ");

    return;
}

DateTime today = DateTime.Today;
DateTime firstDayOfCurrentMonth = new
DateTime(today.Year, today.Month, 1);
DateTime maxDate = today.AddDays(14);

if (orderDate <
firstDayOfCurrentMonth)
{
    MessageBox.Show(Lang.lang == "en"
? $"Order date cannot be earlier than the current month
({firstDayOfCurrentMonth:dd.MM.yyyy})"
:
$"Дата заказа не может быть раньше текущего месяца
({firstDayOfCurrentMonth:dd.MM.yyyy})");
    return;
}

if (orderDate > maxDate)
{
    MessageBox.Show(Lang.lang == "en"
? "Order date cannot be more than 2 weeks ahead" : "Дата заказа не
может быть более чем на 2 недели вперед");
    return;
}

string startTimeStr =
row["StartTime"].ToString().Trim();
string endTimeStr =
row["EndTime"].ToString().Trim();

if (!IsValidTimeFormat(startTimeStr))
{
    MessageBox.Show(Lang.lang == "en"
? "Invalid start time format. Use HH:MM (00:00 - 23:59)" : "Неверный
формат времени начала. Используйте ЧЧ:ММ (00:00 - 23:59)");
    return;
}

if (!IsValidTimeFormat(endTimeStr))
{
    MessageBox.Show(Lang.lang == "en"
? "Invalid end time format. Use HH:MM (00:00 - 23:59)" : "Неверный
формат времени окончания. Используйте ЧЧ:ММ (00:00 - 23:59)");
    return;
}
}

```

```

        TimeSpan          startTime          =
TimeSpan.Parse(startTimeStr);
        TimeSpan          endTime           =
TimeSpan.Parse(endTimeStr);
        if (endTime <= startTime)
        {
            MessageBox.Show(Lang.lang == "en"
? "End time must be later than start time" : "Время окончания должно
быть позже времени начала");
            return;
        }

        DateTime now = DateTime.Now;
        DateTime      selectedStartDateTime =
orderDate.Date + startTime;

        await
database.UpdateElementInTableOrders(id,  userId,  computerId,  status,
dateOrder,  startTimeStr,  endTimeStr);
        MessageBox.Show(Lang.lang == "en" ?
$"Order data (ID: {id}) updated successfully!" : $"Данные заказа (ID:
{id}) успешно обновлены!");
    }
    catch (FormatException ex)
    {
        MessageBox.Show(Lang.lang == "en" ?
$"Data format error: {ex.Message}" : $"Ошибка формата данных:
{ex.Message}");
    }
    catch (Exception ex)
    {
        MessageBox.Show(Lang.lang == "en" ?
$"Unexpected error while updating order: {ex.Message}" : $"Неожиданная
ошибка при обновлении заказа: {ex.Message}");
    }
}
else if (TableInWin == "Reviews")
{
    int id = Convert.ToInt32(row["Id"]);
    int      userId
Convert.ToInt32(row["UserId"]);
    var      users      =      await
database.ReadElementsInTableUsersAsync();
    var userExists = users.Any(x => x.Id ==
userId);

    if (!userExists)
    {
        MessageBox.Show(Lang.lang == "en" ?
"Error: User with specified ID not found" : "Ошибка: Пользователь с
указанным ID не найден");
        return;
    }
}

```

```

int computerId =
Convert.ToInt32(row["ComputerId"]);
var computers = await
database.ReadElementsInTableComputersAsync();
var computerExists = computers.Any(x =>
x.Id == computerId);
if (!computerExists)
{
    MessageBox.Show(Lang.lang == "en" ?
"Error: Computer with specified ID not found" : "Ошибка: Компьютер с
указанным ID не найден");
    return;
}

if
(!float.TryParse(row["rating"].ToString(), out float rating))
{
    MessageBox.Show(Lang.lang == "en" ?
"Error: Invalid rating value" : "Ошибка: Некорректное значение
рейтинга");
    return;
}
if (rating < 0 || rating > 5)
{
    MessageBox.Show(Lang.lang == "en" ?
"Rating must be between 0 and 5" : "Рейтинг должен быть от 0 до 5");
    return;
}

string comment =
row["Comment"].ToString();
if (string.IsNullOrEmpty(comment))
{
    MessageBox.Show(Lang.lang == "en" ?
"Comment cannot be empty" : "Комментарий не может быть пустым");
    return;
}
if (comment.Length > 500)
{
    MessageBox.Show(Lang.lang == "en" ?
"Comment should not exceed 500 characters" : "Комментарий не должен
превышать 500 символов");
    return;
}

string createAt =
row["CreateAt"].ToString();
if (!DateTime.TryParseExact(createAt,
"dd.MM.yyyy", CultureInfo.InvariantCulture, DateTimeStyles.None, out
DateTime createDate))
{
    MessageBox.Show(Lang.lang == "en" ?
"Invalid date format. Use DD.MM.YYYY" : "Неверный формат даты.
Используйте ДД.ММ.ГГГГ");

```

```

        return;
    }
    DateTime today = DateTime.Today;
    if (createDate > today)
    {
        MessageBox.Show(Lang.lang == "en" ?
"Review date cannot be in the future" : "Дата отзыва не может быть в
будущем");
        return;
    }
    await
database.UpdateElementInTableReviews(id, userId, computerId, rating,
comment, createAt);
        MessageBox.Show(Lang.lang == "en" ?
$"Review data (ID: {id}) updated successfully!" : $"Данные отзыва (ID:
{id}) успешно обновлены!");
    }
    else if (TableInWin == "ApplicationGame")
    {
        try
        {
            int id = Convert.ToInt32(row["Id"]);
            string name =
row["Name"].ToString().Trim();
            if (string.IsNullOrEmpty(name))
            {
                MessageBox.Show(Lang.lang == "en"
? "Name cannot be empty" : "Название не может быть пустым");
                return;
            }

            string img =
row["IMG"].ToString().Trim();
            if (string.IsNullOrEmpty(img))
            {
                MessageBox.Show(Lang.lang == "en"
? "Image path cannot be empty" : "Путь к изображению не может быть
пустым");
                return;
            }

            string description =
row["Description"].ToString().Trim();
            if
(string.IsNullOrEmpty(description))
            {
                MessageBox.Show(Lang.lang == "en"
? "Description cannot be empty" : "Описание не может быть пустым");
                return;
            }

            string url =
row["URL"].ToString().Trim();

```

```

        if (string.IsNullOrEmpty(url))
        {
            MessageBox.Show(Lang.lang == "en"
? "URL cannot be empty" : "URL не может быть пустым");
            return;
        }
        if (!img.EndsWith(".png",
StringComparison.OrdinalIgnoreCase) &&
!img.EndsWith(".jpg",
StringComparison.OrdinalIgnoreCase) &&
!img.EndsWith(".jpeg",
StringComparison.OrdinalIgnoreCase))
        {
            MessageBox.Show(Lang.lang == "en"
? "Image must be in .png or .jpg format"
:
"Изображение должно быть в формате .png или .jpg");
            return;
        }
        if (!url.EndsWith(".exe",
StringComparison.OrdinalIgnoreCase))
        {
            MessageBox.Show(Lang.lang == "en"
? "URL must point to .exe file" : "URL должен указывать на .exe файл");
            return;
        }

        await
database.UpdateElementInTableApplicationGame(id, name, img,
description, url);

        MessageBox.Show(Lang.lang == "en" ?
$"Application data (ID: {id}) updated successfully!" : $"Данные
приложения (ID: {id}) успешно обновлены!");
    }
    catch (Exception ex)
    {
        MessageBox.Show(Lang.lang == "en" ?
$"Error while updating application: {ex.Message}" : $"Ошибка при
обновлении приложения: {ex.Message}");
    }
}
}
catch (Exception ex)
{
    MessageBox.Show(Lang.lang == "en" ? $"Error
while updating: {ex.Message}" : $"Ошибка при обновлении:
{ex.Message}");
}
}
foreach (DataRow row in deletedRows)
{
    try
    {
        if (!row.Table.Columns.Contains("id"))

```

```

        {
            MessageBox.Show(Lang.lang == "en" ?
>Error: 'id' column missing in table" : "Ошибка: отсутствует столбец
'id' в таблице");
            return;
        }

        if (!row.HasVersion(DataRowVersion.Original))
        {
            MessageBox.Show(Lang.lang == "en" ?
>Error: original row version missing" : "Ошибка: отсутствует
оригинальная версия строки");
            return;
        }

        object idValue = row["id",
DataRowVersion.Original];

        if (idValue == null || idValue == DBNull.Value)
        {
            MessageBox.Show(Lang.lang == "en" ?
>Error: ID cannot be NULL" : "Ошибка: ID не может быть NULL");
            return;
        }

        if (!int.TryParse(idValue.ToString(), out int
id) || id <= 0)
        {
            MessageBox.Show(Lang.lang == "en" ?
>Error: Invalid ID. Expected positive integer" : "Ошибка: Некорректный
ID. Ожидается положительное целое число");
            return;
        }

        if (TableInWin == "Users")
        {
            await
database.DeleteElementInTableUsers(id);
            MessageBox.Show(Lang.lang == "en" ? $"User
(ID: {id}) deleted!" : $"Пользователь (ID: {id}) удален!");
        }
        else if (TableInWin == "Computers")
        {
            await
database.DeleteElementInTableComputers(id);
            MessageBox.Show(Lang.lang == "en" ?
$"Computer (ID: {id}) deleted!" : $"Компьютер (ID: {id}) удален!");
        }
        else if (TableInWin == "Orders")
        {
            await
database.DeleteElementInTableOrders(id);
            MessageBox.Show(Lang.lang == "en" ?
$"Order (ID: {id}) deleted!" : $"Заказ (ID: {id}) удален!");
        }
    }
}

```

```

        }
        else if (TableInWin == "Reviews")
        {
            await
database.DeleteElementInTableReviews(id);
            MessageBox.Show(Lang.lang == "en" ?
$"Review (ID: {id}) deleted!" : $"Отзыв (ID: {id}) удален!");
        }
        else if (TableInWin == "ApplicationGame")
        {
            await
database.DeleteElementInTableApplicationGame(id);
            MessageBox.Show(Lang.lang == "en" ?
$"Application (ID: {id}) deleted!" : $"Приложение (ID: {id})
удалено!");
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show(Lang.lang == "en" ? $"Error
while deleting: {ex.Message}" : $"Ошибка при удалении: {ex.Message}");
    }
}

dataTable.AcceptChanges();

// После всех изменений безопасно обновляем таблицу
await LoadComputersAsync();
}
else
{
    MessageBox.Show(Lang.lang == "en" ? "No changes to
save" : "Нет изменений для сохранения");
}

await UpdateRatingAsync();
}

private async Task UpdateRatingAsync()
{
    try
    {
        var allOrders = await
database.ReadElementsInTableOrdersAsync();

        foreach (var _item in allOrders)
        {
            int count = 0;
            float result = 0;

            var allReviews = await
database.ReadElementsInTableReviewsAsync();

            foreach (var rev in allReviews)

```

```

        {
            if (rev.ComputerId == _item.ComputerId)
            {
                count++;
                result += (float)rev.Rating;
            }
        }

        if (count != 0)
        {
            result = (float)Math.Round(result / count, 2);
            var comps = await
database.ReadElementsInTableComputersAsync();
            var comp = comps.FirstOrDefault(x => x.Id ==
_item.ComputerId);
            if (comp != null)
            {
                await
database.UpdateElementInTableComputers(comp.Id, comp.Name,
comp.Description, comp.Cpu
, comp.Gpu, comp.Ram, comp.Storage,
comp.Monitor, result, (float)comp.PricePerHour, comp.GraphicData);
            }
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show(ex.Message);
    }
}

private async Task LoadComputersAsync()
{
    dataGrid.AutoGeneratingColumnn -=
DataGrid_AutoGeneratingColumn;
    dataGrid.AutoGeneratingColumnn +=
DataGrid_AutoGeneratingColumn;

    // безопасно читаем выбор на UI-потоке
    var selectedItem = await Dispatcher.InvokeAsync(() =>
TableBox.SelectedItem);

    try
    {
        if (selectedItem == TableBox.Items[0])
        {
            TableInWin = "Users";
            var dt =
database.GetUsersTableWhere(inputuser.Text);
            dataGrid.ItemsSource = dt.DefaultView;

            ((DataView)dataGrid.ItemsSource).Table.AcceptChanges();
            TableParam.Visibility = Visibility.Collapsed;

```

```

        inputuser.Visibility = Visibility.Visible;
        inputcomputer.Visibility = Visibility.Collapsed;
    }
    else if (selectedItem == TableBox.Items[1])
    {
        inputcomputer.Visibility = Visibility.Visible;
        inputuser.Visibility = Visibility.Collapsed;
        TableInWin = "Computers";
        if (TableParam.SelectedItem ==
TableParam.Items[0])
        {
            dataGrid.ItemsSource =
database.GetComputersTableWhere(inputcomputer.Text, Lang.lang == "en"
? "Powerful processors" : "Мощные процессоры").DefaultView;

            ((DataView)dataGrid.ItemsSource).Table.AcceptChanges();
        }
        else if (TableParam.SelectedItem ==
TableParam.Items[1])
        {
            dataGrid.ItemsSource =
database.GetComputersTableWhere(inputcomputer.Text, Lang.lang == "en"
? "Powerful graphics cards" : "Мощные видеокарты").DefaultView;

            ((DataView)dataGrid.ItemsSource).Table.AcceptChanges();
        }
        else if (TableParam.SelectedItem ==
TableParam.Items[2])
        {
            dataGrid.ItemsSource =
database.GetComputersTableWhere(inputcomputer.Text, Lang.lang == "en"
? "Expensive" : "Дорогие").DefaultView;

            ((DataView)dataGrid.ItemsSource).Table.AcceptChanges();
        }
        else
        {
            dataGrid.ItemsSource =
database.GetComputersTableWhere(inputcomputer.Text).DefaultView;

            ((DataView)dataGrid.ItemsSource).Table.AcceptChanges();
        }
        TableParam.Visibility = Visibility.Visible;
        comp1.Visibility = Visibility.Visible;
        comp2.Visibility = Visibility.Visible;
        comp3.Visibility = Visibility.Visible;
        ord1.Visibility = Visibility.Collapsed;
        ord2.Visibility = Visibility.Collapsed;
        ord3.Visibility = Visibility.Collapsed;
    }
    else if (selectedItem == TableBox.Items[2])
    {
        inputcomputer.Visibility = Visibility.Collapsed;
        inputuser.Visibility = Visibility.Collapsed;
    }

```

```

        TableInWin = "Orders";
        if (TableParam.SelectedItem ==
TableParam.Items[3])
        {
            dataGrid.ItemsSource =
database.GetOrdersTable(Lang.lang == "en" ? "Completed" :
"Завершенные").DefaultView;

((DataView)dataGrid.ItemsSource).Table.AcceptChanges();
        }
        else if (TableParam.SelectedItem ==
TableParam.Items[4])
        {
            dataGrid.ItemsSource =
database.GetOrdersTable(Lang.lang == "en" ? "Active" :
"Активные").DefaultView;

((DataView)dataGrid.ItemsSource).Table.AcceptChanges();
        }
        else if (TableParam.SelectedItem ==
TableParam.Items[5])
        {
            dataGrid.ItemsSource =
database.GetOrdersTable(Lang.lang == "en" ? "Cancelled" :
"Отмененные").DefaultView;

((DataView)dataGrid.ItemsSource).Table.AcceptChanges();
        }
        else
        {
            dataGrid.ItemsSource =
database.GetOrdersTable().DefaultView;

((DataView)dataGrid.ItemsSource).Table.AcceptChanges();
        }
        TableParam.Visibility = Visibility.Visible;
        comp1.Visibility = Visibility.Collapsed;
        comp2.Visibility = Visibility.Collapsed;
        comp3.Visibility = Visibility.Collapsed;
        ord1.Visibility = Visibility.Visible;
        ord2.Visibility = Visibility.Visible;
        ord3.Visibility = Visibility.Visible;
    }
    else if (selectedItem == TableBox.Items[3])
    {
        TableInWin = "Reviews";
        dataGrid.ItemsSource =
database.GetReviewsTable().DefaultView;

((DataView)dataGrid.ItemsSource).Table.AcceptChanges();
        TableParam.Visibility = Visibility.Collapsed;
        inputuser.Visibility = Visibility.Collapsed;
        inputcomputer.Visibility = Visibility.Collapsed;
    }

```

```

        else if (selectedItem == TableBox.Items[4])
        {
            TableInWin = "ApplicationGame";
            dataGrid.ItemsSource =
database.GetApplicationGameTable().DefaultView;

            ((DataView)dataGrid.ItemsSource).Table.AcceptChanges();
            TableParam.Visibility = Visibility.Collapsed;
            inputuser.Visibility = Visibility.Collapsed;
            inputcomputer.Visibility = Visibility.Collapsed;
        }
    }
    catch (Exception ex)
    {
        Debug.WriteLine("LoadComputersAsync error: " +
ex.Message);
    }
}

private void DataGrid_AutoGeneratingColumn(object sender,
DataGridAutoGeneratingColumnEventArgs e)
{
    if (e.PropertyName == "Id" || e.PropertyName == "id")
    {
        e.Column.IsReadOnly = true;
        if (e.Column is DataGridTextColumn textColumn)
        {
            textColumn.Foreground = Brushes.Gray;
        }
    }
}
}
}

```

App.xaml

```

<Application x:Class="COURSEPROJECT.App"

xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        ShutdownMode="OnMainWindowClose"
        StartupUri="MainWindow.xaml">
    <Application.Resources>
        <ResourceDictionary>
            <Style TargetType="TextElement">
                <Setter Property="FontFamily" Value="KZ Supercell-
Magic"/>
            </Style>
        </ResourceDictionary>
    </Application.Resources>
</Application>

```

App.xaml.cs

```

using System;
using System.Linq;
using System.Windows;

namespace COURSEPROJECT
{
    public static class Lang
    {
        public static string _lang = "";
        public static event Action LanguageChanged;

        public static string lang
        {
            get => _lang;
            set
            {
                if (_lang != value)
                {
                    _lang = value;
                    LanguageChanged?.Invoke();
                }
            }
        }
    }

    public partial class App : Application
    {
        protected override void OnStartup(StartupEventArgs e)
        {
            base.OnStartup(e);

            var settings = COURSEPROJECT.Properties.Settings.Default;

            string theme = !string.IsNullOrEmpty(settings.Theme) ?
settings.Theme : "StyleTheme1";
            ThemeManager.ApplyTheme(theme, false);
            string culture = !string.IsNullOrEmpty(settings.Strings) ?
settings.Strings : "en-US";
            LocalizationManager.SwitchLanguage(culture, false);
            Lang.lang = culture.Substring(0, 2);
            database.InitDataBase();

        }

        public static void CloseAllWindowsExceptMain()
        {
            var windowsToClose =
Application.Current.Windows.OfType<Window>().
                .Where(w => w !=
Application.Current.MainWindow)
                .ToList();

            if (windowsToClose.Count == 0)
                return;
        }
    }
}

```

```

        foreach (Window window in windowsToClose)
        {
            window.Close();
        }
    }
    public static LaunchingGames LaunchingGamesWindow { get; set; }
}

public static class ThemeManager
{
    public static void ApplyTheme(string themeName, bool
saveSettings = true)
    {
        try
        {
            var app = Application.Current;
            var oldTheme = app.Resources.MergedDictionaries
                .FirstOrDefault(d
d.Source?.ToString().Contains("styles/StyleTheme") == true); =>

            if (oldTheme != null)
            {
                app.Resources.MergedDictionaries.Remove(oldTheme);
            }
            var newTheme = new ResourceDictionary
            {
                Source = new Uri($"styles/{themeName}.xaml",
UriKind.Relative)
            };
            app.Resources.MergedDictionaries.Add(newTheme);
            if (saveSettings)
            {
                Properties.Settings.Default.Theme = themeName;
                Properties.Settings.Default.Save();
            }
        }
        catch (Exception ex)
        {
            MessageBox.Show(Lang.lang == "en" ? $"Error loading
theme: {ex.Message}" : $"Ошибка загрузки темы: {ex.Message}");
        }
    }
}

public static class LocalizationManager
{
    public static void SwitchLanguage(string culture, bool
saveSettings = true)
    {
        try
        {
            var app = Application.Current;

```

```

        var oldLang = app.Resources.MergedDictionaries
            .FirstOrDefault(d
d.Source?.ToString().StartsWith("Strings.") == true); =>

        if (oldLang != null)
        {
            app.Resources.MergedDictionaries.Remove(oldLang);
        }

        var newLang = new ResourceDictionary
        {
            Source = new Uri($"Strings.{culture}.xaml",
UriKind.Relative)
        };
        app.Resources.MergedDictionaries.Add(newLang);

        if (saveSettings)
        {
            Properties.Settings.Default.Strings = culture;
            Properties.Settings.Default.Save();
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show(Lang.lang == "en" ? $"Error loading
language: {ex.Message}" : $"Ошибка загрузки языка: {ex.Message}");
    }
}
}

```

authorization.xaml

```

<Window x:Class="COURSEPROJECT.authorization"

xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006"
xmlns:local="clr-namespace:COURSEPROJECT"
mc:Ignorable="d"
Title="{DynamicResource Auth}" Height="300" Width="400"
WindowStartupLocation="CenterScreen">

    <Grid Style="{DynamicResource GridAuthorization}">
        <Border Style="{DynamicResource AuthorizationBorderStyle}">
            <StackPanel
AuthorizationStackPanelStyle}">
                <TextBlock
AuthorizationTextBlockStyle}" Text="{DynamicResource Auth}"/>

```

```

                <TextBlock                                Style="{DynamicResource
AuthorizationLabelTextBlockStyle}" Text="{DynamicResource Login}"/>
                <Border                                    Style="{DynamicResource
AuthorizationInputBorderBorderStyle}">
                    <TextBox                                Style="{DynamicResource
AuthorizationTextBoxStyle}" x:Name="Login"/>
                </Border>
                <TextBlock                                Style="{DynamicResource
AuthorizationLabelTextBlockStyle}" Text="{DynamicResource Password}"/>
                <Border                                    Style="{DynamicResource
AuthorizationInputBorderBorderStyle}">
                    <PasswordBox                            Style="{DynamicResource
AuthorizationPasswordBoxStyle}" x:Name="Password"/>
                </Border>
                <Border>
                    <Button                                Style="{DynamicResource
AuthorizationButtonStyleTrigger}" Content="{DynamicResource
Registration}" Command="{Binding RegistrationCommand}"/>
                </Border>
                <Border>
                    <Button                                Style="{DynamicResource
AuthorizationButtonStyleTrigger}" Content="{DynamicResource Entrance}"
Command="{Binding AuthorizationClickCommand}"/>
                </Border>
            </StackPanel>
        </Border>
    </Grid>
</Window>

```

authorization.xaml.cs

```

using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Shapes;

namespace COURSEPROJECT
{
    /// <summary>
    /// Логика взаимодействия для authorization.xaml
    /// </summary>
    public partial class authorization : Window
    {
        public ICommand AuthorizationClickCommand { get; }
    }
}

```

```

public ICommand RegistrationCommand { get; }

public authorization()
{
    InitializeComponent();
    DataContext = this;

    AuthorizationClickCommand = new
RelayCommand(AuthorizationClick);
    RegistrationCommand = new RelayCommand(Registration);
    try
    {
        this.Cursor = new
Cursor(Application.GetResourceStream(new Uri("/images/cursor.cur",
UriKind.Relative)).Stream);
    }
    catch (Exception ex)
    {
        MessageBox.Show(Lang.lang == "en"
            ? $"Cursor loading error: {ex.Message}"
            : $"Ошибка загрузки курсора: {ex.Message}");
    }
}

private void CloseAllGames()
{
    if (!string.IsNullOrEmpty(LaunchedGames.Id))
    {
        int launchedGameId;
        if (int.TryParse(LaunchedGames.Id, out
launchedGameId))
        {
            foreach (string expectedPath in
LaunchedGames.PathsGames)
            {
                foreach (Process proc in
Process.GetProcessesByName(System.IO.Path.GetFileNameWithoutExtension
(expectedPath)))
                {
                    try
                    {
                        proc.Kill();
                        proc.WaitForExit();
                    }
                    catch (Exception ex)
                    {
                        Console.WriteLine(Lang.lang == "en"
                            ? $"Error terminating process:
{ex.Message}"
                            : $"Ошибка при завершении
процесса: {ex.Message}");
                    }
                }
            }
        }
    }
}

```

```

    }
}

private void AuthorizationClick(object parametr)
{
    if (Login.Text == "admin" && Password.Password == "admin")
    {
        Global.CurrentUser = Login.Text;
        this.Close();
        return;
    }

    if (string.IsNullOrEmpty(Login.Text) ||
string.IsNullOrEmpty(Password.Password))
    {
        MessageBox.Show(Lang.lang == "en"
            ? "Login and password cannot be empty"
            : "Логин и пароль не могут быть пустыми");
        return;
    }

    if (Login.Text.Length < 5 || Login.Text.Length > 100)
    {
        MessageBox.Show(Lang.lang == "en"
            ? "Login must be between 5 and 99 characters"
            : "Логин должен содержать от 5 до 99 символов");
        return;
    }

    if (Password.Password.Length < 8 ||
Password.Password.Length > 100)
    {
        MessageBox.Show(Lang.lang == "en"
            ? "Password must be between 8 and 99 characters"
            : "Пароль должен содержать от 8 до 99 символов");
        return;
    }

    if (database.CorrectEntrance(Login.Text,
Password.Password))
    {
        Global.CurrentUser = Login.Text;
        App.CloseAllWindowsExceptMain();
        Application.Current.MainWindow?.Show();
        CloseAllGames();
        this.Close();
    }
    else
    {
        string userid = database.GetUserIdByName(Login.Text);
        if (userid == "0")
        {
            MessageBox.Show(Lang.lang == "en"

```

```

        ? "User not found"
        : "Такого пользователя не существует");
    }
    else
    {
        MessageBox.Show(Lang.lang == "en"
            ? "Wrong password"
            : "Неверный пароль");
    }
}

private void Registration(object parametr)
{
    var window =
Application.Current.Windows.OfType<registration>().FirstOrDefault();

    if (window == null)
    {
        window = new registration();
        window.Show();
    }
    else
    {
        window.Activate();
        if (window.WindowState == WindowState.Minimized)
            window.WindowState = WindowState.Normal;
    }
}
}
}

```

CustomCommand.cs

```

using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Input;

namespace COURSEPROJECT
{
    public static class CustomCommand
    {
        public static readonly RoutedCommand OpenProfileWindow = new
RoutedUICommand(
            "Open Profile Window",
            "OpenProfileWindow",
            typeof(CustomCommand),
            new InputGestureCollection {
                new KeyGesture(Key.P, ModifierKeys.Control)
            }
        );
    }
}

```

```

        );
        public static readonly RoutedCommand SwitchTheme = new
RoutedUICommand(
            "Switch Theme",
            "SwitchTheme",
            typeof(CustomCommand),
            new InputGestureCollection {
                new KeyGesture(Key.T, ModifierKeys.Control)
            }
        );
    }
}

```

CustomPasswordBox.xaml

```

<UserControl x:Class="COURSEPROJECT.CustomPasswordBox"

xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
            xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
            xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006"

xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
            xmlns:local="clr-namespace:COURSEPROJECT"
            mc:Ignorable="d"
            d:DesignHeight="450" d:DesignWidth="800">
    <Grid>
        <Border Background="White"
            BorderBrush="LightGray"
            BorderThickness="2"
            CornerRadius="10"
            Padding="4">
            <PasswordBox Name="PasswordInput"
                Width="200"
                Height="20"
                FontSize="16"
                VerticalContentAlignment="Center"
                Background="Transparent"
                BorderThickness="0"/>
        </Border>
    </Grid>
</UserControl>

```

CustomPasswordBox.xaml.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;

```

```

using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Navigation;
using System.Windows.Shapes;

namespace COURSEPROJECT
{
    public partial class CustomPasswordBox : UserControl
    {
        public static readonly DependencyProperty
        PasswordLengthProperty = DependencyProperty.Register(
            "PasswordLength",
            typeof(int),
            typeof(CustomPasswordBox),
            new FrameworkPropertyMetadata(
                8,
                FrameworkPropertyMetadataOptions.None,
                null,
                CoercePasswordLength
            ),
            ValidatePasswordLength
        );
        public int PasswordLength
        {
            get => (int)GetValue(PasswordLengthProperty);
            set => SetValue(PasswordLengthProperty, value);
        }
        public CustomPasswordBox() {
            InitializeComponent();
        }
        private static bool ValidatePasswordLength(object value)
        {
            int length = (int)value;
            return length >= 8;
        }
        private static object CoercePasswordLength(DependencyObject d,
object value)
        {
            int length = (int)value;
            return length > 30 ? 30 : length;
        }
        public string Password
        {
            get => PasswordInput.Password;
            set => PasswordInput.Password = value;
        }
    }
}

```

database.cs

```

using Microsoft.Data.Sqlite;
using SQLitePCL;
using System;

```

```

using System.Collections.Generic;
using System.ComponentModel.DataAnnotations.Schema;
using System.ComponentModel.DataAnnotations;
using System.Configuration;
using System.Data;
using System.Data.Entity;
using System.IO.Packaging;
using System.Linq;
using System.Security.RightsManagement;
using System.Text;
using System.Threading;
using System.Threading.Tasks;
using System.Web;
using System.Windows;
using System.Windows.Media;
using System.Xml.Linq;
using static COURSEPROJECT.AdminTable;
using System.Net;
using System.Reflection.Emit;
using System.Diagnostics;

namespace COURSEPROJECT
{
    public class User
    {
        [Key]
        [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
        [Column("id")]
        public int Id { get; set; }
        [Required]
        [StringLength(100)]
        [Index(IsUnique = true)]
        [Column(TypeName = "nvarchar")]
        public string Name { get; set; }
        [Required]
        [StringLength(100)]
        [Column(TypeName = "nvarchar")]
        public string Password { get; set; }
        public virtual ICollection<Order> Orders { get; set; }
    }
    public class Reviews
    {
        [Key]
        [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
        public int Id { get; set; }

        [Required]
        public int UserId { get; set; }

        [Required]
        public int ComputerId { get; set; }

        [Required]
        [Range(0.0, 5.0)]

```

```

    public double Rating { get; set; }

    [Required]
    [StringLength(500)]
    public string Comment { get; set; }

    public string CreateAt { get; set; }

    [ForeignKey("UserId")]
    public virtual User User { get; set; }

    [ForeignKey("ComputerId")]
    public virtual Computer Computer { get; set; }
}
public class Computer
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
    [Column("id")]
    public int Id { get; set; }

    [Required]
    [StringLength(100)]
    [Column(TypeName = "nvarchar")]
    public string Name { get; set; }

    [Required]
    [Column(TypeName = "nvarchar")]
    [MaxLength(500)]
    public string Description { get; set; }

    [Required]

    [StringLength(50)]
    [Column(TypeName = "nvarchar")]
    public string Cpu { get; set; }

    [Required]
    [Column(TypeName = "nvarchar")]
    [StringLength(50)]
    public string Gpu { get; set; }

    [Required]
    [Column(TypeName = "nvarchar")]
    [StringLength(50)]
    public string Ram { get; set; }

    [Required]
    [Column(TypeName = "nvarchar")]
    [StringLength(50)]
    public string Storage { get; set; }

    [Required]

```

```

    [Column(TypeName = "nvarchar")]
    [StringLength(50)]
    public string Monitor { get; set; }

    public double Rating { get; set; } = 0.0;

    [Required]
    public double PricePerHour { get; set; }

    [MaxLength]
    public byte[] GraphicData { get; set; }

    public virtual ICollection<Order> Orders { get; set; }
}
public class Order
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
    [Column("id")]
    public int Id { get; set; }

    [Required]
    [Column("user_id")]
    public int UserId { get; set; }

    [Required]
    [Column("computer_id")]
    public int ComputerId { get; set; }

    [Required]
    [StringLength(20)]
    [Column(TypeName = "nvarchar")]
    public string Status { get; set; } = "active";

    [Column(TypeName = "nvarchar")]
    public string DateOrder { get; set; }

    [Required]
    [Column(TypeName = "nvarchar")]
    public string StartTime { get; set; }

    [Column(TypeName = "nvarchar")]
    public string EndTime { get; set; }

    [ForeignKey("UserId")]
    public virtual User User { get; set; }

    [ForeignKey("ComputerId")]
    public virtual Computer Computer { get; set; }
}
[Table("ApplicationGame")]
public class ApplicationGame
{
    [Key]

```

```

[DatabaseGenerated(DatabaseGeneratedOption.Identity)]
public int Id { get; set; }

[Required]
public string Name { get; set; } = string.Empty;

public string IMG { get; set; } = string.Empty;

public string Description { get; set; } = string.Empty;

public string URL { get; set; } = string.Empty;
}
public class AppDbContext : DbContext, IDisposable
{
    public AppDbContext() : base("name=DBConnection")
    {
        Database.SetInitializer<AppDbContext>(null);
    }
    public DbSet<Computer> Computers { get; set; }
    public DbSet<Order> Orders { get; set; }
    public DbSet<User> Users { get; set; }
    public DbSet<Reviews> Reviews { get; set; }
    public DbSet<ApplicationGame> ApplicationGame { get; set; }
}
static class database
{
    public static string connectionString =
ConfigurationManager.ConnectionStrings["DBConnection"].ConnectionStri
ng;

    public static void InitDataBase()
    {
        using (var db = new AppDbContext())
        {
            db.Database.CreateIfNotExists();
            var connection = db.Database.Connection;
            MessageBox.Show($"Сервер:
{connection.DataSource}\nБаза данных: {connection.Database}");
        }

        try
        {
            // Выполняем сидинг вне контекста сообщения
            пользователю
            SeedInitialData();
        }
        catch (Exception ex)
        {
            Debug.WriteLine("SeedInitialData error: " +
ex.Message);
        }
    }

    private static void SeedInitialData()
    {

```

```

try
{
    using (var db = new AppDbContext())
    {
        // АДМИН
        if (!db.Users.Any(u => u.Name == "admin"))
        {
            db.Users.Add(new User
            {
                Name = "admin",
                Password = HashCode.HashPassword("admin")
            });
        }

        // ТЕСТОВЫЙ ПОЛЬЗОВАТЕЛЬ
        if (!db.Users.Any(u => u.Name == "testuser"))
        {
            db.Users.Add(new User
            {
                Name = "testuser",
                Password
HashCode.HashPassword("password123")
            });
        }

        // ДЕМО-КОМПЬЮТЕРЫ
        if (!db.Computers.Any())
        {
            db.Computers.Add(new Computer
            {
                Name = "Demo PC 1",
                Description = "Demo gaming PC",
                Cpu = "Intel Core i5",
                Gpu = "GTX 1650",
                Ram = "16",
                Storage = "512GB SSD",
                Monitor = "144",
                Rating = 4.2,
                PricePerHour = 5.0
            });
            db.Computers.Add(new Computer
            {
                Name = "Demo PC 2",
                Description = "Demo workstation",
                Cpu = "Intel Core i7",
                Gpu = "RTX 2060",
                Ram = "32",
                Storage = "1TB SSD",
                Monitor = "60",
                Rating = 4.5,
                PricePerHour = 7.5
            });
        }
    }
}

```

```

// Демо-приложение
if (!db.ApplicationGame.Any())
{
    db.ApplicationGame.Add(new ApplicationGame
    {
        Name = "Demo Game",
        IMG = "images/demo.png",
        Description = "Demo application",
        URL = "" // оставьте пустым или укажите
реальный .exe путь для теста
    });
}

db.SaveChanges();
}
catch (Exception ex)
{
    Debug.WriteLine("SeedInitialData error: " +
ex.Message);
}

// --- async-friendly user creation (existing AddUserInDataBase
left for compatibility) ---
public static async Task AddUserInDataBase(string name, string
password)
{
    using (var db = new AppDbContext())
    {
        using (var tr = db.Database.BeginTransaction())
        {
            try
            {
                if (!db.Users.Any(u => u.Name == name))
                {
                    var newHasgCode =
HashCode.HashPassword(password);
                    var user = new User
                    {
                        Name = name,
                        Password = newHasgCode
                    };

                    db.Users.Add(user);
                    await
db.SaveChangesAsync().ConfigureAwait(false);
                    tr.Commit();
                }
            }
            catch (Exception ex)
            {
                tr.Rollback();
            }
        }
    }
}

```

```

        Debug.WriteLine("AddUserInDataBase error: " +
ex.Message);
        throw;
    }
}

public static bool HaveThisUser(string user)
{
    try
    {
        using (var db = new AppDbContext())
        {
            return db.Users.Any(u => u.Name == user);
        }
    }
    catch (Exception ex)
    {
        Debug.WriteLine("HaveThisUser error: " + ex.Message);
        return false;
    }
}

public static string GetUserIdByName(string name)
{
    if (string.IsNullOrEmpty(name))
        return "0";

    try
    {
        using (var db = new AppDbContext())
        {
            var id = db.Users.Where(u => u.Name ==
name).Select(u => (int?)u.Id).FirstOrDefault();
            return (id ?? 0).ToString();
        }
    }
    catch (Exception ex)
    {
        Debug.WriteLine("GetUserIdByName error: " +
ex.Message);
        return "0";
    }
}

public static string GetUserNameById(int id)
{
    try
    {
        using (var db = new AppDbContext())
        {
            var name = db.Users.Where(x => x.Id == id).Select(x
=> x.Name).FirstOrDefault();

```

```

        return name ?? "None";
    }
}
catch (Exception ex)
{
    Debug.WriteLine("GetUserNameById    error:    "    +
ex.Message);
    return "None";
}
}

public static bool CorrectEntrance(string user, string
userpassword)
{
    try
    {
        using (var db = new AppDbContext())
        {
            var users = db.Users.FirstOrDefault(x => x.Name ==
user);
            if (users != null &&
HashCode.VerifyPassword(userpassword, users.Password))
            {
                return true;
            }
            else
            {
                return false;
            }
        }
    }
    catch (Exception ex)
    {
        Debug.WriteLine("CorrectEntrance    error:    "    +
ex.Message);
        return false;
    }
}

public static List<string> GetInfoAboutUser(string userName)
{
    var userInfo = new List<string>();

    if (string.IsNullOrEmpty(userName))
    {
        return userInfo;
    }

    try
    {
        using (var db = new AppDbContext())
        {
            var user = db.Users

```

```

        .Where(u => u.Name.Equals(userName,
StringComparison.OrdinalIgnoreCase))
        .Select(u => new { u.Id, u.Name, u.Password })
        .FirstOrDefault();

        if (user != null)
        {
            userInfo.Add(user.Id.ToString());
            userInfo.Add(user.Name);
            userInfo.Add(user.Password);
        }
    }
    catch (Exception ex)
    {
        Debug.WriteLine("GetInfoAboutUser error: " +
ex.Message);
    }

    return userInfo;
}

public static void SaveChanges(string name, string password)
{
    using (var db = new AppDbContext())
    {
        using (var tr = db.Database.BeginTransaction())
        {
            try
            {
                var user = db.Users.FirstOrDefault(x => x.Name
== name);

                if (user == null)
                {
                    MessageBox.Show("Пользователь не найден");
                    return;
                }

                var newHashCode =
HashCode.HashPassword(password);
                db.Entry(user).Property(x
=> x.Password).IsModified = true;
                user.Password = newHashCode;

                db.SaveChanges();
                tr.Commit();

                MessageBox.Show("Пароль успешно изменён");
            }
            catch (Exception ex)
            {
                tr.Rollback();
            }
        }
    }
}

```

```

        Debug.WriteLine("SaveChanges error: " +
ex.Message);
        MessageBox.Show($"Ошибка: {ex.Message}");
    }
}

// --- CREATE (existing async methods preserved) ---

public static async Task AddElementInTableUsers(string name,
string password)
{
    using (var db = new AppDbContext())
    {
        using (var tr = db.Database.BeginTransaction())
        {
            try
            {
                if (!db.Users.Any(u => u.Name == name))
                {
                    var newHasgCode =
HashCode.HashPassword(password);
                    var user = new User
                    {
                        Name = name,
                        Password = newHasgCode
                    };

                    db.Users.Add(user);
                    await
db.SaveChangesAsync().ConfigureAwait(false);
                    tr.Commit();
                }
            }
            catch (Exception ex)
            {
                tr.Rollback();
                Debug.WriteLine("AddElementInTableUsers
error: " + ex.Message);
                throw;
            }
        }
    }

    public static async Task AddElementInTableComputers(string
name, string description, string cpu, string gpu, string ram, string
storage, string monitor, float rating, float price, byte[] image)
    {
        using (var db = new AppDbContext())
        {
            using (var tr = db.Database.BeginTransaction())
            {
                try

```

```

        {
            var computer = new Computer
            {
                Name = name,
                Description = description,
                Cpu = cpu,
                Gpu = gpu,
                Ram = ram,
                Storage = storage,
                Monitor = monitor,
                Rating = rating,
                PricePerHour = price,
                GraphicData = image
            };

            db.Computers.Add(computer);
            await
db.SaveChangesAsync().ConfigureAwait(false);
            tr.Commit();
        }
        catch (Exception ex)
        {
            tr.Rollback();
            Debug.WriteLine("AddElementInTableComputers
error: " + ex.Message);
            throw;
        }
    }
}

public static async Task AddElementsInTableOrders(int user_id,
int computer_id, string status, string date_order, string start_time,
string end_time)
{
    using (var db = new AppDbContext())
    {
        using (var tr = db.Database.BeginTransaction())
        {
            try
            {
                var order = new Order
                {
                    UserId = user_id,
                    ComputerId = computer_id,
                    Status = status,
                    DateOrder = date_order,
                    StartTime = start_time,
                    EndTime = end_time
                };

                db.Orders.Add(order);
                await
db.SaveChangesAsync().ConfigureAwait(false);
                tr.Commit();
            }
            catch { }
        }
    }
}

```

```

        }
        catch (Exception ex)
        {
            tr.Rollback();
            Debug.WriteLine("AddElementsInTableOrders
error: " + ex.Message);
            throw;
        }
    }
}

public static async Task AddElementsInTableReviews(int
user_id, int computer_id, float rating, string comment, string
createAt)
{
    using (var db = new AppDbContext())
    {
        try
        {
            var review = new Reviews
            {
                UserId = user_id,
                ComputerId = computer_id,
                Rating = rating,
                Comment = comment,
                CreateAt = createAt
            };

            db.Reviews.Add(review);
            await
db.SaveChangesAsync().ConfigureAwait(false);
        }
        catch (Exception ex)
        {
            Debug.WriteLine("AddElementsInTableReviews error:
" + ex.Message);
            throw;
        }
    }
}

public static async Task
AddElementsInTableApplicationGame(string name, string img, string
description, string url)
{
    using (var db = new AppDbContext())
    {
        using (var tr = db.Database.BeginTransaction())
        {
            try
            {
                var app = new ApplicationGame
                {
                    Name = name,
                    IMG = img,

```

```

        Description = description,
        URL = url
    };

    db.ApplicationGame.Add(app);
    await
db.SaveChangesAsync().ConfigureAwait(false);
    tr.Commit();
}
catch (Exception ex)
{
    tr.Rollback();

Debug.WriteLine("AddElementsInTableApplicationGame    error:    "    +
ex.Message);

    throw;
}
}
}

// --- READ (synchronous wrappers kept for compatibility) ---

public static List<User> ReadElementsInTableUsers()
{
    try
    {
        using (var db = new AppDbContext())
        {
            return db.Users.ToList();
        }
    }
    catch (Exception ex)
    {
        Debug.WriteLine("ReadElementsInTableUsers    error:    "    +
ex.Message);

        return new List<User>();
    }
}

public static List<Computer> ReadElementsInTableComputers()
{
    try
    {
        using (var db = new AppDbContext())
        {
            return db.Computers.ToList();
        }
    }
    catch (Exception ex)
    {
        Debug.WriteLine("ReadElementsInTableComputers error: "
+ ex.Message);

        return new List<Computer>();
    }
}

```

```

    }
}

public static List<Order> ReadElementsInTableOrders()
{
    try
    {
        using (var db = new AppDbContext())
        {
            return db.Orders.ToList();
        }
    }
    catch (Exception ex)
    {
        Debug.WriteLine("ReadElementsInTableOrders error: " +
ex.Message);
        return new List<Order>();
    }
}

public static List<Reviews> ReadElementsInTableReviews()
{
    try
    {
        using (var db = new AppDbContext())
        {
            return db.Reviews.ToList();
        }
    }
    catch (Exception ex)
    {
        Debug.WriteLine("ReadElementsInTableReviews error: " +
ex.Message);
        return new List<Reviews>();
    }
}

public static List<ApplicationGame>
ReadElementsInTableApplicationGame()
{
    try
    {
        using (var db = new AppDbContext())
        {
            return db.ApplicationGame.ToList();
        }
    }
    catch (Exception ex)
    {
        Debug.WriteLine("ReadElementsInTableApplicationGame
error: " + ex.Message);
        return new List<ApplicationGame>();
    }
}

```

```

        // --- ASYNC READ helpers (preferred for UI) ---

        public          static          async          Task<List<User>>
ReadElementsInTableUsersAsync()
        {
            try
            {
                using (var db = new AppDbContext())
                {
                    return                                await
db.Users.ToListAsync().ConfigureAwait(false);
                }
            }
            catch (Exception ex)
            {
                Debug.WriteLine("ReadElementsInTableUsersAsync  error:
" + ex.Message);
                return new List<User>();
            }
        }

        public          static          async          Task<List<Computer>>
ReadElementsInTableComputersAsync()
        {
            try
            {
                using (var db = new AppDbContext())
                {
                    return                                await
db.Computers.ToListAsync().ConfigureAwait(false);
                }
            }
            catch (Exception ex)
            {
                Debug.WriteLine("ReadElementsInTableComputersAsync
error: " + ex.Message);
                return new List<Computer>();
            }
        }

        public          static          async          Task<List<Order>>
ReadElementsInTableOrdersAsync()
        {
            try
            {
                using (var db = new AppDbContext())
                {
                    return                                await
db.Orders.ToListAsync().ConfigureAwait(false);
                }
            }
            catch (Exception ex)
            {

```

```

        Debug.WriteLine("ReadElementsInTableOrdersAsync
error: " + ex.Message);
        return new List<Order>();
    }
}

    public static async Task<List<Reviews>>
ReadElementsInTableReviewsAsync()
    {
        try
        {
            using (var db = new AppDbContext())
            {
                return await
db.Reviews.ToListAsync().ConfigureAwait(false);
            }
        }
        catch (Exception ex)
        {
            Debug.WriteLine("ReadElementsInTableReviewsAsync
error: " + ex.Message);
            return new List<Reviews>();
        }
    }

    public static async Task<List<ApplicationGame>>
ReadElementsInTableApplicationGameAsync()
    {
        try
        {
            using (var db = new AppDbContext())
            {
                return await
db.ApplicationGame.ToListAsync().ConfigureAwait(false);
            }
        }
        catch (Exception ex)
        {
            Debug.WriteLine("ReadElementsInTableApplicationGameAsync error: " +
ex.Message);
            return new List<ApplicationGame>();
        }
    }

    // --- UPDATE (async existing) ---

    public static async Task UpdateElementInTableUsers(int id,
string name, string password)
    {
        if (HashCode.IsHash(password))
        {
            MessageBox.Show(
                Lang.lang == "en"

```

```

        ? "The password field contains a hash. Please enter a new
password, not an existing hash."
        : "В поле пароля обнаружен хэш. Введите новый пароль, а не
существующий хэш."
    );
    return;
}
using (var db = new AppDbContext())
{
    using (var tr = db.Database.BeginTransaction())
    {
        try
        {
            var user = db.Users.Where(x => x.Id ==
id).FirstOrDefault();
            if (user == null) return;
            user.Name = name;
            user.Password =
HashCode.HashPassword(password);
            await
db.SaveChangesAsync().ConfigureAwait(false);
            tr.Commit();
        }
        catch (Exception ex)
        {
            tr.Rollback();
            Debug.WriteLine("UpdateElementInTableUsers
error: " + ex.Message);
            throw;
        }
    }
}

public static async Task UpdateElementInTableComputers(int id,
string name, string description, string cpu, string gpu, string ram,
string storage, string monitor, float rating, float price, byte[]
image)
{
    using (var db = new AppDbContext())
    {
        using (var tr = db.Database.BeginTransaction())
        {
            try
            {
                var computer = db.Computers.Where(x => x.Id ==
id).FirstOrDefault();
                if (computer == null) return;
                computer.Name = name;
                computer.Description = description;
                computer.Cpu = cpu;
                computer.Gpu = gpu;
                computer.Ram = ram;
                computer.Storage = storage;
                computer.Monitor = monitor;
            }
        }
    }
}

```

```

        computer.Rating = rating;
        computer.PricePerHour = price;
        computer.GraphicData = image;
        await
db.SaveChangesAsync().ConfigureAwait(false);
        tr.Commit();
    }
    catch (Exception ex)
    {
        tr.Rollback();

Debug.WriteLine("UpdateElementInTableComputers error: " + ex.Message);
        throw;
    }
}

}

}

    public static async Task UpdateElementInTableOrders(int id, int
user_id, int computer_id, string status, string date_order, string
start_time, string end_time)
    {
        using (var db = new AppDbContext())
        {
            using (var tr = db.Database.BeginTransaction())
            {
                try
                {
                    var order = db.Orders.Where(x => x.Id ==
id).FirstOrDefault();
                    if (order == null) return;
                    order.UserId = user_id;
                    order.ComputerId = computer_id;
                    order.Status = status;
                    order.DateOrder = date_order;
                    order.StartTime = start_time;
                    order.EndTime = end_time;
                    await
db.SaveChangesAsync().ConfigureAwait(false);
                    tr.Commit();
                }
                catch (Exception ex)
                {
                    tr.Rollback();
                    Debug.WriteLine("UpdateElementInTableOrders
error: " + ex.Message);
                    throw;
                }
            }
        }
    }

    public static async Task UpdateElementInTableReviews(int id,
int user_id, int computer_id, float rating, string comment, string
createAt)
    {

```

```

        using (var db = new AppDbContext())
        {
            try
            {
                var review = db.Reviews.Where(x => x.Id ==
id).FirstOrDefault();
                if (review == null) return;
                review.UserId = user_id;
                review.ComputerId = computer_id;
                review.Rating = rating;
                review.Comment = comment;
                review.CreateAt = createAt;
                await
db.SaveChangesAsync().ConfigureAwait(false);
            }
            catch (Exception ex)
            {
                Debug.WriteLine("UpdateElementInTableReviews
error: " + ex.Message);
                throw;
            }
        }
    }
    public static async Task
UpdateElementInTableApplicationGame(int id, string name, string img,
string description, string url)
    {
        using (var db = new AppDbContext())
        {
            try
            {
                var app = db.ApplicationGame.Where(x => x.Id ==
id).FirstOrDefault();
                if (app == null) return;
                app.Name = name;
                app.IMG = img;
                app.Description = description;
                app.URL = url;
                await
db.SaveChangesAsync().ConfigureAwait(false);
            }
            catch (Exception ex)
            {
                Debug.WriteLine("UpdateElementInTableApplicationGame    error:    "    +
ex.Message);
                throw;
            }
        }
    }

    // --- DELETE (async existing) ---

    public static async Task DeleteElementInTableUsers(int id)

```

```

{
    using (var db = new AppDbContext())
    {
        using (var tr = db.Database.BeginTransaction())
        {
            try
            {
                var user = db.Users.FirstOrDefault(x => x.Id
== id);
                if (user != null)
                {
                    var rewToDel = db.Reviews.Where(x
x.UserId == id);
                    if (rewToDel != null)
                    {
                        foreach (var el in rewToDel)
                        {
                            db.Reviews.Remove(el);
                        }
                    }
                    db.Users.Remove(user);
                }
                await
db.SaveChangesAsync().ConfigureAwait(false);
                tr.Commit();
            }
            catch (Exception ex)
            {
                tr.Rollback();
                Debug.WriteLine("DeleteElementInTableUsers
error: " + ex.Message);
                throw;
            }
        }
    }
}

public static async Task DeleteElementInTableComputers(int id)
{
    using (var db = new AppDbContext())
    {
        using (var tr = db.Database.BeginTransaction())
        {
            try
            {
                var computer = db.Computers.FirstOrDefault(x
=> x.Id == id);
                if (computer != null)
                {
                    var reviewsToDelete = db.Reviews.Where(x
=> x.ComputerId == id);
                    if (reviewsToDelete != null)
                    {
                        foreach (var review in
reviewsToDelete)

```

```

        {
            db.Reviews.Remove(review);
        }
    }

    var ordersToDelete = db.Orders.Where(x =>
x.ComputerId == id);
    if (ordersToDelete != null)
    {
        foreach (var order in ordersToDelete)
        {
            db.Orders.Remove(order);
        }

        db.Computers.Remove(computer);
    }
    await
db.SaveChangesAsync().ConfigureAwait(false);
    tr.Commit();
}
catch (Exception ex)
{
    tr.Rollback();

    Debug.WriteLine("DeleteElementInTableComputers error: " + ex.Message);
    throw;
}
}

}

public static async Task DeleteElementInTableOrders(int id)
{
    using (var db = new AppDbContext())
    {
        using (var tr = db.Database.BeginTransaction())
        {
            try
            {
                var order = db.Orders.FirstOrDefault(x => x.Id
== id);

                if (order != null)
                {
                    db.Orders.Remove(order);
                }

                await
db.SaveChangesAsync().ConfigureAwait(false);
                tr.Commit();
            }
            catch (Exception ex)
            {
                tr.Rollback();
                Debug.WriteLine("DeleteElementInTableOrders
error: " + ex.Message);
            }
        }
    }
}

```

```

        throw;
    }
}
}
}
public static async Task DeleteElementInTableReviews(int id)
{
    using (var db = new AppDbContext())
    {
        using (var tr = db.Database.BeginTransaction())
        {
            try
            {
                var review = db.Reviews.FirstOrDefault(x =>
x.Id == id);
                if (review != null)
                {
                    db.Reviews.Remove(review);
                }
                await
db.SaveChangesAsync().ConfigureAwait(false);
                tr.Commit();
            }
            catch (Exception ex)
            {
                tr.Rollback();
                Debug.WriteLine("DeleteElementInTableReviews
error: " + ex.Message);
                throw;
            }
        }
    }
}
}
public static async Task
DeleteElementInTableApplicationGame(int id)
{
    using (var db = new AppDbContext())
    {
        using (var tr = db.Database.BeginTransaction())
        {
            try
            {
                var app = db.ApplicationGame.FirstOrDefault(x
=> x.Id == id);
                if (app != null)
                {
                    db.ApplicationGame.Remove(app);
                }
                await
db.SaveChangesAsync().ConfigureAwait(false);
                tr.Commit();
            }
            catch (Exception ex)
            {

```

```

        tr.Rollback();

Debug.WriteLine("DeleteElementInTableApplicationGame error: " +
ex.Message);

        throw;
    }
}

// --- DataTable helpers (unchanged logic, но упрощены
транзакции) ---

public static DataTable GetComputersTable(string filter =
"default")
{
    try
    {
        using (var db = new AppDbContext())
        {
            IQueryable<Computer> query = db.Computers;

            if (filter == "Мощные процессоры")
            {
                query = query.Where(c => c.Cpu.Contains("Core
i7") ||
                                c.Cpu.Contains("Core
i5") ||
                                c.Cpu.Contains("Core
i9") ||
                                c.Cpu.Contains("Ryzen
5") ||
                                c.Cpu.Contains("Ryzen
7") ||
                                c.Cpu.Contains("Ryzen
9"));
            }
            else if (filter == "Мощные видеокарты")
            {
                query = query.Where(c => c.Gpu.Contains("GTX")
                                ||
                                c.Gpu.Contains("RTX")
                                ||
                                c.Gpu.Contains("RX"));
            }
            else if (filter == "Дорогие")
            {
                query = query.Where(c => c.PricePerHour >=
1000);
            }

            var computers = query.ToList();

            DataTable table = new DataTable("Computers");

```

```

        table.Columns.Add("Id", typeof(int));
        table.Columns.Add("Name", typeof(string));
        table.Columns.Add("Description", typeof(string));
        table.Columns.Add("Cpu", typeof(string));
        table.Columns.Add("Gpu", typeof(string));
        table.Columns.Add("Ram", typeof(string));
        table.Columns.Add("Storage", typeof(string));
        table.Columns.Add("Monitor", typeof(string));
        table.Columns.Add("Rating", typeof(float));
        table.Columns.Add("PricePerHour", typeof(float));
        table.Columns.Add("GraphicData", typeof(byte[]));

        foreach (var computer in computers)
        {
            table.Rows.Add(
                computer.Id,
                computer.Name,
                computer.Description,
                computer.Cpu,
                computer.Gpu,
                computer.Ram,
                computer.Storage,
                computer.Monitor,
                computer.Rating,
                computer.PricePerHour,
                computer.GraphicData
            );
        }

        return table;
    }
    catch (Exception ex)
    {
        Debug.WriteLine("GetComputersTable error: " +
            ex.Message);
        return new DataTable("Computers");
    }
}

public static DataTable GetComputersTableWhere(string text,
string filter = "default")
{
    try
    {
        text = string.Empty; // защита от null
        string[] filters = text.Split(',');
        string cpuFilter = filters.Length > 0 ?
filters[0].Trim() : "";
        string gpuFilter = filters.Length > 1 ?
filters[1].Trim() : "";
    }
    catch { }
}

```

```

using (var db = new AppDbContext())
{
    IQueryable<Computer> query = db.Computers;

    if (!string.IsNullOrEmpty(cpuFilter))
    {
        query = query.Where(c =>
c.Cpu.Contains(cpuFilter));
    }

    if (!string.IsNullOrEmpty(gpuFilter))
    {
        query = query.Where(c =>
c.Gpu.Contains(gpuFilter));
    }
    if (filter == "Мощные процессоры" || filter ==
"Powerful processors")
    {
        query = query.Where(c => c.Cpu.Contains("Core
i7") ||
                                c.Cpu.Contains("Core
i5") ||
                                c.Cpu.Contains("Core
i9") ||
                                c.Cpu.Contains("Ryzen
5") ||
                                c.Cpu.Contains("Ryzen
7") ||
                                c.Cpu.Contains("Ryzen
9"));
    }
    else if (filter == "Мощные видеокарты" || filter
== "Powerful graphics cards")
    {
        query = query.Where(c => c.Gpu.Contains("GTX")
||
                                c.Gpu.Contains("RTX")
||
                                c.Gpu.Contains("RX"));
    }
    else if (filter == "Дорогие" || filter ==
"Expensive")
    {
        // согласованный порог с GetComputersTable
        query = query.Where(c => c.PricePerHour >=
1000);
    }

    var computers = query.ToList();

    DataTable table = new DataTable("Computers");

    table.Columns.Add("Id", typeof(int));
    table.Columns.Add("Name", typeof(string));

```

```

        table.Columns.Add("Description", typeof(string));
        table.Columns.Add("Cpu", typeof(string));
        table.Columns.Add("Gpu", typeof(string));
        table.Columns.Add("Ram", typeof(string));
        table.Columns.Add("Storage", typeof(string));
        table.Columns.Add("Monitor", typeof(string));
        table.Columns.Add("Rating", typeof(float));
        table.Columns.Add("PricePerHour", typeof(float));
        table.Columns.Add("GraphicData", typeof(byte[]));

        foreach (var computer in computers)
        {
            table.Rows.Add(
                computer.Id,
                computer.Name,
                computer.Description,
                computer.Cpu,
                computer.Gpu,
                computer.Ram,
                computer.Storage,
                computer.Monitor,
                computer.Rating,
                computer.PricePerHour,
                computer.GraphicData
            );
        }

        return table;
    }
}
catch (Exception ex)
{
    Debug.WriteLine("GetComputersTableWhere error: " +
ex.Message);
    return new DataTable("Computers");
}

public static DataTable GetUsersTable()
{
    try
    {
        using (var db = new AppDbContext())
        {
            var users = db.Users.ToList();

            DataTable table = new DataTable("Users");

            table.Columns.Add("Id", typeof(int));
            table.Columns.Add("Name", typeof(string));
            table.Columns.Add("Password", typeof(string));

            foreach (var user in users)

```

```

        {
            DataRow row = table.NewRow();
            row["Id"] = user.Id;
            row["Name"] = user.Name;
            row["Password"] = user.Password;
            table.Rows.Add(row);
        }

        return table;
    }
}
catch (Exception ex)
{
    Debug.WriteLine("GetUsersTable error: " + ex.Message);
    return new DataTable("Users");
}
}

public static DataTable GetUsersTableWhere(string text)
{
    try
    {
        using (var db = new AppDbContext())
        {
            var users = db.Users.ToList();

            DataTable table = new DataTable("Users");

            table.Columns.Add("Id", typeof(int));
            table.Columns.Add("Name", typeof(string));
            table.Columns.Add("Password", typeof(string));

            foreach (var user in users)
            {
                if (user.Name.Contains(text) && text.Length >
0)
                {
                    DataRow row = table.NewRow();
                    row["Id"] = user.Id;
                    row["Name"] = user.Name;
                    row["Password"] = user.Password;
                    table.Rows.Add(row);
                }
                else if (text.Length == 0)
                {
                    DataRow row = table.NewRow();
                    row["Id"] = user.Id;
                    row["Name"] = user.Name;
                    row["Password"] = user.Password;
                    table.Rows.Add(row);
                }
            }

            return table;
        }
    }
}

```

```

        }
        catch (Exception ex)
        {
            Debug.WriteLine("GetUsersTableWhere error: " +
ex.Message);
            return new DataTable("Users");
        }
    }

    public static DataTable GetOrdersTable(string filter =
"default")
    {
        try
        {
            using (var db = new AppDbContext())
            {
                var query = db.Orders.AsQueryable();

                if (filter == "Завершенные" || filter ==
"Completed")
                    query = query.Where(o => o.Status ==
"completed");
                else if (filter == "Активные" || filter ==
"Active")
                    query = query.Where(o => o.Status == "active");
                else if (filter == "Отмененные" || filter ==
"Cancelled")
                    query = query.Where(o => o.Status ==
"cancelled");

                var orders = query.ToList();

                DataTable table = new DataTable("Orders");

                table.Columns.Add("Id", typeof(int));
                table.Columns.Add("UserId", typeof(int));
                table.Columns.Add("ComputerId", typeof(int));
                table.Columns.Add("Status", typeof(string));
                table.Columns.Add("DateOrder", typeof(string));
                table.Columns.Add("StartTime", typeof(string));
                table.Columns.Add("EndTime", typeof(string));

                foreach (var order in orders)
                {
                    table.Rows.Add(
                        order.Id,
                        order.UserId,
                        order.ComputerId,
                        order.Status,
                        order.DateOrder,
                        order.StartTime,
                        order.EndTime

```

```

        );
    }

    return table;
}
}
catch (Exception ex)
{
    Debug.WriteLine("GetOrdersTable error: " +
ex.Message);
    return new DataTable("Orders");
}

public static DataTable GetReviewsTable()
{
    try
    {
        using (var db = new AppDbContext())
        {
            var reviews = db.Reviews.ToList();

            DataTable table = new DataTable("Reviews");

            table.Columns.Add("Id", typeof(int));
            table.Columns.Add("UserId", typeof(int));
            table.Columns.Add("ComputerId", typeof(int));
            table.Columns.Add("Rating", typeof(float));
            table.Columns.Add("Comment", typeof(string));
            table.Columns.Add("CreateAt", typeof(string));

            foreach (var review in reviews)
            {
                DataRow row = table.NewRow();
                row["Id"] = review.Id;
                row["UserId"] = review.UserId;
                row["ComputerId"] = review.ComputerId;
                row["Rating"] = review.Rating;
                row["Comment"] = review.Comment;
                row["CreateAt"] = review.CreateAt;
                table.Rows.Add(row);
            }

            return table;
        }
    }
    catch (Exception ex)
    {
        Debug.WriteLine("GetReviewsTable error: " +
ex.Message);
        return new DataTable("Reviews");
    }
}

```

```

public static DataTable GetApplicationGameTable()
{
    try
    {
        using (var db = new AppDbContext())
        {
            var apps = db.ApplicationGame.ToList();

            DataTable table = new
DataTable("ApplicationGame");

            table.Columns.Add("Id", typeof(int));
            table.Columns.Add("Name", typeof(string));
            table.Columns.Add("IMG", typeof(string));
            table.Columns.Add("Description", typeof(string));
            table.Columns.Add("URL", typeof(string));

            foreach (var app in apps)
            {
                DataRow row = table.NewRow();
                row["Id"] = app.Id;
                row["Name"] = app.Name;
                row["IMG"] = app.IMG;
                row["Description"] = app.Description;
                row["URL"] = app.URL;
                table.Rows.Add(row);
            }

            return table;
        }
    }
    catch (Exception ex)
    {
        Debug.WriteLine("GetApplicationGameTable error: " +
ex.Message);
        return new DataTable("ApplicationGame");
    }
}
}

```

hash.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Security.Cryptography;
using System.Text;
using System.Threading.Tasks;

namespace COURSEPROJECT
{
    public static class HashCode

```

```

    {
        public static string HashPassword(string password)
        {
            string salt = "course2025";
            using (var sha256 = SHA256.Create())
            {
                var saltedPassword = password + salt;
                var bytes = sha256.ComputeHash(Encoding.UTF8.GetBytes(saltedPassword));
                return Convert.ToBase64String(bytes);
            }
        }
        public static bool VerifyPassword(string enteredPassword,
string storedHash)
        {
            var hashOfEntered = HashPassword(enteredPassword);
            return hashOfEntered == storedHash;
        }
        public static bool IsHash(string input)
        {
            if (string.IsNullOrEmpty(input))
                return false;

            try
            {
                return input.Length == 44 &&
Convert.FromBase64String(input) != null;
            }
            catch
            {
                return false;
            }
        }
    }
}

```

Item.xaml

```

<Window x:Class="COURSEPROJECT.Item"

xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006"

xmlns:materialDesign="http://materialdesigninxaml.net/winfx/xaml/them
es"

    xmlns:local="clr-namespace:COURSEPROJECT"

    mc:Ignorable="d"
    WindowStartupLocation="CenterScreen"
    Title="{DynamicResource Computerbooking}" Height="500"
Width="800">

```

```

<Grid>
    <Border>
        <Border.Background>
            <ImageBrush ImageSource="/images/pcphoto.png"
Stretch="UniformToFill" Opacity="1"/>
        </Border.Background>
        <StackPanel Background="#7F000000">
            <TextBlock x:Name="computerid" Text="{Binding Id}"
FontSize="24" FontWeight="Bold" Foreground="White"
HorizontalAlignment="Center"/>
            <TextBlock Text="{Binding Name}" FontSize="24"
FontSize="24" FontWeight="Bold" Foreground="White" HorizontalAlignment="Center"/>
            <TextBlock Text="{DynamicResource PC}" FontSize="24"
FontSize="24" FontWeight="Bold" Foreground="White" HorizontalAlignment="Center"/>
            <TextBlock Text="{Binding Cpu}" FontSize="24"
FontSize="24" FontWeight="Bold" Foreground="White" HorizontalAlignment="Center"/>
            <TextBlock Text="{Binding Gpu}" FontSize="24"
FontSize="24" FontWeight="Bold" Foreground="White" HorizontalAlignment="Center"/>
            <TextBlock x:Name="ram" FontSize="24"
FontSize="24" FontWeight="Bold" Foreground="White" HorizontalAlignment="Center"/>
            <TextBlock Text="{Binding Storage}" FontSize="24"
FontSize="24" FontWeight="Bold" Foreground="White" HorizontalAlignment="Center"/>
            <TextBlock x:Name="monitor" FontSize="24"
FontSize="24" FontWeight="Bold" Foreground="White" HorizontalAlignment="Center"/>
            <TextBlock x:Name="rating" Text="" FontSize="24"
FontSize="24" FontWeight="Bold" Foreground="White" HorizontalAlignment="Center"/>
            <TextBlock x:Name="price" Text="" FontSize="24"
FontSize="24" FontWeight="Bold" Foreground="White" HorizontalAlignment="Center"/>
            <DatePicker Margin="0 7 0 0" x:Name="selectedDate"
Width="200"
materialDesign:HintAssist.Hint="{DynamicResource
DatePickerHintText}"
SelectedDate="{Binding SelectedDate}">
                <DatePicker.Resources>
                    <Style TargetType="DatePickerTextBox">
                        <Setter Property="IsReadOnly"
Value="True"/>
                        <Setter Property="Foreground"
Value="Gray"/>
                        <Setter Property="Text"
Value="{DynamicResource DatePickerHintText}"/>
                    </Style>
                </DatePicker.Resources>
            </DatePicker>
            <StackPanel Margin="0 7 0 0" Orientation="Horizontal"
HorizontalAlignment="Center">
                <TextBox Margin="0 0 5 0"
x:Name="starttime"
Width="80"
Text="00:00"
materialDesign:HintAssist.Hint="Начальное
время"
PreviewTextInput="NumericTextBox_PreviewTextInput" />

```

```

        <TextBox Margin="5 0 0 0"
            x:Name="endtime"
            Width="80"
            Text="23:59"
            materialDesign:HintAssist.Hint="Конечное
время"
PreviewTextInput="NumericTextBox_PreviewTextInput" />
    </StackPanel>
    <TextBlock Margin="0 10 0 0" x:Name="costpc" Text=""
Foreground="White" HorizontalAlignment="Center" FontSize="16"
FontWeight="Bold"/>
    <Button Margin="0 7 0 0" Style="{DynamicResource
AuthorizationButtonStyleTrigger}" Click="WriteInfoByItem"
MaxWidth="100" Content="{DynamicResource Toorder}"/>
    </StackPanel>
</Border>
</Grid>
</Window>

```

Item.xaml.cs

```

using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Text;
using System.Text.Json;
using System.Text.Json.Serialization;
using System.Text.RegularExpressions;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Shapes;
using System.Diagnostics;

namespace COURSEPROJECT
{
    public partial class Item : Window
    {
        private readonly Computer _tariff;
        public Item(Computer tariff)
        {
            InitializeComponent();
            _tariff = tariff;
            DataContext = tariff;
            try
            {

```

```

        this.Cursor = new
Cursor(Application.GetResourceStream(new Uri("/images/cursor.cur",
UriKind.Relative)).Stream);
    }
    catch (Exception ex)
    {
        MessageBox.Show(Lang.lang == "en"
            ? $"Cursor loading error: {ex.Message}"
            : $"Ошибка загрузки курсора: {ex.Message}");
    }
    CostChange();
    starttime.SelectionChanged += (s, e) => CostChange();
    endtime.SelectionChanged += (s, e) => CostChange();
    FormattedPrice();
    CorrectRating();
    GenerateRamAndMonitor();
    Lang.LanguageChanged += CostChange;
    this.Closed += (s, e) => Lang.LanguageChanged -=
CostChange;
    Lang.LanguageChanged += FormattedPrice;
    this.Closed += (s, e) => Lang.LanguageChanged -=
FormattedPrice;
    Lang.LanguageChanged += GenerateRamAndMonitor;
    this.Closed += (s, e) => Lang.LanguageChanged -=
GenerateRamAndMonitor;
    }
    bool IsValidTime(string time)
    {
        return Regex.IsMatch(time, @"^(?:[01][0-9]|2[0-3]):[0-
5][0-9]$");
    }

    // Поместил обработчик в async void — безопасно для UI-
    // обработчика события
    public async void WriteInfoByItem(object sender, EventArgs e)
    {
        try
        {
            // читаем заказы асинхронно
            var items = await
database.ReadElementsInTableOrdersAsync();

            // получаем id пользователя (синхронный метод оставлен
            // в базе)
            var userIdStr =
database.GetUserIdByName(Global.CurrentUser);
            if (!int.TryParse(userIdStr, out int currentUserId))
            {
                MessageBox.Show(Lang.lang == "en" ? "User not
found" : "Пользователь не найден");
                return;
            }

            int userActiveBookings = items.Count(item =>

```

```

        item.UserId == currentUserId &&
        item.Status == "active");

        if (!IsValidTime(starttime.Text) ||
!IsValidTime(endtime.Text))
        {
            MessageBox.Show(Lang.lang == "en" ? "Invalid time
format" : "Неверный формат времени");
            return;
        }

        if (userActiveBookings >= 3)
        {
            MessageBox.Show(Lang.lang == "en"
? "You can have no more than 3 active
bookings!"
: "Вы можете иметь не более 3 активных
бронирований!");
            return;
        }

        if (!selectedDate.SelectedDate.HasValue)
        {
            MessageBox.Show(Lang.lang == "en"
? "Please select a date!"
: "Пожалуйста, выберите дату!");
            return;
        }

        DateTime selectedDateValue =
selectedDate.SelectedDate.Value.Date;
        DateTime currentDate = DateTime.Now.Date;
        DateTime maxAllowedDate = currentDate.AddDays(14);

        if (selectedDateValue < currentDate)
        {
            MessageBox.Show(Lang.lang == "en"
? "Date cannot be in the past!"
: "Дата не может быть в прошлом!");
            return;
        }

        if (selectedDateValue > maxAllowedDate)
        {
            MessageBox.Show(Lang.lang == "en"
? "Booking is available only for 2 weeks in
advance!"
: "Бронирование доступно только на 2 недели
вперед!");
            return;
        }

        if (!TimeSpan.TryParse(starttime?.Text, out TimeSpan
start) ||

```

```

end) )
        !TimeSpan.TryParse(endtime?.Text, out TimeSpan
    {
        MessageBox.Show(Lang.lang == "en"
            ? "Invalid time format!"
            : "Некорректный формат времени!");
        return;
    }

    if (selectedDateValue == currentDate && start <=
DateTime.Now.TimeOfDay)
    {
        MessageBox.Show(Lang.lang == "en"
            ? "Booking start time must be later than
current time!"
            : "Время начала бронирования должно быть позже
текущего времени!");
        return;
    }

    if (start >= end)
    {
        MessageBox.Show(Lang.lang == "en"
            ? "Start time must be earlier than end time!"
            : "Время начала должно быть меньше времени
окончания!");
        return;
    }

    var selectedDateStr =
selectedDateValue.ToString("dd.MM.yyyy");
    if (!int.TryParse(computerid?.Text, out int
currentComputerId))
    {
        MessageBox.Show(Lang.lang == "en" ? "Invalid
computer id" : "Некорректный ID компьютера");
        return;
    }

    // безопасная проверка конфликта времени — пропускаем
записи с некорректными временными полями
    bool isComputerBusy = items.Any(item =>
    {
        if (item.ComputerId != currentComputerId) return
false;
        if (item.DateOrder != selectedDateStr) return
false;
        if (item.Status != "active") return false;

        if (!TimeSpan.TryParse(item.StartTime, out
TimeSpan exStart)) return false;
        if (!TimeSpan.TryParse(item.EndTime, out TimeSpan
exEnd)) return false;
    });

```

```

        return IsTimeOverlap(exStart, exEnd, start, end);
    });

    if (isComputerBusy)
    {
        MessageBox.Show(Lang.lang == "en"
            ? "This computer is already booked for the
selected time!"
            : "Этот компьютер уже занят на выбранное
время!");
        return;
    }

    bool hasTimeConflict = items.Any(item =>
    {
        if (item.UserId != currentUserId) return false;
        if (item.DateOrder != selectedDateStr) return
false;
        if (item.Status != "active") return false;

        if (!TimeSpan.TryParse(item.StartTime, out
TimeSpan exStart)) return false;
        if (!TimeSpan.TryParse(item.EndTime, out TimeSpan
exEnd)) return false;

        return IsTimeOverlap(exStart, exEnd, start, end);
    });

    if (hasTimeConflict)
    {
        MessageBox.Show(Lang.lang == "en"
            ? "You already have a booking for this time!"
            : "У вас уже есть бронирование на это время!");
        return;
    }

    // формируем заказ (Id не задаём – ID генерирует БД)
    Order el = new Order
    {
        UserId = currentUserId,
        ComputerId = currentComputerId,
        Status = "active",
        DateOrder = selectedDateStr,
        StartTime = starttime?.Text ?? string.Empty,
        EndTime = endtime?.Text ?? string.Empty
    };

    // сохраняем асинхронно
    await database.AddElementsInTableOrders(el.UserId,
el.ComputerId, el.Status, el.DateOrder, el.StartTime,
el.EndTime).ConfigureAwait(false);

    // показать результат на UI-потоке
    await Dispatcher.InvokeAsync(() =>

```

```

        {
            MessageBox.Show(Lang.lang == "en"
                ? "Record successfully added!"
                : "Запись успешно добавлена!");
            this.Close();
        });
    }
    catch (Exception ex)
    {
        Debug.WriteLine("WriteInfoByItem      error:      "      +
ex.Message);
        MessageBox.Show(Lang.lang      ==      "en"      ?      $"Error:
{ex.Message}" : $"Ошибка: {ex.Message}");
    }
    private void CostChange() {
        if (TimeSpan.TryParse(starttime?.Text, out TimeSpan start)
&&
            TimeSpan.TryParse(endtime?.Text,      out      TimeSpan
end))
        {
            double      totalCost      =      (end      -      start).TotalHours      *
_tariff.PricePerHour;
            double roundedCost = Math.Round(totalCost, 2);
            if (roundedCost >= 0)
            {
                costpc.Text = roundedCost.ToString() + (Lang.lang
== "en" ? " byn." : " pyб.");
            }
            else
            {
                costpc.Text = "0" + (Lang.lang == "en" ? " byn." :
" pyб.");
            }
        }
        else {
            costpc.Text = "0" + (Lang.lang == "en" ? " byn." : "
pyб.");
        }
    }
    public void FormattedPrice() {
        price.Text = _tariff.PricePerHour + " " + (Lang.lang ==
"en" ? "byn." : "pyб.");
    }
    public void CorrectRating() {
        rating.Text = Math.Round(_tariff.Rating, 2).ToString();
    }
    public void GenerateRamAndMonitor() {
        monitor.Text = _tariff.Monitor + " " + (Lang.lang == "en"
? "Hz" : "Гц");
        ram.Text = _tariff.Ram + " " + (Lang.lang == "en" ? "Gb" :
"Гб");
    }
}

```

```

        private bool IsTimeOverlap(TimeSpan existingStart, TimeSpan
existingEnd, TimeSpan newStart, TimeSpan newEnd)
        {
            return (newStart < existingEnd && newEnd > existingStart);
        }
        private void NumericTextBox_PreviewTextInput(object sender,
TextCompositionEventArgs e)
        {
            foreach (char c in e.Text)
            {
                if (!char.IsDigit(c) && c != ':')
                {
                    e.Handled = true;
                    return;
                }
            }

            TextBox textBox = sender as TextBox;
            string newText = textBox.Text.Insert(textBox.CaretIndex,
e.Text);

            if (newText.Count(c => c == ':') > 1)
            {
                e.Handled = true;
            }
        }
    }
}

```

LaunchingGames.xaml

```

<Window x:Class="COURSEPROJECT.LaunchingGames"

xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006"
    xmlns:gif="http://wpfanimatedgif.codeplex.com"
    xmlns:local="clr-namespace:COURSEPROJECT"
    mc:Ignorable="d"
    Title="{DynamicResource      Launchinggames}"      Height="720"
Width="1280"
    WindowStartupLocation="CenterScreen">
    <Grid>
        <Image Source="/images/animfonframe.png"
            Stretch="UniformToFill"
            Panel.ZIndex="-1"/>
        <Image x:Name="AnimatedGif"
            Source="{DynamicResource DynamicGifSource}"
            gif:ImageBehavior.AnimatedSource="{DynamicResource
DynamicGifSource}"
            gif:ImageBehavior.RepeatBehavior="Forever"
            Stretch="UniformToFill"

```

```

        Panel.ZIndex="-1"
        Opacity="0"
        Loaded="AnimatedGif_Loaded"/>
    <DockPanel LastChildFill="True">
        <StackPanel Margin="0 10 0 0" DockPanel.Dock="Top"
Height="50" Orientation="Horizontal" x:Name="StackHeader">
            <Border
                Style="{DynamicResource
FilteringSearchBorderStyle}">
                <TextBox
                    Style="{DynamicResource
TextBoxDataTrigger}" x:Name="ResearchPCInformation" MaxLength="20"/>
            </Border>
            <Button
                Style="{DynamicResource
TransparentButtonStyle}" Background="Transparent" BorderThickness="0"
x:Name="search" Click="SearchClick">
                <Image
                    Style="{DynamicResource
FilteringSearchImageStyle}" Source="/images/search2.png"/>
            </Button>
        </StackPanel>
        <ScrollViewer VerticalScrollBarVisibility="Auto">
            <UniformGrid DockPanel.Dock="Top" Margin="10 0 10 0"
x:Name="Items" Columns="4" Rows="1" VerticalAlignment="Top">

                </UniformGrid>
            </ScrollViewer>
        </DockPanel>
    </Grid>
</Window>

```

LaunchingGames.xaml.cs

```

using Microsoft.Xaml.Behaviors.Core;
using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.IO;
using System.Linq;
using System.Net.Http;
using System.Net;
using System.Text;
using System.Text.Json;
using System.Text.Json.Serialization;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Animation;
using System.Windows.Media.Imaging;
using System.Windows.Shapes;
using static MaterialDesignThemes.Wpf.Theme.ToolBar;
using System.Text.RegularExpressions;
using System.IO.Compression;

```

```

using System.Windows.Controls.Primitives;
using System.Diagnostics;
using System.IO.Compression;

namespace COURSEPROJECT
{
    public partial class LaunchingGames : Window
    {
        public LaunchingGames()
        {
            InitializeComponent();
            try
            {
                this.Cursor = new
Cursor(Application.GetResourceStream(new Uri("/images/cursor.cur",
UriKind.Relative)).Stream);
            }
            catch (Exception ex)
            {
                MessageBox.Show(Lang.lang == "en"
? $"Cursor loading error: {ex.Message}"
: $"Ошибка загрузки курсора: {ex.Message}");
            }
            _ = GeneratePCFromFileAsync();
        }
        private int rows = 0;
        private int columns = 0;
        private void AnimatedGif_Loaded(object sender, RoutedEventArgs
e)
        {
            var animation = new DoubleAnimation(0, 1,
TimeSpan.FromSeconds(0.5));
            AnimatedGif.BeginAnimation(OpacityProperty, animation);
        }

        public async Task<Border> GenerateInformation(string[] texts,
int currentRow, int currentColumn)
        {
            Border border = new Border
            {
                Margin = new Thickness(10),
                Background = new
SolidColorBrush((Color)ColorConverter.ConvertFromString("#8001464b"))
,
                BorderBrush = new
SolidColorBrush((Color)ColorConverter.ConvertFromString("#8001464b"))
,
                CornerRadius = new CornerRadius(10),
            };

            Grid grid = new Grid
            {
                Background = Brushes.Transparent
            };

```

```

        grid.RowDefinitions.Add(new RowDefinition { Height = new
GridLength(1, GridUnitType.Star) });
        grid.RowDefinitions.Add(new RowDefinition { Height =
GridLength.Auto });

        string title = texts[1];
        string img = texts[2];
        string description = texts[3];
        string link = texts[4];

        StackPanel infoPanel = new StackPanel
        {
            Orientation = Orientation.Vertical,
            Margin = new Thickness(0, 0, 0, 10)
        };

        TextBlock titleBlock = new TextBlock
        {
            HorizontalAlignment = HorizontalAlignment.Center,
            Text = title,
            Margin = new Thickness(0, 10, 0, 0),
            FontWeight = FontWeights.Bold,
            FontSize = 24,
            Foreground = Brushes.White,
            TextWrapping = TextWrapping.Wrap
        };

        Border imageBorder = new Border
        {
            CornerRadius = new CornerRadius(10),
            Width = 200,
            Height = 200,
            ClipToBounds = true,
            HorizontalAlignment = HorizontalAlignment.Center,
            Margin = new Thickness(0, 5, 0, 0),
            Child = new Image
            {
                Stretch = Stretch.UniformToFill,
                HorizontalAlignment = HorizontalAlignment.Center,
                VerticalAlignment = VerticalAlignment.Center
            }
        };

        try
        {
            BitmapImage bitmap = new BitmapImage();
            bitmap.BeginInit();
            bitmap.UriSource = new Uri(img,
UriKind.RelativeOrAbsolute);
            bitmap.CacheOption = BitmapCacheOption.OnLoad;
            bitmap.EndInit();

            if (bitmap.Width > 1)

```

```

        {
            ((Image)imageBorder.Child).Source = bitmap;
        }
    }
    catch (Exception ex)
    {
        Debug.WriteLine(Lang.lang == "en"
            ? $"Image loading error: {ex.Message}"
            : $"Ошибка загрузки изображения: {ex.Message}");
    }

    TextBlock descBlock = new TextBlock
    {
        Text = description,
        FontSize = 14,
        Foreground = Brushes.White,
        TextWrapping = TextWrapping.Wrap,
        Margin = new Thickness(0, 5, 0, 0),
        MaxWidth = 200
    };

    infoPanel.Children.Add(titleBlock);
    infoPanel.Children.Add(imageBorder);
    infoPanel.Children.Add(descBlock);

    Button launchButton = new Button
    {
        Content = await TranslateGoogleAsync("Запустить",
Lang.lang),
        Tag = link,
        HorizontalAlignment = HorizontalAlignment.Center,
        Padding = new Thickness(10, 5, 10, 5),
        Margin = new Thickness(0, 0, 0, 10),
        Background = new
SolidColorBrush((Color)ColorConverter.ConvertFromString("#FF007ACC"))
,
        Foreground = Brushes.White,
        Cursor = Cursors.Hand,
        Width = 200,
        BorderThickness = new Thickness(0)
    };

    launchButton.Click += async (sender, e) =>
    {
        try
        {
            // 1) проверка авторизации
            if (Global.CurrentUser == "Пользователь" || Global.CurrentUser
== "User")
            {
                MessageBox.Show(Lang.lang == "en"
                    ? "Please authorize to launch games."
                    : "Пожалуйста, авторизуйтесь, чтобы запускать игры.");
                return;
            }
        }
    }
}

```

```

    }

    // 2) проверка разрешения играть сейчас (активное бронирование)
    bool canPlay = await UserCanPlayAsync(Global.CurrentUser);

    // Разрешаем администратору запускать игры даже без активного
    бронирования
    if (!canPlay && !Global.CurrentUser.Equals("admin",
StringComparison.OrdinalIgnoreCase))
    {
        MessageBox.Show(Lang.lang == "en"
            ? "You don't have an active booking right now."
            : "У вас сейчас нет активного бронирования для запуска
игры.");
        return;
    }

    if (!(sender is Button button) || !(button.Tag is string url)
|| string.IsNullOrWhiteSpace(url))
    {
        MessageBox.Show(Lang.lang == "en"
            ? "Game path is missing."
            : "Путь к игре не указан.");
        return;
    }

    // 3) Попытка запустить: если это URL -> открыть в браузере;
    иначе — проверить файл
    try
    {
        // абсолютный путь (если относительный)
        string pathCandidate = url;
        string fullPath = pathCandidate;
        try
        {
            fullPath = System.IO.Path.GetFullPath(pathCandidate);
        }
        catch { /* оставляем как есть если Path.GetFullPath бросит
*/ }

        if (Uri.IsWellFormedUriString(url, UriKind.Absolute))
        {
            Process.Start(new ProcessStartInfo(url) {
UseShellExecute = true });
            LaunchedGames.PathsGames.Add(url);
            LaunchedGames.Id = await
UserProcessPlayAsync(Global.CurrentUser);
            return;
        }

        if (!File.Exists(fullPath))
        {
            MessageBox.Show(Lang.lang == "en"
                ? $"File not found: {fullPath}"

```

```

        : $"Файл не найден: {fullPath}");
        return;
    }

    var psi = new ProcessStartInfo(fullPath)
    {
        UseShellExecute = true,
        WorkingDirectory
System.IO.Path.GetDirectoryName(fullPath)
    };

    Process proc = Process.Start(psi);
    if (proc != null)
    {
        LaunchedGames.PathsGames.Add(fullPath);
        LaunchedGames.Id
UserProcessPlayAsync(Global.CurrentUser);
    }
    else
    {
        MessageBox.Show(Lang.lang == "en"
            ? "Failed to start process."
            : "Не удалось запустить процесс.");
    }
}
catch (Exception ex)
{
    MessageBox.Show(Lang.lang == "en"
        ? $"Error starting game: {ex.Message}"
        : $"Ошибка при запуске игры: {ex.Message}");
}
}
catch (Exception ex)
{
    Debug.WriteLine("Launch button click error: " + ex.Message);
}
};

Grid.SetRow(infoPanel, 0);
Grid.SetRow(launchButton, 1);

grid.Children.Add(infoPanel);
grid.Children.Add(launchButton);

border.Child = grid;

return border;
}

private void SearchClick(object sender, EventArgs e)
{
    _ = GeneratePCFromFileAsync();
}

```

```

private async Task<bool> UserCanPlayAsync(string name)
{
    try
    {
        var userid =
database.GetUserIdByName(Global.CurrentUser);
        if (!int.TryParse(userid, out int uid)) return false;

        var orders = await
database.ReadElementsInTableOrdersAsync();
        foreach (Order item in orders)
        {
            if (item.UserId == uid && item.Status == "active")
            {
                if (DateTime.TryParse($"{item.DateOrder}
{item.StartTime}", out DateTime startDateTime) &&
                    DateTime.TryParse($"{item.DateOrder}
{item.EndTime}", out DateTime endDateTime))
                {
                    DateTime now = DateTime.Now;
                    if (now >= startDateTime && now <=
endDateTime)
                    {
                        return true;
                    }
                }
            }
        }
    }
    catch (Exception ex)
    {
        Debug.WriteLine("UserCanPlayAsync error: " +
ex.Message);
    }
    return false;
}

private async Task<string> UserProcessPlayAsync(string name)
{
    try
    {
        var userid =
database.GetUserIdByName(Global.CurrentUser);
        if (!int.TryParse(userid, out int userId)) return "";

        var orders = await
database.ReadElementsInTableOrdersAsync();
        foreach (Order item in orders)
        {
            if (item.UserId == userId && item.Status ==
"active")
            {
                if (!string.IsNullOrEmpty(item.DateOrder) &&
                    !string.IsNullOrEmpty(item.StartTime) &&

```



```

        using (var stream = new
MemoryStream(responseData))
        using (var gzip = new GZipStream(stream,
CompressionMode.Decompress))
        using (var reader = new StreamReader(gzip))
        {
            string response = reader.ReadToEnd();
            return ParseTranslation(response);
        }
    }
    else
    {
        string response =
Encoding.UTF8.GetString(responseData);
        return ParseTranslation(response);
    }
}
catch (Exception ex)
{
    Debug.WriteLine("TranslateGoogleAsync error: " +
ex.Message);
    return text;
}
}

private bool IsGameOrApp(string url)
{
    if (string.IsNullOrEmpty(url))
        return false;

    try
    {
        string lower = url.ToLowerInvariant();

        // 1) Локальные файлы/ярлыки/исполняемые файлы
        string[] execExt = { ".exe", ".bat", ".lnk", ".msi",
".appx" };
        if (execExt.Any(ext => lower.EndsWith(ext)))
            return true;

        // 2) Явное упоминание слова "game"/"игра" в имени или
пути
        if (lower.Contains("game") || lower.Contains("игра")
|| lower.Contains("игр"))
            return true;

        // 3) URL-хосты известных игровых площадок или пути,
содержащие game/play
        if (Uri.IsWellFormedUriString(url, UriKind.Absolute))
        {
            var uri = new Uri(url);
            string host = uri.Host.ToLowerInvariant();
            string path = uri.AbsolutePath.ToLowerInvariant();

```

```

        string[] gamingHosts = { "store.steampowered.com",
"steamcommunity.com", "epicgames.com", "gog.com", "itch.io",
"humblebundle.com", "origin.com", "uplay.com" };
        if (gamingHosts.Any(h => host.Contains(h) ||
host.EndsWith(h)))
            return true;

        if (path.Contains("/game") ||
path.Contains("/games") || path.Contains("/play"))
            return true;

        // если в конце URL имя файла с расширением
        string fileName =
System.IO.Path.GetFileName(path);
        if (!string.IsNullOrEmpty(fileName) &&
execExt.Any(ext => fileName.EndsWith(ext)))
            return true;
    }
    catch
    {
        // в случае исключения считаем, что это не игра (без
краха)
    }

    return false;
}
private static readonly string[] GamingGpuKeywords = { "RTX",
"GTX", "RX", "Radeon", "GeForce" };

private async Task<bool> IsUserHaveNoActiveAsync()
{
    try
    {
        var userIdStr =
database.GetUserIdByName(Global.CurrentUser);
        if (string.IsNullOrEmpty(userIdStr)) return true;

        var today = DateTime.Today.ToString("dd.MM.yyyy");

        var orders = await
database.ReadElementsInTableOrdersAsync();
        var activeOrder = orders.FirstOrDefault(o =>
            o.UserId == int.Parse(userIdStr) &&
            o.Status == "active" &&
            o.DateOrder == today &&
            TimeSpan.TryParse(o.StartTime, out TimeSpan s) &&
            TimeSpan.TryParse(o.EndTime, out TimeSpan en) &&
            IsTimeInRange(s, en));

        return activeOrder == null;
    }
    catch (Exception ex)

```

```

        {
            Debug.WriteLine("IsUserHaveNoActiveAsync error: " +
ex.Message);
            return true;
        }
    }

    private async Task<bool> IsGamingComputerAsync(string
userName)
    {
        try
        {
            var userId = database.GetUserIdByName(userName);
            if (string.IsNullOrEmpty(userId)) return false;

            var today = DateTime.Today.ToString("dd.MM.yyyy");

            var orders = await
database.ReadElementsInTableOrdersAsync();
            var activeOrder = orders.FirstOrDefault(o =>
                o.UserId == int.Parse(userId) &&
                o.Status == "active" &&
                o.DateOrder == today &&
                TimeSpan.TryParse(o.StartTime, out TimeSpan s) &&
                TimeSpan.TryParse(o.EndTime, out TimeSpan en) &&
                IsTimeInRange(s, en));

            if (activeOrder == null) return false;

            var computers = await
database.ReadElementsInTableComputersAsync();
            var computer = computers.FirstOrDefault(c => c.Id ==
activeOrder.ComputerId);
            if (computer == null) return false;

            return GamingGpuKeywords.Any(keyword =>
                computer.Gpu.IndexOf(keyword,
StringComparison.OrdinalIgnoreCase) >= 0);
        }
        catch (Exception ex)
        {
            Debug.WriteLine("IsGamingComputerAsync error: " +
ex.Message);
            return false;
        }
    }

    public bool IsTimeInRange(TimeSpan startTime, TimeSpan
endTime)
    {
        TimeSpan currentTime = DateTime.Now.TimeOfDay;

        if (startTime <= endTime)
        {

```

```

        return currentTime >= startTime && currentTime <=
endTime;
    }
    else
    {
        return currentTime >= startTime || currentTime <=
endTime;
    }
}

private string ParseTranslation(string response)
{
    try
    {
        int start = response.IndexOf("\"",
StringComparison.Ordinal) + 1;
        int end = response.IndexOf("\"", start);
        return end > start ? response.Substring(start, end -
start) : response;
    }
    catch
    {
        return response;
    }
}

private async Task GeneratePCFromFileAsync()
{
    try
    {
        Items.Children.Clear();
        var list = await
database.ReadElementsInTableApplicationGameAsync();
        Items.Rows = (int)Math.Ceiling((double)list.Count /
4);

        foreach (ApplicationGame item in list)
        {
            if (Global.CurrentUser == "Пользователь" ||
Global.CurrentUser == "User" || await IsUserHaveNoActiveAsync())
            {
                string translatedDesc = await
TranslateGoogleAsync(item.Description, Lang.lang);

                List<string> array = new List<string>
                {
                    item.Id.ToString(),
                    item.Name,
                    item.IMG,
                    translatedDesc,
                    item.URL
                };
            }
        }
    }
}

```

```

        if
(string.IsNullOrEmpty(ResearchPCInformation.Text))
        {
            var border = await
GenerateInformation(array.ToArray(), rows, columns);
            Items.Children.Add(border);
        }
        else if
(item.Name.IndexOf(ResearchPCInformation.Text,
StringComparison.OrdinalIgnoreCase) >= 0)
        {
            var border = await
GenerateInformation(array.ToArray(), rows, columns);
            Items.Children.Add(border);
        }
        else
        {
            if (await
IsGamingComputerAsync(Global.CurrentUser))
            {
                string translatedDesc = await
TranslateGoogleAsync(item.Description, Lang.lang);

                List<string> array = new List<string>
                {
                    item.Id.ToString(),
                    item.Name,
                    item.IMG,
                    translatedDesc,
                    item.URL
                };
                if (IsGameOrApp(array[4]))
                {
                    if
(string.IsNullOrEmpty(ResearchPCInformation.Text))
                    {
                        var border = await
GenerateInformation(array.ToArray(), rows, columns);
                        Items.Children.Add(border);
                    }
                    else if
(item.Name.IndexOf(ResearchPCInformation.Text,
StringComparison.OrdinalIgnoreCase) >= 0)
                    {
                        var border = await
GenerateInformation(array.ToArray(), rows, columns);
                        Items.Children.Add(border);
                    }
                }
            }
            else
            {

```

```

        string translatedDesc = await
TranslateGoogleAsync(item.Description, Lang.lang);

        List<string> array = new List<string>
        {
            item.Id.ToString(),
            item.Name,
            item.IMG,
            translatedDesc,
            item.URL
        };
        if (!IsGameOrApp(array[4]))
        {
            if
(string.IsNullOrEmpty(ResearchPCInformation.Text))
            {
                var border = await
GenerateInformation(array.ToArray(), rows, columns);
                Items.Children.Add(border);
            }
            else if
(item.Name.IndexOf(ResearchPCInformation.Text,
StringComparison.OrdinalIgnoreCase) >= 0)
            {
                var border = await
GenerateInformation(array.ToArray(), rows, columns);
                Items.Children.Add(border);
            }
        }
    }
}
catch (Exception ex)
{
    Debug.WriteLine("GeneratePCFromFileAsync error: " +
ex.Message);
}
}
}

```

MainWindow.xaml

```

<Window x:Class="COURSEPROJECT.MainWindow"

xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006"
xmlns:local="clr-namespace:COURSEPROJECT"
mc:Ignorable="d"

```

```

xmlns:materialDesign="http://materialdesigninxaml.net/winfx/xaml/themes"
    Title="{DynamicResource Mainwindow}" Height="720" Width="1280"
    WindowStartupLocation="CenterScreen">

    <Grid Background="#3f3f3f">
        <DockPanel LastChildFill="True">
            <Border DockPanel.Dock="Left" MinWidth="60"
VerticalAlignment="Stretch" Background="#3f3f3f">
                <DockPanel LastChildFill="False" Margin="10">
                    <Image Width="60" Source="/images/translogo.png"
DockPanel.Dock="Top" Margin="0 30 0 0"/>
                    <Button Style="{DynamicResource
TransparentButtonStyle}" Command="{Binding OpenMapbookingCommand}"
Background="Transparent" BorderThickness="0" DockPanel.Dock="Top">
                        <Image Width="60" Source="/images/time.png"
Margin="0 15 0 0" x:Name="UserBooking" Visibility="Collapsed"/>
                    </Button>
                    <Button Style="{DynamicResource
TransparentButtonStyle}" Command="{Binding OpenGamesCommand}"
Background="Transparent" BorderThickness="0" DockPanel.Dock="Top">
                        <Image Width="60" Source="/images/game.png"
DockPanel.Dock="Top" Margin="0 5 0 0"/>
                    </Button>
                    <Button Style="{DynamicResource
TransparentButtonStyle}" Command="{Binding OpenAdminTableCommand}"
Background="Transparent" BorderThickness="0" DockPanel.Dock="Top">
                        <Image Width="60"
Source="/images/database_icon.png" Margin="0 5 0 0"
x:Name="AdminDBTable" Visibility="Collapsed"/>
                    </Button>
                    <local:MusicValueElement DockPanel.Dock="Bottom"
/>
                    <TextBlock DockPanel.Dock="Bottom" Text=""
Height="10"/>
                    <local:RadiusButton DockPanel.Dock="Bottom"
Padding="0 10 0 0" Click="SwitchMusicStatus" />
                </DockPanel>
            </Border>
            <Border HorizontalAlignment="Stretch"
VerticalAlignment="Stretch">
                <Border.Background>
                    <DynamicResource ResourceKey="mainfon"/>
                </Border.Background>
                <DockPanel LastChildFill="True">
                    <DockPanel LastChildFill="False" Width="auto"
DockPanel.Dock="Top" MinHeight="80">
                        <Border Style="{DynamicResource
RoundedBorderStyle}" Width="300" Height="50" Margin="30 0 0 0">
                            <TextBox x:Name="ResearchPCInformation"
Style="{DynamicResource TextBoxDataTrigger}" MaxLength="20"/>
                        </Border>

```

```

<Button Style="{DynamicResource
TransparentButtonStyle}" Command="{Binding ReseachPCCommand}"
Background="Transparent" BorderThickness="0">
    <Image Width="60"
Source="/images/search2.png" Margin="10 0 0 0"/>
</Button>
<StackPanel Orientation="Horizontal"
Margin="10 0 0 0" Style="{DynamicResource UserStackPanelStyle}">
    <Button Width="90"
Content="{DynamicResource Russian}" Click="SwitchToRussian"
Style="{DynamicResource AnimationLangBtn}" Template="{DynamicResource
LangButtonTemplate}"/>
    <Button Width="90"
Content="{DynamicResource English}" Click="SwitchToEnglish"
Style="{DynamicResource AnimationLangBtn}" Template="{DynamicResource
LangButtonTemplate}"/>
</StackPanel>
<Button Width="140" Content="{DynamicResource
ChangeTheme}" Command="{Binding ChangeThemeBtnCommand}"
Style="{DynamicResource AnimationLangBtn}" Template="{DynamicResource
ThemeButtonTemplate}"/>
<Button DockPanel.Dock="Right"
Style="{DynamicResource TransparentButtonStyle}"
Background="Transparent" BorderThickness="0" Padding="0">
    <Border Margin="0 0 30 0"
CornerRadius="10" MinWidth="200" Height="70" Style="{DynamicResource
UserButtonStyle}">
        <StackPanel Style="{DynamicResource
UserStackPanelStyle}">
            <Button Style="{DynamicResource
TransparentButtonStyle}" Command="{Binding OpenAuthorizationCommand}"
Background="Transparent" BorderThickness="0">
                <Button.ToolTip>
                    <ToolTip>
                        <TextBlock
Text="{DynamicResource Auth}"/>
                    </ToolTip>
                </Button.ToolTip>
                <Image Margin="10 0 0 0"
Width="40" Source="/images/user.png"/>
            </Button>
            <Button Style="{DynamicResource
TransparentButtonStyle}" Command="{Binding OpenPersonalCommand}"
Background="Transparent" BorderThickness="0" Visibility="Collapsed">
                <TextBlock x:Name="UserName"
Style="{DynamicResource UserNameTextBlockStyle}" Text=""
HorizontalAlignment="Center"/>
            </Button>
            <Button Style="{DynamicResource
TransparentButtonStyle}" Command="{Binding OpenPersonalCommand}"
Background="Transparent" BorderThickness="0">
                <Button.ToolTip>

```

```

                                <ToolTip
x:Name="ToolPersonal" Style="{DynamicResource CustomToolTip}">
                                <TextBlock
Text="{DynamicResource Personalaccount}"/>
                                </ToolTip>
                                </Button.ToolTip>
                                <TextBlock
x:Name="UserNameView" Style="{DynamicResource UserNameTextBlockStyle}"
Text="{DynamicResource user}" HorizontalAlignment="Center"/>
                                </Button>
                                </StackPanel>
                                </Border>
                                </Button>
                                </DockPanel>
                                <ScrollViewer VerticalScrollBarVisibility="Auto">
                                <Grid DockPanel.Dock="Top" Margin="10 0 10 0"
x:Name="ContainerPc">
                                <Grid.RowDefinitions>
                                    <RowDefinition Height="Auto"/>
                                    <RowDefinition Height="Auto"/>
                                    <RowDefinition Height="Auto"/>
                                    <RowDefinition Height="Auto"/>
                                    <RowDefinition Height="Auto"/>
                                    <RowDefinition Height="Auto"/>
                                </Grid.RowDefinitions>
                                <Grid.ColumnDefinitions>
                                    <ColumnDefinition Width="*" />
                                    <ColumnDefinition Width="*" />
                                    <ColumnDefinition Width="*" />
                                </Grid.ColumnDefinitions>
                                </Grid>
                                </ScrollViewer>
                                </DockPanel>
                                </Border>
                                </DockPanel>
                                </Grid>
</Window>

```

MainWindow.xaml.cs

```

using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Text;
using System.Text.Json;
using System.Threading;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;

```

```

using System.Windows.Media.Imaging;
using System.Windows.Navigation;
using System.Windows.Shapes;
using System.Text.Json.Serialization;
using System.Runtime.CompilerServices;
using System.ComponentModel;
using System.Windows.Threading;
using System.Diagnostics;
using MaterialDesignThemes.Wpf;
using System.Net.Http;
using System.Web;
using System.Runtime.InteropServices;
using System.Windows.Interop;

namespace COURSEPROJECT
{
    public class RelayCommand : ICommand
    {
        private readonly Action<object> _execute;
        private readonly Predicate<object> _canExecute;

        public RelayCommand(Action<object> execute, Predicate<object>
canExecute = null)
        {
            _execute = execute ?? throw new
ArgumentNullException(nameof(execute));
            _canExecute = canExecute;
        }

        public bool CanExecute(object parameter)
        {
            return _canExecute == null || _canExecute(parameter);
        }

        public void Execute(object parameter)
        {
            _execute(parameter);
        }

        public event EventHandler CanExecuteChanged
        {
            add { CommandManager.RequerySuggested += value; }
            remove { CommandManager.RequerySuggested -= value; }
        }
    }

    public static class LaunchedGames
    {
        public static string Id;
        public static List<string> PathsGames = new List<string>();
    }

    public static class Global
    {

```

```

        public static string CurrentUser = Lang.lang == "en" ? "User"
: "Пользователь";
    };

    public partial class MainWindow : Window
    {
        public DispatcherTimer timer;
        public DispatcherTimer timerupd;

        public ICommand OpenAuthorizationCommand { get; }
        public ICommand OpenEditingCommand { get; }
        public ICommand OpenFilteringCommand { get; }
        public ICommand OpenAddCommand { get; }
        public ICommand ReseachPCCommand { get; }
        public ICommand ChangeThemeBtnCommand { get; }
        public ICommand OpenPersonalCommand { get; }
        public ICommand OpenMapbookingCommand { get; }
        public ICommand OpenGamesCommand { get; }
        public ICommand OpenAdminTableCommand { get; }

        // для debounce поиска
        private CancellationTokenSource _searchCts;

        public MainWindow()
        {
            InitializeComponent();

            DataContext = this;

            OpenAuthorizationCommand = new
RelayCommand(OpenAuthorization);
            ReseachPCCommand = new RelayCommand(ReseachPC);
            ChangeThemeBtnCommand = new RelayCommand(ChangeThemeBtn);
            OpenPersonalCommand = new RelayCommand(OpenPersonal);
            OpenMapbookingCommand = new RelayCommand(OpenMapbooking);
            OpenGamesCommand = new RelayCommand(OpenGames);
            OpenAdminTableCommand = new RelayCommand(OpenAdminTable);

            try
            {
                this.Cursor = new
Cursor(Application.GetResourceStream(new Uri("/images/cursor.cur",
UriKind.Relative)).Stream);
            }
            catch (Exception ex)
            {
                MessageBox.Show(Lang.lang == "en"
? $"Cursor loading error: {ex.Message}"
: $"Ошибка загрузки курсора: {ex.Message}");
            }

            // первоначальная загрузка
            generatePCfromFile();

```

```

        // Таймер для лёгких UI-обновлений (не частый)
        timer = new DispatcherTimer();
        timer.Interval = TimeSpan.FromSeconds(1); // уменьшил
частоту (бывшие 0.1s слишком часто)
        timer.Tick += Timer_Tick;
        timer.Start();

        // Таймер обновления рейтингов — выполняем реже и в фоне
        timerupd = new DispatcherTimer();
        timerupd.Interval = TimeSpan.FromMinutes(1); //
выполняется раз в минуту
        timerupd.Tick += async (s, e) => await
UpdateAllComputersRatingAsync();
        timerupd.Start();

        DependencyPropertyDescriptor descriptor =
DependencyPropertyDescriptor.FromProperty(TextBlock.TextProperty,
typeof(TextBlock));

        descriptor.AddValueChanged(Username, (sender, args) =>
        {
            if (Username.Text != "Пользователь" && Username.Text
!= "User")
            {
                UsernameView.Text = Username.Text;
            }
        });

        // Подписка на изменение текста поиска — debounce
        if (ResearchPCInformation != null)
        {
            ResearchPCInformation.TextChanged +=
ResearchPCInformation_TextChanged;
        }

        Music.PlayMusic();
        CommandBindings.Add
            (new CommandBinding(CustomCommand.OpenProfileWindow,
OpenPersonalEvent));
        CommandBindings.Add
            (new CommandBinding(CustomCommand.SwitchTheme,
ChangeThemeBtnEvent));
        this.AddHandler(ComputerClickedEvent, new
RoutedEventHandler(OnComputerClicked));
        this.Closing += MainWindow_Closing;
    }

    [DllImport("Dwmapi.dll")]
    public static extern int DwmSetWindowAttribute(IntPtr hwnd, int
attr, ref int attrValue, int attrSize);

```

```

public static readonly RoutedEvent ComputerClickedEvent =
    EventManager.RegisterRoutedEvent(
        "ComputerClicked",
        RoutingStrategy.Bubble,
        typeof(RoutedEventHandler),
        typeof(MainWindow));

public event RoutedEventHandler ComputerClicked
{
    add => AddHandler(ComputerClickedEvent, value);
    remove => RemoveHandler(ComputerClickedEvent, value);
}

private void ChangeThemeBtn(object parametr)
{
    var currentTheme = Properties.Settings.Default.Theme;
    var newTheme = currentTheme == "StyleTheme1" ?
"StyleTheme2" : "StyleTheme1";
    ThemeManager.ApplyTheme(newTheme);
}

private void ChangeThemeBtnEvent(object sender, EventArgs e)
{
    var currentTheme = Properties.Settings.Default.Theme;
    var newTheme = currentTheme == "StyleTheme1" ?
"StyleTheme2" : "StyleTheme1";
    ThemeManager.ApplyTheme(newTheme);
}

// Асинхронная безопасная переработка обновления рейтингов
public async Task UpdateAllComputersRatingAsync()
{
    try
    {
        // Снимем тяжёлую агрегацию на фон
        var computerRatings = await Task.Run(() =>
        {
            List<Reviews> allReviews =
database.ReadElementsInTableReviews();
            var grouped = allReviews
                .GroupBy(r => r.ComputerId)
                .Select(g => new
                {
                    ComputerId = g.Key,
                    AverageRating =
(float)Math.Round(g.Average(r => r.Rating), 2),
                    ReviewCount = g.Count()
                })
                .ToList();
            return grouped;
        });

        // Получаем список компьютеров один раз в фоне
        var allComputers = await Task.Run(() =>
database.ReadElementsInTableComputers());
    }
}

```

```

        // Выполняем обновления БД в фоне последовательно
(можно улучшить батчем)
        foreach (var computer in allComputers)
        {
            var ratingInfo = computerRatings.FirstOrDefault(cr
=> cr.ComputerId == computer.Id);
            float newRating = ratingInfo?.AverageRating ?? 0f;

            // Используем существующий асинхронный метод
обновления
            await database.UpdateElementInTableComputers(
                computer.Id,
                computer.Name,
                computer.Description,
                computer.Cpu,
                computer.Gpu,
                computer.Ram,
                computer.Storage,
                computer.Monitor,
                newRating,
                (float)computer.PricePerHour,
                computer.GraphicData);
        }
    }
    catch (Exception ex)
    {
        // Логируем в Output, показывать MessageBox слишком
агрессивно для периодической задачи
        Debug.WriteLine("UpdateAllComputersRatingAsync error:
" + ex.Message);
    }
}

private void Timer_Tick(object sender, EventArgs e)
{
    // лёгкие операции — не дергаем БД здесь напрямую
    if (UserName.Text != Global.CurrentUser)
    {
        UserName.Text = Global.CurrentUser;
    }
    IsAdmin();
    isRegUser();

    // Проверка статусов заказов делается асинхронно
    _ = CheckOrdersAndNotifyAsync(); // fire-and-forget —
ошибки логируются внутри
}

private void IsAdmin()
{
    if (Global.CurrentUser == "admin")
    {
        AdminDBTable.Visibility = Visibility.Visible;
    }
}

```

```

        }
        else
        {
            AdminDBTable.Visibility = Visibility.Collapsed;
        }
    }
    private void isRegUser()
    {
        if (Global.CurrentUser == "Пользователь" ||
Global.CurrentUser == "User")
        {
            UserBooking.Visibility = Visibility.Collapsed;
            ToolPersonal.Visibility = Visibility.Collapsed;
        }
        else
        {
            UserBooking.Visibility = Visibility.Visible;
            ToolPersonal.Visibility = Visibility.Visible;
        }
    }

    // Асинхронно получить заказы и выполнить UI-операции корректно
    на UI-потоке
    private async Task CheckOrdersAndNotifyAsync()
    {
        try
        {
            var orders = await Task.Run(() =>
database.ReadElementsInTableOrders());
            DateTime now = DateTime.Now;

            // Перенести минимальную логику на UI-поток, чтобы
            безопасно создавать окна/MessageBox
            await Dispatcher.InvokeAsync(() =>
            {
                foreach (var item in orders)
                {
                    if (item.Status == "active" &&
DateTime.TryParse($"{item.DateOrder} {item.EndTime}", out DateTime
endTime))
                    {
                        DateTime.TryParse(item.StartTime, out
DateTime startTime);
                        DateTime.TryParse(item.EndTime, out
DateTime endTime);
                        bool isCurrentUserBooking =
Global.CurrentUser == database.GetUserNameById(item.UserId);

                        if (DateTime.TryParse(item.DateOrder, out
DateTime rd) &&
                            TimeSpan.TryParse(item.StartTime, out
TimeSpan st) &&
                                TimeSpan.TryParse(item.EndTime, out
TimeSpan et))

```

```

        {
            DateTime sdt = rd.Date + st;
            DateTime edt = rd.Date + et;

            bool isc = Global.CurrentUser ==
database.GetUserNameById(item.UserId);
            if (isc && now >= sdt && now <=
sdt.AddSeconds(2))
            {
                if (App.LaunchingGamesWindow !=
null)
                {
                    App.LaunchingGamesWindow.Close();
                    App.LaunchingGamesWindow =
null;
                }
            }

            if (now > endDateTime)
            {
                item.Status = "completed";
                var review =
Application.Current.Windows.OfType<WritingReview>().FirstOrDefault();
                if (isCurrentUserBooking)
                {
                    if (review == null)
                    {
                        review = new
WritingReview(item);
                        review.Show();
                    }
                    else
                    {
                        review.Activate();
                        if (review.WindowState ==
WindowState.Minimized)
                        {
                            review.WindowState =
WindowState.Normal;
                        }
                    }
                    // Обновление статуса в БД
                    выполняем асинхронно (fire-and-forget)
                    database.UpdateElementInTableOrders(item.Id, item.UserId,
item.ComputerId, item.Status, item.DateOrder, item.StartTime,
item.EndTime);
                }
            }

            if (now > endDateTime &&
Global.CurrentUser == database.GetUserNameById(item.UserId))
            {
                item.Status = "completed";
            }
        }

```

```

// Корректное завершение процессов
лучше вынести отдельно; здесь - минимальная логика
if
(!string.IsNullOrEmpty(LaunchedGames.Id))
{
    int launchedGameId;
    if
    (int.TryParse(LaunchedGames.Id, out launchedGameId))
    {
        foreach (string expectedPath
in LaunchedGames.PathsGames)
        {
            foreach (Process proc in
Process.GetProcessesByName(System.IO.Path.GetFileNameWithoutExtension
(expectedPath)))
            {
                try
                {
                    // сначала мягкое
                    завершение
                    try
                    {
                        proc.CloseMainWindow();

                        if
                        (!proc.WaitForExit(2000))
                        {
                            proc.Kill();
                            proc.WaitForExit();
                        }
                    }
                    catch
                    {
                        proc.Kill();
                    }
                }
                catch (Exception ex)
                {
                    Debug.WriteLine(Lang.lang == "en"
? $"Error
terminating process: {ex.Message}"
: $"Ошибка при
завершении процесса: {ex.Message}");
                }
            }
        }
    }
}

```

```

        if (App.LaunchingGamesWindow != null)
        {
            App.LaunchingGamesWindow.Close();
            App.LaunchingGamesWindow = null;
        }

        var review =
Application.Current.Windows.OfType<WritingReview>().FirstOrDefault();
        if (isCurrentUserBooking)
        {
            if (review == null)
            {
                review = new
WritingReview(item);

                review.Show();
            }
            else
            {
                review.Activate();
                if (review.WindowState ==
WindowState.Minimized)
                    review.WindowState =
WindowState.Normal;
            }
        }

        database.UpdateElementInTableOrders(item.Id, item.UserId,
item.ComputerId, item.Status, item.DateOrder, item.StartTime,
item.EndTime);
    }
}

});
}
}

catch (Exception ex)
{
    Debug.WriteLine("CheckOrdersAndNotifyAsync error: " +
ex.Message);
}

private void MainWindow_Closing(object sender,
System.ComponentModel.CancelEventArgs e)
{
    CloseAllGames();
}

private void CloseAllGames()
{
    if (!string.IsNullOrEmpty(LaunchedGames.Id))
    {
        int launchedGameId;

```

```

        if (int.TryParse(LaunchedGames.Id, out
launchedGameId))
        {
            foreach (string expectedPath in
LaunchedGames.PathsGames)
            {
                foreach (Process proc in
Process.GetProcessesByName(System.IO.Path.GetFileNameWithoutExtension
(expectedPath)))
                {
                    try
                    {
                        // сначала попытаться корректно
закрыть
                        try
                        {
                            proc.CloseMainWindow();
                            if (!proc.WaitForExit(2000))
                            {
                                proc.Kill();
                                proc.WaitForExit();
                            }
                        }
                        catch
                        {
                            proc.Kill();
                            proc.WaitForExit();
                        }
                    }
                    catch (Exception ex)
                    {
                        Debug.WriteLine(Lang.lang == "en"
? $"Process termination error:
{ex.Message}"
: $"Ошибка при завершении
процесса: {ex.Message}");
                    }
                }
            }
        }
    }

    private void OpenAuthorization(object parameter)
    {
        var window =
Application.Current.Windows.OfType<authorization>().FirstOrDefault();

        if (window == null)
        {
            window = new authorization();
            window.Show();
        }
        else

```

```

        {
            window.Activate();
            if (window.WindowState == WindowState.Minimized)
                window.WindowState = WindowState.Normal;
        }
    }
    private void OpenPersonal(object parameter)
    {
        if (Global.CurrentUser != "Пользователь" &&
Global.CurrentUser != "User")
        {

            var window =
Application.Current.Windows.OfType<PersonalAccount>().FirstOrDefault(
);

            if (window == null)
            {
                window = new PersonalAccount();
                window.Show();
            }
            else
            {
                window.Activate();
                if (window.WindowState == WindowState.Minimized)
                    window.WindowState = WindowState.Normal;
            }
        }
    }
    private void OpenPersonalEvent(object sender, EventArgs e)
    {
        if (Global.CurrentUser != "Пользователь" &&
Global.CurrentUser != "User")
        {
            var window =
Application.Current.Windows.OfType<PersonalAccount>().FirstOrDefault(
);

            if (window == null)
            {
                window = new PersonalAccount();
                window.Show();
            }
            else
            {
                window.Activate();
                if (window.WindowState == WindowState.Minimized)
                    window.WindowState = WindowState.Normal;
            }
        }
    }
    private void OpenMapbooking(object parameter)
    {

```

```

var window =
Application.Current.Windows.OfType<mapbooking>().FirstOrDefault();

    if (window == null)
    {
        window = new mapbooking();
        window.Show();
    }
    else
    {
        window.Activate();
        if (window.WindowState == WindowState.Minimized)
            window.WindowState = WindowState.Normal;
    }
}
private void OpenGames(object parameter)
{
    var window =
Application.Current.Windows.OfType<LaunchingGames>().FirstOrDefault()
;

    if (window == null)
    {
        window = new LaunchingGames();
        App.LaunchingGamesWindow = window;
        window.Show();
    }
    else
    {
        window.Activate();
        if (window.WindowState == WindowState.Minimized)
            window.WindowState = WindowState.Normal;
    }
}
private void OpenAdminTable(object parameter)
{
    var window =
Application.Current.Windows.OfType<AdminTable>().FirstOrDefault();

    if (window == null)
    {
        window = new AdminTable();
        window.Show();
    }
    else
    {
        window.Activate();
        if (window.WindowState == WindowState.Minimized)
            window.WindowState = WindowState.Normal;
    }
}

private void ReseachPC(object parameter)
{

```

```

        // Вызов асинхронной версии обновления списка компьютеров
        _ = RefreshComputersAsync();
    }

    public void ReseachPCPublic()
    {
        // устаревший синхронный вызов — заменяем на асинхронный
        _ = RefreshComputersAsync();
    }

    // Общая асинхронная реализация загрузки/фильтрации
    компьютеров + UI-обновление
    private async Task RefreshComputersAsync()
    {
        try
        {
            string searchText = string.Empty;
            await Dispatcher.InvokeAsync(() => searchText =
ResearchPCInformation?.Text?.Trim() ?? string.Empty);

            // читаем список компьютеров в фоне
            var list = await Task.Run(() =>
database.ReadElementsInTableComputers());

            // фильтруем в фоне
            var filtered = list.Where(item =>
            {
                if (string.IsNullOrEmpty(searchText)) return true;

                return (item.Name?.IndexOf(searchText,
StringComparison.OrdinalIgnoreCase) ?? -1) >= 0 ||
(item.Description?.IndexOf(searchText,
StringComparison.OrdinalIgnoreCase) ?? -1) >= 0 ||
(item.Cpu?.IndexOf(searchText,
StringComparison.OrdinalIgnoreCase) ?? -1) >= 0 ||
(item.Gpu?.IndexOf(searchText,
StringComparison.OrdinalIgnoreCase) ?? -1) >= 0 ||
(item.Ram?.IndexOf(searchText,
StringComparison.OrdinalIgnoreCase) ?? -1) >= 0 ||
(item.Storage?.IndexOf(searchText,
StringComparison.OrdinalIgnoreCase) ?? -1) >= 0 ||
(item.Monitor?.IndexOf(searchText,
StringComparison.OrdinalIgnoreCase) ?? -1) >= 0 ||

(item.Rating.ToString().IndexOf(searchText,
StringComparison.OrdinalIgnoreCase) >= 0) ||

(item.PricePerHour.ToString().IndexOf(searchText,
StringComparison.OrdinalIgnoreCase) >= 0);
            }).ToList();

            // обновляем UI на UI-потоке
            await Dispatcher.InvokeAsync(() =>
            {

```

```

        ContainerPc.Children.Clear();
        int column = 0;
        int row = 0;
        foreach (Computer item in filtered)
        {
            List<string> array = new List<string>
            {
                item.Name,
                item.Description,
                item.Cpu,
                item.Gpu,
                item.Ram,
                item.Storage,
                item.Monitor,
                Math.Round(item.Rating, 2).ToString(),
                item.PricePerHour.ToString() + (Lang.lang
== "en" ? " byn." : " руб.")
            };

            ContainerPc.Children.Add(GenerateComputerInfoBorder(array.ToArray(),
row, column));

            if (column + 1 > 2)
            {
                column = 0;
                row++;
            }
            else
            {
                column++;
            }
        }
    });
}
catch (Exception ex)
{
    Debug.WriteLine("RefreshComputersAsync error: " +
ex.Message);
}

private void generatePCfromFile()
{
    // начальная синхронная загрузка при старте; при больших
объёмах лучше вызвать RefreshComputersAsync
    List<Computer> list =
database.ReadElementsInTableComputers();

    int column = 0;
    int row = 0;

    foreach (Computer item in list)
    {
        List<string> array = new List<string>

```

```

        {
            item.Name,
            item.Description,
            item.Cpu,
            item.Gpu,
            item.Ram,
            item.Storage,
            item.Monitor,
            Math.Round(item.Rating, 2).ToString(),
            item.PricePerHour.ToString() + (Lang.lang == "en"
? " byn." : " руб.")
        };

ContainerPc.Children.Add(GenerateComputerInfoBorder(array.ToArray(),
row, column));

        if (column + 1 > 2)
        {
            column = 0;
            row++;
        }
        else
        {
            column++;
        }
    }

    private void ResearchPCInformation_TextChanged(object sender,
TextChangedEventArgs e)
    {
        // debounce: отменяем предыдущую задачу и запускаем новое
        обновление через 350ms
        try
        {
            _searchCts?.Cancel();
            _searchCts?.Dispose();
        }
        catch { }

        _searchCts = new CancellationTokenSource();
        var token = _searchCts.Token;

        Task.Run(async () =>
        {
            try
            {
                await Task.Delay(350, token);
                if (token.IsCancellationRequested) return;
                await RefreshComputersAsync();
            }
            catch (TaskCanceledException) { /* ожидалось */ }
            catch (Exception ex)
            {

```

```

        Debug.WriteLine("Research  debounce  error:  " +
ex.Message);
    }
    }, token);
}

// ... остальная часть файла без изменений (OnComputerClicked,
GenerateComputerInfoBorder, локализация и класс Music) ...

private void OnComputerClicked(object sender, RoutedEventArgs
e)
{
    if (Global.CurrentUser != "Пользователь" &&
Global.CurrentUser != "User")
    {
        string computerId = e.Source.ToString();
        var border = e.OriginalSource as Border;
        string computerIdFromTag = border?.Tag?.ToString();
        var window
Application.Current.Windows.OfType<mapbooking>().FirstOrDefault();

        if (window == null)
        {
            window = new mapbooking(computerIdFromTag);
            window.Show();
        }
        else
        {
            window.Activate();
            if (window.WindowState == WindowState.Minimized)
                window.WindowState = WindowState.Normal;
        }

        e.Handled = true;
    }
}

public Border GenerateComputerInfoBorder(string[] texts, int
row, int column)
{
    Border border = new Border
    {
        Margin = new Thickness(10),
        Background = new
SolidColorBrush((Color)ColorConverter.ConvertFromString("#8001464b"))
,
        BorderBrush = new
SolidColorBrush((Color)ColorConverter.ConvertFromString("#8001464b"))
,
        BorderThickness = new Thickness(1),
        Cursor = Cursors.Hand,
        CornerRadius = new CornerRadius(10),
        Tag = texts[0]
    };
}

```

```

border.MouseDown += (sender, e) =>
{
    var clickedBorder = sender as Border;
    if (clickedBorder != null)
    {
        var args = new
RoutedEventArgs(ComputerClickedEvent, clickedBorder)
        {
            Source = clickedBorder.Tag
        };
        clickedBorder.RaiseEvent(args);
        e.Handled = true;
    }
};

StackPanel stackPanel = new StackPanel
{
    Orientation = Orientation.Vertical,
    Background = Brushes.Transparent
};
int counterItem = -1;
foreach (string text in texts)
{
    counterItem++;
    TextBlock textBlock = new TextBlock
    {
        TextAlignment = TextAlignment.Center,
        FontSize = 20,
        Foreground = Brushes.White,
        Margin = new Thickness(2),
        TextWrapping = TextWrapping.Wrap,
        HorizontalAlignment = HorizontalAlignment.Stretch
    };
    if (counterItem == 1 &&
text.ToLower().Contains("компьютер"))
    {
        string dynamicText =
Application.Current.FindResource("PC") as string;
        string combinedText = $"{dynamicText}";
        textBlock.Text = combinedText;
    }
    else if (counterItem == 2)
    {
        string dynamicText =
Application.Current.FindResource("cpu") as string;
        string combinedText = $"{dynamicText}: {text}";
        textBlock.Text = combinedText;
    }
    else if (counterItem == 3)
    {
        string dynamicText =
Application.Current.FindResource("gpu") as string;
        string combinedText = $"{dynamicText}: {text}";
    }
}

```

```

        textBlock.Text = combinedText;
    }
    else if (counterItem == 4)
    {
        string dynamicText =
Application.Current.FindResource("ram") as string;
        string combinedText = $"{dynamicText}: {text} +
(Lang.lang == "en" ? "Gb" : "Гб)"}";
        textBlock.Text = combinedText;
    }
    else if (counterItem == 5)
    {
        string dynamicText =
Application.Current.FindResource("storage") as string;
        string combinedText = $"{dynamicText}: {text}";
        textBlock.Text = combinedText;
    }
    else if (counterItem == 6)
    {
        string dynamicText =
Application.Current.FindResource("monitor") as string;
        string combinedText = $"{dynamicText}: {text} +
(Lang.lang == "en" ? "Hz" : "Гц)"}";
        textBlock.Text = combinedText;
    }
    else if (counterItem == 7)
    {
        string dynamicText =
Application.Current.FindResource("rating") as string;
        string combinedText = $"{dynamicText}:
{Math.Round(double.Parse(text), 2)}";
        textBlock.Text = combinedText;
    }
    else
    {
        textBlock.Text = text;
    }
    stackPanel.Children.Add(textBlock);
}

border.Child = stackPanel;

Grid.SetRow(border, row);
Grid.SetColumn(border, column);

return border;
}

private void SwitchToRussian(object sender, EventArgs e)
{
    LocalizationManager.SwitchLanguage("ru-RU");
    Lang.lang = "ru";
}

private void SwitchToEnglish(object sender, EventArgs e)

```

```

    {
        LocalizationManager.SwitchLanguage("en-US");
        Lang.lang = "en";
    }
    public void SwitchMusicStatus(object sender, RoutedEventArgs
e)
    {
        Music.ToggleMusic();
    }

}

static public class Music
{
    static public MediaPlayer player = new MediaPlayer();
    static public bool isPlay = true;

    static public void PlayMusic()
    {
        try
        {
            try
            {
                player.Stop();
                player.Close();
            }
            catch { }

            string relativePath = @"sounds\music.mp3";
            string fullPath
System.IO.Path.GetFullPath(relativePath);

            if (!File.Exists(fullPath))
            {
                MessageBox.Show(Lang.lang == "en"
                    ? $"File not found!\nLook here: {fullPath}"
                    : $"Файл не найден!\nИщите здесь:
{fullPath}");

                return;
            }

            player.MediaEnded -= LoopMusic;
            player.MediaFailed -= OnMediaFailed;

            player.Open(new Uri(fullPath, UriKind.Absolute));
            player.Volume = 0.1;
            player.MediaEnded += LoopMusic;
            player.MediaFailed += OnMediaFailed;

            player.Play();
            isPlay = true;
        }
        catch (Exception ex)
        {

```

```

        MessageBox.Show(Lang.lang == "en"
            ? $"Initialization error: {ex.Message}"
            : $"Ошибка инициализации: {ex.Message}");
    }
}

private static void LoopMusic(object sender, EventArgs e)
{
    player.Position = TimeSpan.Zero;
    player.Play();
}

private static void OnMediaFailed(object sender,
ExceptionEventArgs e)
{
    MessageBox.Show(Lang.lang == "en"
        ? $"Media error: {e.Exception?.Message ??
"Unknown"}"
        : $"Ошибка медиа: {e.Exception?.Message ??
"Неизвестная ошибка}");
}

static public void ToggleMusic()
{
    if (isPlay)
    {
        player.Stop();
        isPlay = false;
    }
    else
    {
        player.Play();
        isPlay = true;
    }
}
static public void SetValue(int value)
{
    player.Volume = value / 100f;
}
}
}

```

mapbooking.xaml

```

<Window x:Class="COURSEPROJECT.mapbooking"

xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006"
    xmlns:local="clr-namespace:COURSEPROJECT"
    mc:Ignorable="d"

```

```

        Title="{DynamicResource      Seatreservations}"      Height="600"
Width="800"
        WindowStartupLocation="CenterScreen">
        <Grid Style="{DynamicResource GridStyle}">
            <Canvas      x:Name="ClubMap"      Width="800"      Height="600"
Style="{DynamicResource CanvasStyle}">
                <Rectangle Canvas.Left="50" Canvas.Top="50" Width="700"
Height="500"
                    Stroke="White" StrokeThickness="3" Fill="Transparent"/>
            </Canvas>
        </Grid>
    </Window>

```

mapbooking.xaml.cs

```

using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Runtime.InteropServices;
using System.Security.Cryptography;
using System.Text;
using System.Text.Json;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Shapes;
using System.Windows.Threading;
using static System.Net.Mime.MediaTypeNames;

namespace COURSEPROJECT
{
    static public class StatesBooking
    {
        static public List<string> elem = new List<string>();
        static public int id = -1;

        static public void AddElem(string item)
        {
            elem.Add(item);
            id = elem.Count;
        }
        static public void PrevState(Canvas ClubMap)
        {
            if (id > 0)
            {
                var computerGrids =
ClubMap.Children.OfType<Grid>().ToList();
                foreach (var grid in computerGrids)

```

```

        {
            var rectangles =
grid.Children.OfType<Rectangle>().ToList();
            var textBlocks =
grid.Children.OfType<TextBlock>().ToList();
            if (textBlocks[0].Text == elem[id])
            {
                rectangles[0].Fill = (Brush)new
BrushConverter().ConvertFrom("#FF695AC3");
                id--;
            }
        }
    }
}
static public void NextState(Canvas ClubMap)
{
    if (id < elem.Count - 1)
    {
        var computerGrids =
ClubMap.Children.OfType<Grid>().ToList();
        foreach (var grid in computerGrids)
        {
            var rectangles =
grid.Children.OfType<Rectangle>().ToList();
            var textBlocks =
grid.Children.OfType<TextBlock>().ToList();
            if (textBlocks[0].Text == elem[id])
            {
                rectangles[0].Fill = (Brush)new
BrushConverter().ConvertFrom("#FF62c259");
                id++;
            }
        }
    }
}

public partial class mapbooking : Window
{
    public DispatcherTimer timer;
    private string _usercompid;
    public mapbooking(string usercompid = null)
    {
        InitializeComponent();
        _usercompid = usercompid;
        try
        {
            this.Cursor = new
Cursor(System.Windows.Application.GetResourceStream(new
Uri("/images/cursor.cur", UriKind.Relative)).Stream);
        }
        catch (Exception ex)
        {
            MessageBox.Show(Lang.lang == "en"

```

```

        ? $"Cursor loading error: {ex.Message}"
        : $"Ошибка загрузки курсора: {ex.Message}");
    }

    timer = new DispatcherTimer();
    timer.Interval = TimeSpan.FromSeconds(.1);
    timer.Tick += Timer_Tick;
    timer.Start();
}
private void Timer_Tick(object sender, EventArgs e)
{
    GenerateComputers(_usercompid);
}
private void GenerateComputers(string usercompid = null)
{
    var positions = new[]
    {
        new Point(67, 64),
        new Point(150, 64),
        new Point(236, 64),
        new Point(317, 64),
        new Point(400, 64),
        new Point(584, 64),
        new Point(666, 64),
        new Point(67, 240),
        new Point(150, 240),
        new Point(236, 240),
        new Point(317, 240),
        new Point(400, 240),
        new Point(584, 240),
        new Point(666, 240),
        new Point(400, 471),
        new Point(495, 471),
        new Point(584, 471),
        new Point(666, 471)
    };

    List<Computer> computers =
database.ReadElementsInTableComputers()
    .OrderBy(c => c.Id)
    .ToList();

    string user = Global.CurrentUser;
    string userid = database.GetUserIdByName(user);
    List<Order> userOrders = new List<Order>();
    List<Order> allOrders =
database.ReadElementsInTableOrders();

    if (!string.IsNullOrEmpty(userid))
    {
        userOrders = allOrders
            .Where(x => x.UserId == int.Parse(userid))
            .ToList();
    }
}

```

```

        for (int i = 0; i < positions.Length && i < computers.Count;
i++)
        {
            var computer = computers[i];
            var pos = positions[i];

            var grid = new Grid
            {
                Width = 60,
                Height = 60,
                Tag = computer.Id
            };

            Shape element = new Rectangle
            {
                Fill = (Brush)new
BrushConverter().ConvertFrom("#FF695AC3"),
                Stroke = (Brush)new
BrushConverter().ConvertFrom("#FF000000"),
                StrokeThickness = 2
            };

            element.MouseDown += (sender, e) =>
            {
                var window =
System.Windows.Application.Current.Windows.OfType<Item>().FirstOrDefault();
                if (window == null)
                {
                    window = new Item(computer);
                    window.Show();
                }
                else
                {
                    window.Activate();
                    if (window.WindowState ==
WindowState.Minimized)
                        window.WindowState = WindowState.Normal;
                }
            };

            var textBlock = new TextBlock
            {
                Text = computer.Id.ToString(),
                Foreground = Brushes.White,
                FontWeight = FontWeights.Bold,
                HorizontalAlignment = HorizontalAlignment.Center,
                VerticalAlignment = VerticalAlignment.Center
            };

            var userActiveOrder = userOrders.FirstOrDefault(o =>
                o.ComputerId == computer.Id &&

```

```

        o.Status == "active");

    if (userActiveOrder != null)
    {
        bool isTime = IsCurrentTimeInRange(
            userActiveOrder.StartTime,
            userActiveOrder.EndTime,
            DateTime.Parse(userActiveOrder.DateOrder));

        element.Fill = isTime
            ? (Brush)new
BrushConverter().ConvertFrom("#1ea855")
            : (Brush)new
BrushConverter().ConvertFrom("#d63176");
    }
    else
    {
        var otherActiveOrder = allOrders.FirstOrDefault(o
=>
            o.ComputerId == computer.Id &&
            o.Status == "active" &&
            o.UserId.ToString() != userid);

        if (otherActiveOrder != null)
        {
            bool isTime = IsCurrentTimeInRange(
                otherActiveOrder.StartTime,
                otherActiveOrder.EndTime,
                DateTime.Parse(otherActiveOrder.DateOrder));

            if (isTime)
            {
                element.Fill = (Brush)new
BrushConverter().ConvertFrom("#b2b51b");
            }
        }

        if (usercompid != null && computer.Name == usercompid)
        {
            element.Fill = (Brush)new
BrushConverter().ConvertFrom("#25baba");
        }

        grid.Children.Add(element);
        grid.Children.Add(textBlock);

        Canvas.SetLeft(grid, pos.X);
        Canvas.SetTop(grid, pos.Y);
        ClubMap.Children.Add(grid);
    }

```

```

    }
    public bool IsCurrentTimeInRange(string startTimeStr, string
endTimeStr, DateTime dateOrder)
    {
        DateTime currentDateTime = DateTime.Now;
        DateTime startDateTime = dateOrder.Date +
TimeSpan.Parse(startTimeStr);
        DateTime endDateTime = dateOrder.Date +
TimeSpan.Parse(endTimeStr);

        if (TimeSpan.Parse(startTimeStr) >
TimeSpan.Parse(endTimeStr))
        {
            endDateTime = endDateTime.AddDays(1);
        }

        return currentDateTime >= startDateTime && currentDateTime
<= endDateTime;
    }
}
}

```

PersonalAccount.xaml

```

<Window x:Class="COURSEPROJECT.PersonalAccount"

xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006"
    xmlns:local="clr-namespace:COURSEPROJECT"
    mc:Ignorable="d"
    Title="{DynamicResource Personalaccount}" Height="500"
Width="600"
    WindowStartupLocation="CenterScreen">

    <Grid Style="{DynamicResource GridAuthorization}">
        <Border Style="{DynamicResource PersonalBorderStyle}">
            <StackPanel Style="{DynamicResource
AuthorizationStackPanelStyle}">
                <TextBlock Style="{DynamicResource
AuthorizationTextBlockStyle}"
                Text="{DynamicResource
Personalaccount}"/>
                <TextBlock Style="{DynamicResource
AuthorizationLabelTextBlockStyle}" Text="{DynamicResource Login}"/>
                <TextBlock Style="{DynamicResource
AuthorizationLabelTextBlockStyle}" x:Name="LoginText" Text=""/>
                <!--<TextBlock Style="{DynamicResource
AuthorizationLabelTextBlockStyle}" Text="{DynamicResource Password}"/>
                <TextBlock Style="{DynamicResource
AuthorizationLabelTextBlockStyle}" x:Name="PasswordText" Text=""/>-->
            </StackPanel>
        </Border>
    </Grid>

```

```

                <TextBlock
AuthorizationLabelTextBlockStyle"
Numberofcompletedorders"/>
                <TextBlock
AuthorizationLabelTextBlockStyle"
Text=""/>
                <TextBlock
AuthorizationLabelTextBlockStyle"
Totaltimeofcompletedorders"/>
                <TextBlock
AuthorizationLabelTextBlockStyle" x:Name="allTime" Text=""/>
                <WrapPanel x:Name="itemsuser" Orientation="Vertical">

                </WrapPanel>
                <Button x:Name="GetStat" Click="GenerateStatsAdmin"
Content="{DynamicResource SystemStatistics}" Visibility="Collapsed"
Style="{DynamicResource AuthorizationButtonStyleTrigger}"/>
                <TextBlock
AuthorizationLabelTextBlockStyle"
Changepassword"/>
                <local:CustomPasswordBox VerticalAlignment="Center"
HorizontalAlignment="Center" x:Name="Password"/>
                <Border Margin="0 10 0 0">
                    <Button
AuthorizationButtonStyleTrigger"
Savechanges" Command="{Binding SaveInfoCommand}"/>
                </Border>
            </StackPanel>
        </Border>
    </Grid>
</Window>

```

PersonalAccount.xaml.cs

```

using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Text;
using System.Text.Json;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Shapes;
using static System.Net.Mime.MediaTypeNames;
using System.Diagnostics;

namespace COURSEPROJECT
{

```

```

/// <summary>
/// Логика взаимодействия для PersonalAccount.xaml
/// </summary>
public partial class PersonalAccount : Window
{
    public ICommand SaveInfoCommand { get; }

    // сохраняем делегат для корректной отписки
    private readonly Action _languageChangedHandler;

    public PersonalAccount()
    {
        InitializeComponent();
        DataContext = this;
        SaveInfoCommand = new RelayCommand(SaveInfo);
        try
        {
            this.Cursor = new
Cursor(System.Windows.Application.GetResourceStream(new
Uri("/images/cursor.cur", UriKind.Relative)).Stream);
        }
        catch (Exception ex)
        {
            MessageBox.Show(Lang.lang == "en"
                ? $"Cursor loading error: {ex.Message}"
                : $"Ошибка загрузки курсора: {ex.Message}");
        }
        _ = GenerateInfoAsync();
        if (Global.CurrentUser == "admin")
        {
            GetStat.Visibility = Visibility.Visible;
        }

        _languageChangedHandler = async () => await
GenerateInfoAsync();
        Lang.LanguageChanged += _languageChangedHandler;
        this.Closed += (s, e) => Lang.LanguageChanged -=
_languageChangedHandler;
    }
    public bool IsTimeInRange(TimeSpan startTime, TimeSpan
endTime)
    {
        TimeSpan currentTime = DateTime.Now.TimeOfDay;

        if (startTime <= endTime)
        {
            return currentTime >= startTime && currentTime <=
endTime;
        }
        else
        {
            return currentTime >= startTime || currentTime <=
endTime;
        }
    }
}

```

```

    }

    // Асинхронная версия GenerateInfo
    public async Task GenerateInfoAsync()
    {
        try
        {
            List<string> info =
            database.GetInfoAboutUser(Global.CurrentUser);
            LoginText.Text = info.Count > 1 ? info[1] :
            string.Empty;

            var jsonList = await
            database.ReadElementsInTableOrdersAsync();
            if (int.TryParse(info.FirstOrDefault(), out int
            userId))
            {
                jsonList = jsonList.Where(x => x.UserId ==
            userId).ToList();
            }
            else
            {
                jsonList = new List<Order>();
            }

            int countActive = 0;
            int totalMinutes = 0;

            // Обновляем UI на UI-потоке
            await Dispatcher.InvokeAsync(async () =>
            {
                itemsuser.Children.Clear();
                foreach (Order item in jsonList)
                {
                    if (item.Status == "active")
                    {
                        WrapPanel wrap = new WrapPanel();
                        TextBlock text = new TextBlock();
                        text.TextWrapping = TextWrapping.Wrap;
                        text.Text = Lang.lang == "en"
                        ? $"Computer: {item.ComputerId}, Date:
            {item.DateOrder}, Status: {item.Status}, Time: {item.StartTime} -
            {item.EndTime}"
                        : $"Компьютер: {item.ComputerId},
            Дата: {item.DateOrder}, Статус: {item.Status}, Время: {item.StartTime}
            - {item.EndTime}";

                        wrap.Children.Add(text);
                        Button btnDel = new Button();
                        btnDel.Content = Lang.lang == "en" ?
            "Delete record" : "Удалить запись";

                        btnDel.Click += async (sender, args) =>
                        {

```

```

                                if
(!IsTimeInRange(TimeSpan.Parse(item.StartTime),
TimeSpan.Parse(item.EndTime)))
    {
        if
(TimeSpan.TryParse(item.StartTime, out TimeSpan start) &&
TimeSpan.TryParse(item.EndTime, out TimeSpan end))
        {
            var comps = await
database.ReadElementsInTableComputersAsync();
            Computer comp =
comps.FirstOrDefault(x => x.Id == item.ComputerId);
            double totalCost = (end -
start).TotalHours * comp.PricePerHour;
            double roundedCost =
Math.Round(totalCost, 2);

            string currency = Lang.lang ==
"en" ? "byn." : "руб.";
            string message = roundedCost
>= 0
?
$"{roundedCost}{currency} {(Lang.lang == "en" ? "returned for order" :
"вернулось за заказ)}"
:
$"{roundedCost}{currency}
{(Lang.lang == "en" ? "returned for order" : "вернулось за заказ)}}.";
            MessageBox.Show(message);
        }
        else
        {
            string currency = Lang.lang ==
"en" ? "byn." : "руб.";
            MessageBox.Show($"{roundedCost}{currency}
{(Lang.lang == "en" ? "returned for order" : "вернулось за заказ)}}.");
        }

        item.Status = "cancelled";
        await
database.UpdateElementInTableOrders(item.Id, item.UserId,
item.ComputerId, item.Status, item.DateOrder, item.StartTime,
item.EndTime);

        // Регенерируем UI
        await GenerateInfoAsync();
    }
    else
    {
        MessageBox.Show(Lang.lang == "en"
? "Cannot cancel an order that
has already started"
: "Нельзя отменить заказ,
время которого уже наступило");
    }

```

```

        };

        wrap.Children.Add(btnDel);
        itemsuser.Children.Add(wrap);
    }

    if (item.Status == "completed")
    {
        countActive++;
        if (TimeSpan.TryParse(item.StartTime, out
TimeSpan startTime) &&
TimeSpan.TryParse(item.EndTime, out
TimeSpan endTime))
        {
            totalMinutes += (int)(endTime -
startTime).TotalMinutes;
        }
    }

    CountOrdersActive.Text = countActive.ToString();
    allTime.Text = totalMinutes.ToString();
});
}
catch (Exception ex)
{
    Debug.WriteLine("GenerateInfoAsync error: " +
ex.Message);
}

public void SaveInfo(object parametr)
{
    if (string.IsNullOrEmpty>Password.Password)
    {
        MessageBox.Show(Lang.lang == "en"
            ? "Password cannot be empty"
            : "Пароль не может быть пустым");
        return;
    }

    if (Password.Password.Length < 8 ||
Password.Password.Length > 100)
    {
        MessageBox.Show(Lang.lang == "en"
            ? "Password must be between 8 and 99 characters"
            : "Пароль должен содержать от 8 до 99 символов");
        return;
    }

    database.SaveChanges(LoginText.Text, Password.Password);
    this.Close();
}

```

```

private async void GenerateStatsAdmin(object sender, EventArgs
ev)
{
    if (Global.CurrentUser == "admin")
    {
        try
        {
            var usersTask =
database.ReadElementsInTableUsersAsync();
            var ordersTask =
database.ReadElementsInTableOrdersAsync();
            var reviewsTask =
database.ReadElementsInTableReviewsAsync();

            await Task.WhenAll(usersTask, ordersTask,
reviewsTask);

            int allusers = usersTask.Result.Count;
            int allorders = ordersTask.Result.Count;
            int allreviews = reviewsTask.Result.Count;

            string message = Lang.lang == "en"
                ? $"=== System Statistics ===\n" +
                  $"Total users: {allusers}\n" +
                  $"Total orders: {allorders}\n" +
                  $"Total reviews: {allreviews}"
                : $"=== Статистика системы ===\n" +
                  $"Всего пользователей: {allusers}\n" +
                  $"Всего заказов: {allorders}\n" +
                  $"Всего отзывов: {allreviews}";

            MessageBox.Show(message,
                Lang.lang == "en" ? "System Statistics" :
"Статистика системы",
                MessageBoxButton.OK,
                MessageBoxImage.Information);
        }
        catch (Exception ex)
        {
            MessageBox.Show(Lang.lang == "en"
                ? $"Error generating statistics: {ex.Message}"
                : $"Ошибка при формировании статистики:
{ex.Message}",
                Lang.lang == "en" ? "Error" : "Ошибка",
                MessageBoxButton.OK,
                MessageBoxImage.Error);
        }
    }
}

```

registration.xaml

```

<Window x:Class="COURSEPROJECT.registration"

xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006"
    xmlns:local="clr-namespace:COURSEPROJECT"
    mc:Ignorable="d"
    Title="{DynamicResource      Registration}"      Height="300"
Width="400"
    WindowStartupLocation="CenterScreen">

    <Grid Style="{DynamicResource GridAuthorization}">
        <Border Style="{DynamicResource AuthorizationBorderStyle}">
            <StackPanel
AuthorizationStackPanelStyle}">
                <TextBlock
AuthorizationTextBlockStyle}" Text="{DynamicResource Registration}"/>
                <TextBlock
AuthorizationLabelTextBlockStyle}" Text="{DynamicResource Login}"/>
                <Border
AuthorizationInputBorderBorderStyle}">
                    <TextBox
AuthorizationTextBoxStyle}" x:Name="Login"/>
                    </Border>
                    <TextBlock
AuthorizationLabelTextBlockStyle}" Text="{DynamicResource Password}"/>
                    <Border
AuthorizationInputBorderBorderStyle}">
                        <PasswordBox
AuthorizationPasswordBoxStyle}" x:Name="Password1"/>
                        </Border>
                        <TextBlock
AuthorizationLabelTextBlockStyle}"
Repeatatthepassword}"/>
                        <Border
AuthorizationInputBorderBorderStyle}">
                            <PasswordBox
AuthorizationPasswordBoxStyle}" x:Name="Password2"/>
                            </Border>
                            <Border>
                                <Button
AuthorizationButtonStyleTrigger}"
Registration}" Command="{Binding AddNewUserCommand}"/>
                                </Border>
                            </StackPanel>
                        </Border>
                    </Grid>
                </Window>

```

registration.xaml.cs

```

using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Shapes;

namespace COURSEPROJECT
{
    /// <summary>
    /// Interaction logic for registration.xaml
    /// </summary>
    public partial class registration : Window
    {
        public ICommand AddNewUserCommand { get; }
        public registration()
        {
            InitializeComponent();
            DataContext = this;

            AddNewUserCommand = new RelayCommand(async (p) => await
AddNewUserAsync());
            try
            {
                this.Cursor = new
Cursor(Application.GetResourceStream(new Uri("/images/cursor.cur",
UriKind.Relative)).Stream);
            }
            catch (Exception ex)
            {
                MessageBox.Show(Lang.lang == "en"
? $"Cursor loading error: {ex.Message}"
: $"Ошибка загрузки курсора: {ex.Message}");
            }
        }

        private static readonly HashSet<string> ForbiddenUsernames =
new HashSet<string>(StringComparer.OrdinalIgnoreCase)
{
    "user", "пользователь",
    "admin", "админ",
    "administrator", "администратор"
};

```

```

public async Task AddNewUserAsync()
{
    if (ForbiddenUsernames.Contains(Login.Text.Trim()))
    {
        MessageBox.Show(Lang.lang == "en"
            ? "This is a reserved name and cannot be used"
            : "Это зарезервированное имя, его нельзя
использовать");
        return;
    }

    if (string.IsNullOrEmpty(Login.Text) ||
string.IsNullOrEmpty>Password1.Password))
    {
        MessageBox.Show(Lang.lang == "en"
            ? "Login and password cannot be empty"
            : "Логин и пароль не могут быть пустыми");
        return;
    }

    if (Login.Text.Length < 5 || Login.Text.Length > 100)
    {
        MessageBox.Show(Lang.lang == "en"
            ? "Login must contain between 5 and 99 characters"
            : "Логин должен содержать от 5 до 99 символов");
        return;
    }

    if (Password1.Password.Length < 8 ||
Password1.Password.Length > 100)
    {
        MessageBox.Show(Lang.lang == "en"
            ? "Password must contain between 8 and 99
characters"
            : "Пароль должен содержать от 8 до 99 символов");
        return;
    }

    if (Password1.Password != Password2.Password)
    {
        MessageBox.Show(Lang.lang == "en"
            ? "Passwords do not match"
            : "Пароли не совпадают");
        return;
    }

    var users = await
database.ReadElementsInTableUsersAsync();
    if (users.Any(u => u.Name.Equals(Login.Text,
StringComparison.OrdinalIgnoreCase)))
    {
        MessageBox.Show(Lang.lang == "en" ? "User already
exists" : "Пользователь с таким именем уже существует");
        return;
    }
}

```

```

    }

    try
    {
        await database.AddElementInTableUsers(Login.Text,
Password1.Password);
        // Возвращаемся на UI-поток для закрытия окна
        await Dispatcher.InvokeAsync(() =>
        {
            MessageBox.Show(Lang.lang == "en" ? "Registration
successful" : "Регистрация прошла успешно");
            this.Close();
        });
    }
    catch (Exception ex)
    {
        Debug.WriteLine("AddNewUserAsync error: " +
ex.Message);
        await Dispatcher.InvokeAsync(() =>
        {
            MessageBox.Show(Lang.lang == "en" ? $"Error:
{ex.Message}" : $"Ошибка: {ex.Message}");
        });
    }
}
}
}

```

WritingReview.xaml

```

<Window x:Class="COURSEPROJECT.WritingReview"

xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006"
xmlns:local="clr-namespace:COURSEPROJECT"
mc:Ignorable="d"

xmlns:materialDesign="http://materialdesigninxaml.net/winfx/xaml/them
es"

    WindowStartupLocation="CenterScreen"
    Title="{DynamicResource UserFeedback}" Height="450"
Width="800">
    <Grid Style="{DynamicResource WritingReviewFon}">
        <StackPanel HorizontalAlignment="Center"
VerticalAlignment="Center" Orientation="Vertical">
            <StackPanel Orientation="Horizontal">
                <TextBlock FontFamily="Arial Black"
Text="{DynamicResource Makinganorderreview}"/>
                <TextBlock Margin="10 0 0 0" FontFamily="Arial Black"
Text="{Binding Id}"/>
            </StackPanel>

```

```

        <StackPanel Orientation="Horizontal" Margin="0 10 0 0">
            <TextBlock
                FontFamily="Arial
                Black"
Text="{DynamicResource PC}"/>
            <TextBlock Margin="10 0 0 0" FontFamily="Arial Black"
Text="{Binding ComputerId}"/>
        </StackPanel>
        <StackPanel Orientation="Horizontal" Margin="0 10 0 0">
            <TextBlock
                FontFamily="Arial
                Black"
Text="{DynamicResource Payment}"/>
            <TextBlock Margin="10 0 0 0" FontFamily="Arial Black"
x:Name="cost" />
        </StackPanel>
        <StackPanel Orientation="Horizontal" Margin="0 10 0 0">
            <TextBlock
                FontFamily="Arial
                Black"
Text="{DynamicResource UserFeedback}"/>
            <TextBlock Margin="10 0 0 0" FontFamily="Arial Black"
x:Name="username" Text=""/>
        </StackPanel>
        <StackPanel Orientation="Horizontal" Margin="0 10 0 0">
            <TextBox x:Name="textReview"
                Width="600"
                Height="200"
                TextWrapping="Wrap"
                AcceptsReturn="True"
                VerticalScrollBarVisibility="Auto"/>
        </StackPanel>
        <StackPanel
            Orientation="Vertical"
            MaxWidth="100"
Margin="0 10 0 10">
            <Slider
                x:Name="stats"
                Minimum="0"
                Maximum="5"
TickFrequency="1" IsSnapToTickEnabled="True" />
        </StackPanel>
        <Button
            Click="SaveReview"
            Content="{DynamicResource
Send}"
            Width="90"
            Style="{DynamicResource
AnimationLangBtn}"
Template="{DynamicResource LangButtonTemplate}">

        </Button>
    </StackPanel>
</Grid>
</Window>

```

WritingReview.xaml.cs

```

using Microsoft.Win32;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;

```

```

using System.Windows.Media.Imaging;
using System.Windows.Shapes;
using System.Diagnostics;

namespace COURSEPROJECT
{
    /// <summary>
    /// Interaction logic for WritingReview.xaml
    /// </summary>
    public partial class WritingReview : Window
    {
        private readonly Order _item;
        private readonly Func<Task> _langHandler;

        public WritingReview(Order item)
        {
            InitializeComponent();
            _item = item;
            DataContext = item;
            _ = InitReviewAsync();

            // сохранённый делегат для корректной отписки
            _langHandler = async () => await InitReviewAsync();
            Lang.LanguageChanged += () => _ = _langHandler();
            this.Closed += (s, e) => Lang.LanguageChanged -= () => _ =
            _langHandler();
        }

        private async Task InitReviewAsync()
        {
            try
            {
                username.Text =
                database.GetUserNameById(_item.UserId);

                // безопасный парсинг времени
                TimeSpan startTime = TimeSpan.Zero;
                TimeSpan endTime = TimeSpan.Zero;
                TimeSpan.TryParse(_item.StartTime, out startTime);
                TimeSpan.TryParse(_item.EndTime, out endTime);
                TimeSpan duration = endTime - startTime;
                double durationDouble = Math.Max(0,
                duration.TotalHours);

                // читаем компьютеры асинхронно
                var computers = await
                database.ReadElementsInTableComputersAsync();
                var pc = computers.FirstOrDefault(x => x.Id ==
                _item.ComputerId);

                string costStr = "0" + (Lang.lang == "en" ? " byn." :
                " руб.");
                if (pc != null)
                {

```

```

        double cost = Math.Round((durationDouble *
pc.PricePerHour), 2);
        costStr = $"{cost}{(Lang.lang == "en" ? " byn." :
" руб.")}";
    }

    await Dispatcher.InvokeAsync(() => cost.Text =
costStr);
    }
    catch (Exception ex)
    {
        Debug.WriteLine("InitReviewAsync error: " +
ex.Message);
    }
}

public async void SaveReview(object sender, EventArgs e)
{
    try
    {
        if (string.IsNullOrEmpty(textReview.Text))
        {
            MessageBox.Show(Lang.lang == "en"
                ? "Cannot submit an empty review"
                : "Нельзя оставить пустой отзыв");
            return;
        }

        if (textReview.Text.Trim().Length > 500)
        {
            MessageBox.Show(Lang.lang == "en"
                ? "Review cannot exceed 500 characters"
                : "Отзыв должен содержать не более 500
СИМВОЛОВ");
            return;
        }

        await
database.AddElementsInTableReviews(_item.UserId, _item.ComputerId,
(float)stats.Value, textReview.Text.Trim(),
DateTime.Now.ToString("dd.MM.yyyy"));

        // пересчёт рейтинга в фоне
        var allReviews = await
database.ReadElementsInTableReviewsAsync();
        var reviewsForComputer = allReviews.Where(r =>
r.ComputerId == _item.ComputerId).ToList();

        if (reviewsForComputer.Count > 0)
        {
            float result =
(float)Math.Round(reviewsForComputer.Average(r => r.Rating), 2);

```

```

var computers = await
database.ReadElementsInTableComputersAsync();
var comp = computers.FirstOrDefault(x => x.Id ==
_item.ComputerId);
if (comp != null)
{
    await
database.UpdateElementInTableComputers(comp.Id, comp.Name,
comp.Description, comp.Cpu, comp.Gpu, comp.Ram, comp.Storage,
comp.Monitor, result, (float)comp.PricePerHour, comp.GraphicData);
}

await Dispatcher.InvokeAsync(() =>
{
    MessageBox.Show(Lang.lang == "en"
        ? "Your review has been saved!"
        : "Ваш отзыв сохранен!");
    this.Close();
});
}
catch (Exception ex)
{
    Debug.WriteLine("SaveReview error: " + ex.Message);
    await Dispatcher.InvokeAsync(() =>
    {
        MessageBox.Show(Lang.lang == "en"
            ? $"Error: {ex.Message}"
            : $"Ошибка: {ex.Message}");
    });
}
}
}

```